LEARNING GUIDE · ARCHITECTURE · INTERVIEW PREP

QUIZZY

A complete guide to the real-time, AI-powered quiz app: how it is built, how the pieces talk to each other, and how to explain it in an interview.



Stack: React 19 · Vite · Tailwind CSS v4 · Node.js · Express 5 · Socket.IO 4 · Supabase (Postgres + Auth) · Google Gemini

Hosting: client on Vercel · server on Render · database and auth on Supabase

Written from: the code in the repository at commit `b28a540` (October 2026)

Author of the app: mani759

Contents

How to read this guide 5

The seven callout boxes 5

Diagrams in this guide 6

1. Overview 7

1.1 What QUIZZY does 7

1.2 Two kinds of users 7

1.3 Student-paced design 7

2. Architecture 9

2.1 The production system 9

2.2 What runs where 11

2.3 Layers inside the server 11

3. Concepts and technologies 14

3.1 Web basics 14

3.2 Real-time communication 25

3.3 React and the browser 31

3.4 JavaScript and Node.js 42

3.5 Server design 49

3.6 Databases 58

3.7 Security and authentication 66

3.8 AI 74

3.9 Building and deploying 77

3.10 The technologies QUIZZY uses 83

4. File map 93

4.1 Client files (client/src) 93

4.2 Server files (server/) 96

4.3 Dependency diagrams 97

5. Pages and routes 100

5.1 How a page decides what to show 100

6. How the frontend talks to the backend 104

6.1 REST: apiFetch and the Authorization header 104

6.2 CORS and CLIENT_URL 104

6.3 requireAuth middleware 105

6.4 Every REST endpoint 106

6.5 Socket.IO: two client sockets 106
6.6 Handshake authentication and socket.data 107
6.7 Socket rooms used by QUIZZY 108
6.8 Every Socket.IO event 109

7. Database 113
7.1 Quizzes in the database, live sessions in memory 113
7.2 Tables and columns 114
7.3 Queries: which server function runs what 118
7.4 Postgres functions and transactions 119
7.5 Secret key vs publishable key 120
7.6 Row Level Security (RLS) 121
7.7 Indexes 121

8. Authentication and authorization 122
8.1 Sign-up and login in the browser 122
8.2 AuthContext and RequireAuth 122
8.3 Sessions and JWTs end to end 123
8.4 How hosts are identified 123
8.5 Every authorization rule 124
8.6 Why "not yours" returns 404, not 403 125

9. End-to-end flows 126
9.1 Host signs up and logs in 126
9.2 Host creates a quiz manually 127
9.3 Host creates a quiz with AI (generate → review → save) 128
9.4 Host creates a room; student joins 130
9.5 Quiz starts; students answer; live leaderboard 132
9.6 Host ends the quiz; final ranks; results saved 134
9.7 Host refresh, student disconnect, closing the room 135

10. Deployment 137
10.1 Vercel (client) 137
10.2 Render (server) 137
10.3 Every environment variable 138
10.4 Why the server can't run on Vercel 139
10.5 Continuous deployment from GitHub 139
10.6 HTTPS and wss:// 139
10.7 Free-tier sleep behaviour 140

11. Security measures 141

12. Known limitations and future improvements 144

13. Interview questions with answers 146

13.1 Basic 146

13.2 Intermediate 148

13.3 Advanced 151

13.4 Behavioral 154

14. Issues found while reading the code 155

14.1 Bugs and risks 155

14.2 Inconsistencies 157

14.3 Dead code and leftovers 158

Appendix: quick reference 159

A. The 20 custom socket events at a glance 159

B. Five files to know by heart 159

C. Numbers worth remembering 159

D. Index of terms 160

Page numbers refer to the counter at the bottom of each page. Every entry is a clickable link.

How to read this guide

This guide explains QUIZZY from the top down. Sections 1 and 2 give you the big picture, section 3 defines every technical term and technology, sections 4 to 9 walk through the code in detail, sections 10 to 12 cover deployment, security and limitations, and sections 13 and 14 are for interview practice and for things to fix.

Everything here was written by reading the code as it exists now. When an older note (such as `DECISIONS.md`) or a code comment disagrees with the code, **the code is treated as the truth** and the difference is pointed out. File references are written as `path → function`, for example `server/src/services/room.service.js → submitAnswer`, so you can open the exact place in your editor.

Two small labels mark statements that could not be checked against the repository:

- **Inferred** means the fact is deduced from how the code uses something (for example, a database table the code queries), because the SQL that created it is not in the repository.
- **Assumed** means a hosting dashboard setting (Render or Vercel) that is not stored in the repository, described as the usual setup for this kind of project.

No secret values appear anywhere in this guide. Environment variables are referred to only by their *names*.

Technical terms are all defined in one place, section 3 ("Concepts and technologies"), grouped by topic. Everywhere else, a term with a dotted underline is a link to its definition, and an alphabetical index of terms with page numbers is at the very end.

The seven callout boxes

DEFINITION
A technical term "in one sentence" (plain English) and "in more detail" (technically precise).

REAL-LIFE EXAMPLE
An everyday analogy that makes a concept concrete.

KEY IDEA
The one thing to remember from a section.

WHERE IN THE CODE
The files and functions involved. In section 3 the gray boxes are "Where real apps use it" and "Where in QUIZZY".

COMMON MISTAKE
An error that is easy to make here, including ones that really happened in this project.

SECURITY
What a protection prevents, and what would happen without it.

INTERVIEW TIP
How to explain the topic to an interviewer.

Diagrams in this guide

Most diagrams are flowcharts (boxes and arrows), state diagrams (the states something can be in) or sequence diagrams. The last kind needs a short introduction:

Sequence diagram

DEFINITION

In one sentence: A drawing that shows who sends which message to whom, in time order, from top to bottom.

In more detail: Each participant (browser, server, database) gets a vertical line, and each horizontal arrow is one message, in the order it happens; dashed arrows are replies. It is the clearest way to show "who talks to whom, and when", which is why section 9 uses one for every flow.

REAL-LIFE EXAMPLE

A comic strip of a conversation, read panel by panel from top to bottom.

WHERE REAL APPS USE IT

- Software design documents and API documentation.
- Explaining login and payment flows to new team members.

WHERE IN QUIZZY

Figures 8.1 and 9.1–9.7 in this guide; for example figure 9.5 shows the answer loop between `PlayerPage`, `server/src/sockets/socket.js` and `room.service.js`.

1. Overview

1.1 What QUIZZY does

QUIZZY is a **live quiz app for classrooms**, similar to Kahoot or Wayground (formerly Quizizz). A teacher, called the **host**, creates a multiple-choice quiz, either by typing the questions or by asking an AI to write them. In class, the host opens a **room**, which gets a 6-digit code and a QR code shown on the projector. Students join from their phones with just a nickname, answer the questions, and the host watches a **live leaderboard** update on the big screen. When the host ends the quiz, every student sees their final rank and the results are saved to the database.

Key terms here (defined in section 3): Real-time

1.2 Two kinds of users

	Host (teacher)	Student (player)
Account	Needs an account (email + password, through Supabase Auth)	No account. Only a nickname, typed when joining
Device	Laptop connected to a projector	Phone, usually
Can do	Create quizzes (manually or with AI), open a room, start the quiz, end it, close the room	Join a room, answer questions, see their own score and final rank
Screens	`/host`, `/host/new`, `/host/:quizId`	`/` (enter code), `/join/:code`
Identified by	Their Supabase **user ID** (from a verified login token)	Their **socket ID** (a random ID for one live connection)

1.3 Student-paced design

In Kahoot's classic mode everyone sees the same question at the same time, and the host clicks "next". QUIZZY works differently: it is **student-paced**. Once the host starts the quiz, every student gets question 1 and then moves on *as soon as they answer*. A fast student may be on question 8 while a slower one is on question 3. The host does not control the question flow at all; they only start and end the quiz.

This decision shapes the whole server design:

- Each player has their **own position** in the quiz, stored as `player.currentQuestionIndex`.
- When a student answers, the server sends **only that student** the result and their next question.
- The leaderboard shows not just scores but also **progress** (questions answered out of total), so the host can see who is still working.

REAL-LIFE EXAMPLE

A teacher-paced quiz is like a class reading a book aloud together, one page at a time. A student-paced quiz is like a written exam: everyone gets the same paper, but each person turns the pages at their own speed, and the teacher can glance around to see who is on which page.

KEY IDEA

QUIZZY stores **quizzes permanently in a database** (Supabase Postgres), but runs each **live game in the server's memory**. The live game is fast and temporary; only the final results are written back to the database at the end.

WHERE IN THE CODE

- Per-player progress: `server/src/services/room.service.js → addPlayer` (creates `currentQuestionIndex: 0`) and `→ submitAnswer` (advances it)
- Sending each player their own question: `server/src/sockets/socket.js`, handlers for `quiz:start` and `quiz:answer`
- Progress on the leaderboard: `room.service.js → getLeaderboard` (`answered: player.currentQuestionIndex`)

INTERVIEW TIP

Open with one sentence: "QUIZZY is a student-paced live quiz app: hosts create quizzes, with AI if they like, and students join from their phones with a room code and answer at their own speed while the host watches a live leaderboard." Then say what makes it technically interesting: real-time WebSockets, server-side scoring so students can't cheat, and AI output that is validated before it reaches the user.

2. Architecture

2.1 The production system

QUIZZY is made of two programs and three external services:

- **The client** (`client/`): a React app. Vite builds it into static files (HTML, JavaScript, CSS), and **Vercel** serves those files.
- **The server** (`server/`): a Node.js program running Express (for normal web requests) and Socket.IO (for the live connection). **Render** runs it.
- **Supabase**: the Postgres database (quizzes, questions, saved results) and the authentication service (host accounts and login tokens).
- **Google Gemini**: the AI model that writes quiz questions.

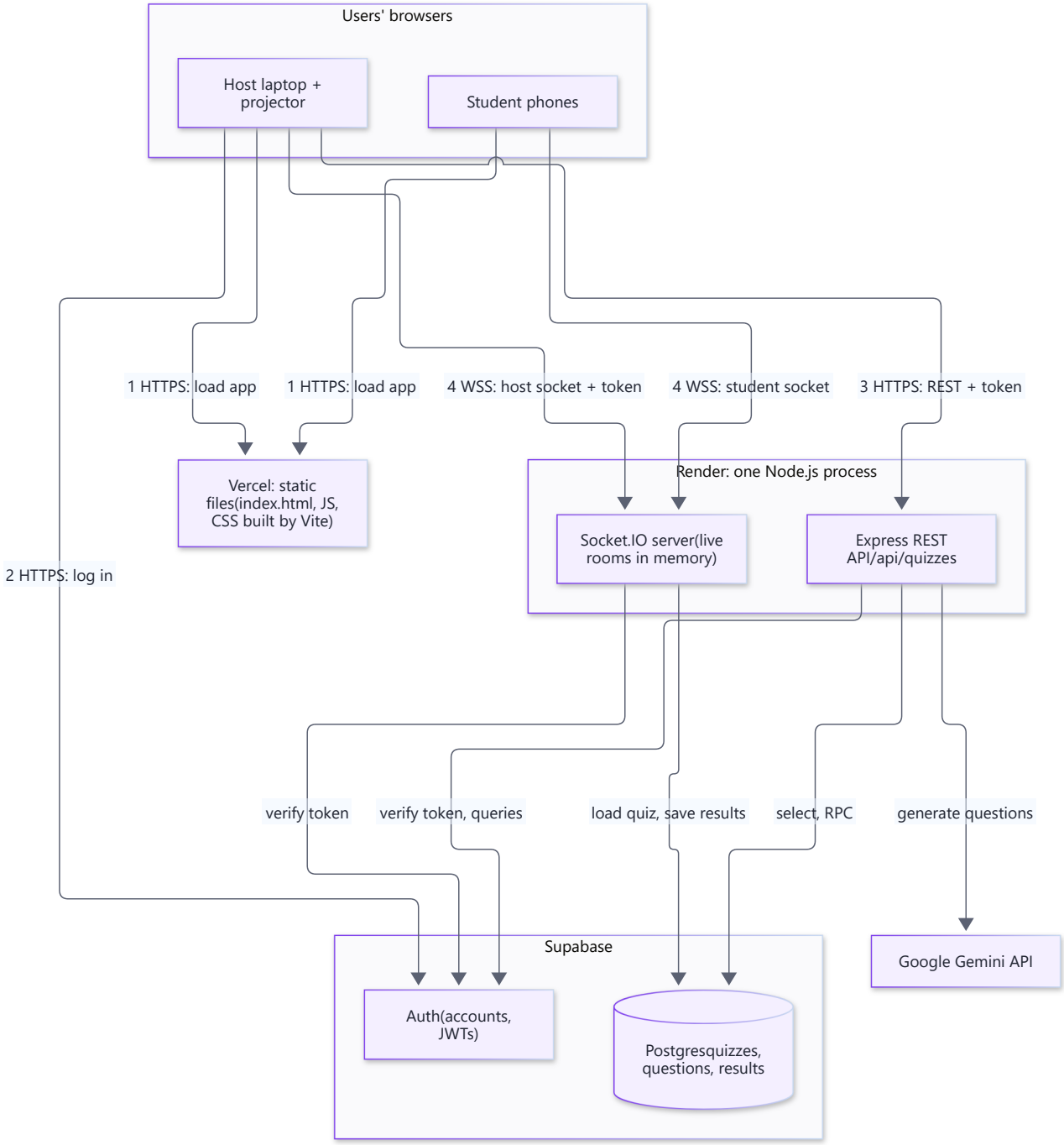


Figure 2.1: Production architecture. Numbers show the usual order of connections.

Key terms here (defined in section 3): Client · Server · HTTPS and TLS · WebSocket · WSS (secure WebSocket) · CDN (Content Delivery Network)

2.2 What runs where

Where	What runs there	State it keeps
Browser (host or student)	The React app: pages, components, the Supabase client (login only), and one Socket.IO client	The host's login session (saved in the browser's local storage by supabase-js); React state for the current screen
Vercel	Nothing runs. It only serves the files produced by `npm run build`, and its rewrite rule sends every URL to `index.html`	None
Render	`server/server.js`: one Node.js process with Express and Socket.IO sharing one HTTP server	**In memory:** the `rooms` Map (every live game) and the AI rate-limit log. Lost on restart
Supabase	Postgres database, Auth service, and two Postgres functions (`create_quiz`, `save_session_results`)	**Permanent:** host accounts, quizzes, questions, saved session results
Gemini	The language model	None (each request is independent)

> **KEY IDEA**
>
> The browser **never talks to the database directly**. It only uses Supabase for logging in. Every quiz read or write goes through the Render server, which checks who you are, validates the data, and only then uses its secret key to reach the database.

2.3 Layers inside the server

The server follows a common layered pattern. Each layer has one job and only calls the layer below it:

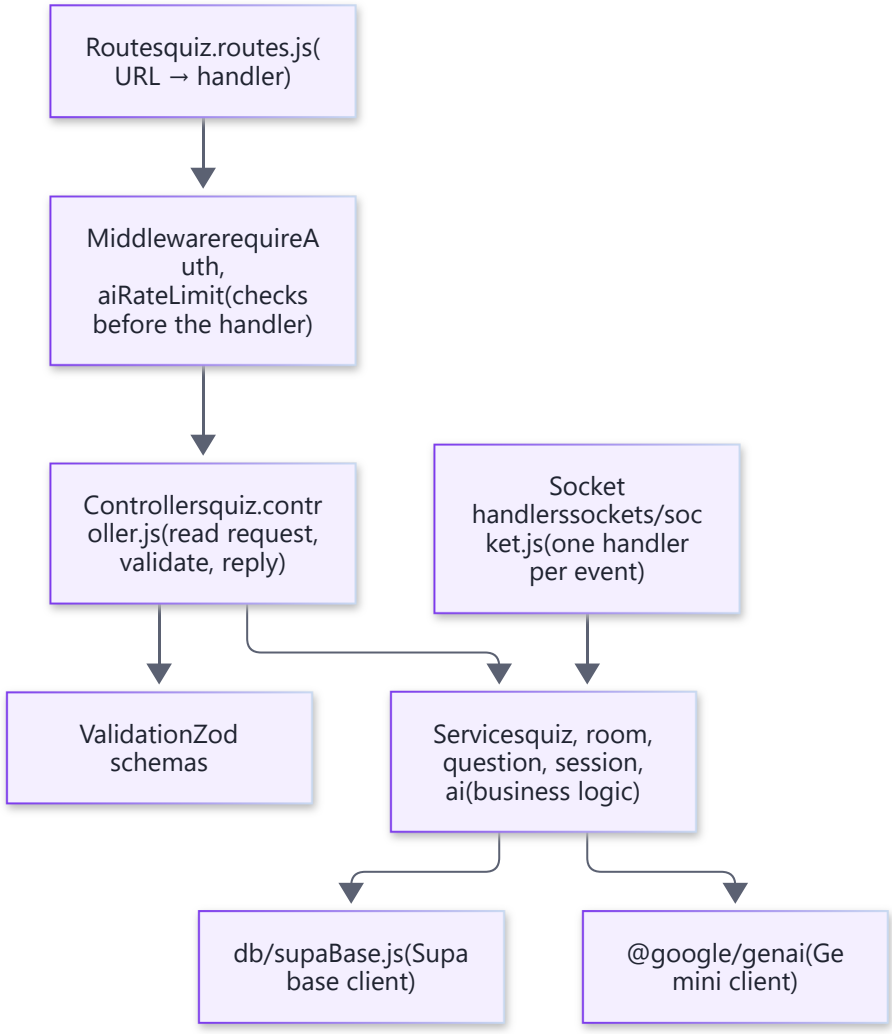


Figure 2.2: Server layers. REST requests enter through routes; live events enter through socket handlers. Both share the same services.

REAL-LIFE EXAMPLE

Think of a restaurant. The **route** is the host stand that sends you to the right table. The **middleware** is the bouncer checking your ID. The **controller** is the waiter who takes your order and checks it makes sense. The **service** is the kitchen that does the real work. The **database** is the pantry.

WHERE IN THE CODE

- `server/server.js`: creates the HTTP server, attaches Express (`src/app.js`) and Socket.IO, and calls `registerSocketHandler(io)`
- `server/src/app.js`: Express setup (CORS, JSON body parsing, routes)
- `client/src/main.jsx` → `App.jsx`: the React entry point and routes

INTERVIEW TIP

Draw figure 2.1 from memory: browser, Vercel, Render, Supabase, Gemini. Then explain the **split**: "Static files on a CDN, a long-running Node server for WebSockets, a managed Postgres for permanent data. I used Render instead of Vercel for the server because Vercel's serverless functions can't hold open WebSocket connections or keep game state in memory."

3. Concepts and technologies

This chapter defines every technical term in the guide, **once**, grouped by topic so you can revise a whole area at a time (for example, all the security terms before an interview). Everywhere else in the guide, a term with a dotted underline is a link back to its definition here. An alphabetical index of all terms is at the end of the guide.

Every term has the same four parts:

- **Definition**: "In one sentence" is the version to say first in an interview; "In more detail" is the accurate follow-up.
- **Real-life example**: an everyday comparison, with a note where it stops being accurate.
- **Where real apps use it**: well-known places you will recognise.
- **Where in QUIZZY**: the exact file and function in this project.

> **INTERVIEW TIP**
> Answer "what is X?" in three moves: the one-sentence definition, one detail that shows you understand how it works, then "in my project, I used it for …". That last step is what makes your answer memorable.

3.1 Web basics

Client

> **DEFINITION**
> **In one sentence:** The program a person actually uses (here, the QUIZZY website open in a browser) that asks a server for data and shows the results on screen.
>
> **In more detail:** In web apps the client is usually a browser running HTML, CSS and JavaScript; a phone app is also a client. It handles what you see and click, and sends requests to a server when it needs data or wants something saved. Because the client runs on the user's own device, the user can inspect or even change it, so a client must never be trusted to enforce rules or security.

> **REAL-LIFE EXAMPLE**
> A customer in a restaurant: they read the menu and place orders, but the kitchen decides what actually gets cooked.

WHERE REAL APPS USE IT

- Web browsers showing Gmail or YouTube: the page you click on is the client.
- Phone apps such as Instagram: the app on your phone is a client of Instagram's servers.

WHERE IN QUIZZY

Everything in `client/`. `client/src/main.jsx` starts the React app in the browser, and pages such as `PlayerPage.jsx` send messages to the server and show what comes back.

Server

DEFINITION

In one sentence: A program that waits for requests from clients, does the work (such as checking a password or saving data) and sends back an answer.

In more detail: A server usually runs on a computer in a data centre and serves many clients at the same time. It holds the things clients must not touch directly (the database, secret keys, the rules of the game) and decides what each client is allowed to do. "Server" can mean the program or the computer it runs on.

REAL-LIFE EXAMPLE

The kitchen of a restaurant: it takes orders from many tables, uses ingredients the customers never touch, and sends back finished dishes.

WHERE REAL APPS USE IT

- Google Search: your question is answered by Google's servers.
- Online multiplayer games: a game server keeps every player's view of the match in sync.

WHERE IN QUIZZY

`server/server.js` starts QUIZZY's server on Render. It answers REST requests (`server/src/app.js`) and runs the live game (`server/src/sockets/socket.js`).

Frontend

DEFINITION

In one sentence: The part of an app that the user sees and interacts with: screens, buttons and forms.

In more detail: Frontend code is about presentation and interaction: layout, styling, reacting to clicks, and asking the backend for data. On the web it is written in HTML, CSS and JavaScript, often with a library such as React, and it runs in the browser. In everyday talk "frontend" and "client" are often used for the same thing; strictly, the frontend is the code and the client is the program running it.

REAL-LIFE EXAMPLE

The dining room of a restaurant: the decor, the menu and the waiters are everything the guests experience.

WHERE REAL APPS USE IT

- The Netflix screens where you browse and pick a film.
- The forms of an online banking website.

WHERE IN QUIZZY

Everything in `client/src`: the pages in `client/src/pages` and the components in `client/src/components`, built with React and Tailwind CSS.

Backend

DEFINITION

In one sentence: The behind-the-scenes part of an app that stores data, applies the rules and talks to other services; users never see it directly.

In more detail: Backend code runs on servers. It usually offers an API to the frontend, checks the incoming data, decides who is allowed to do what, reads and writes the database, and calls outside services such as payments, email or AI. Keeping this logic in the backend means users can't tamper with it.

REAL-LIFE EXAMPLE

The kitchen, storeroom and accounts office of a restaurant together.

WHERE REAL APPS USE IT

- An online shop's backend charging your card and creating the order.
- A ride-hailing app's backend matching riders with nearby drivers.

WHERE IN QUIZZY

Everything in `server/src`: routes, controllers and services. For example, `server/src/services/room.service.js → submitAnswer` decides whether an answer is correct and updates the score.

API (Application Programming Interface)

DEFINITION

In one sentence: A fixed menu of requests one program is allowed to make to another program, with clear rules for what to send and what comes back.

In more detail: An API hides how something works inside and only exposes agreed operations, such as "list my quizzes" or "generate questions". Web APIs are called over the internet using HTTP; code libraries also have APIs (the functions you can call). As long as the API stays the same, the inside can be rebuilt without breaking the programs that use it.

REAL-LIFE EXAMPLE

A restaurant menu: you order "dish 12" without knowing the recipe, and the kitchen can change cooks without changing the menu.

WHERE REAL APPS USE IT

- Weather apps getting forecasts from a weather service's API.
- "Log in with Google" buttons, which use Google's sign-in API.
- Checkout pages that take payments through a payment provider's API.

WHERE IN QUIZZY

QUIZZY's server offers a REST API under `/api/quizzes` (`server/src/routes/quiz.routes.js`), and itself calls Gemini's API (`server/src/services/ai.service.js`) and Supabase's API (`server/src/db/supaBase.js`).

HTTP

DEFINITION

In one sentence: The set of rules browsers and servers use to send each other requests and responses on the web.

In more detail: HTTP (HyperText Transfer Protocol) works as request then response: the client sends a method (such as GET), a URL, headers and sometimes a body; the server replies with a status code, headers and a body. Each request stands on its own, because the server doesn't automatically remember earlier ones (HTTP is "stateless"). That is why a logged-in user's token is sent again with every request.

REAL-LIFE EXAMPLE

Ordering at a counter using a standard order form: every form gets exactly one reply slip, and the clerk doesn't remember your previous forms.

WHERE REAL APPS USE IT

- Every time a browser loads a web page.
- Phone apps fetching your news feed or messages.

WHERE IN QUIZZY

`client/src/api.js → apiFetch` sends HTTP requests with `fetch` to the Express server set up in `server/src/app.js` .

HTTPS and TLS

DEFINITION

In one sentence: HTTP with encryption (scrambling the data so only the intended receiver can read it), so nobody between the browser and the server can read or change what is sent.

In more detail: HTTPS is HTTP sent through TLS (Transport Layer Security). TLS encrypts the data and uses a certificate to prove the server really is the website it claims to be. The padlock in the address bar means HTTPS is in use. Without it, anyone on the same Wi-Fi network could read passwords or login tokens.

REAL-LIFE EXAMPLE

Sending letters in a locked box that only the recipient can open, delivered by a courier who first shows an official ID proving they work for the right company.

WHERE REAL APPS USE IT

- Online banking and shopping sites.
- Every login page; today almost all websites use HTTPS.

WHERE IN QUIZZY

Vercel and Render provide HTTPS automatically. Because `VITE_API_URL` starts with `https://`, `client/src/api.js → apiFetch` and both sockets travel encrypted. TLS is handled by Render's proxy, not by `server/server.js`.

URL

DEFINITION

In one sentence: The full address of something on the web, such as `https://example.com/join/483920`.

In more detail: A URL (Uniform Resource Locator) has parts: the scheme (`https`), the domain (`example.com`), an optional port (`:3000`), the path (`/join/483920`) and sometimes a query string (`?page=2`). The path tells the server, or in a single-page application the router, what is being asked for.

REAL-LIFE EXAMPLE

A postal address: country, city, street and house number narrow down exactly where something is.

WHERE REAL APPS USE IT

- Links you share in chats and emails.
- QR codes on posters and menus, which usually just contain a URL.

WHERE IN QUIZZY

The QR code in the host lobby encodes `joinLink`, built in `client/src/pages/HostPage.jsx` as `window.location.origin + "/join/" + roomCode`; the router then reads the code from the path.

Origin

DEFINITION

In one sentence: The "home address" of a web page: its scheme, domain and port together, for example `https://quizzy.vercel.app`.

In more detail: Two URLs have the same origin only if all three parts match: `http` versus `https`, a different subdomain, or a different port all count as different origins. Browsers use the origin as a security boundary called the same-origin policy: code from one origin can't read data from another origin unless that origin allows it through CORS.

REAL-LIFE EXAMPLE

Surname, street and house number together: two people are "from the same house" only if all three match.

WHERE REAL APPS USE IT

- Being logged in to your bank's site doesn't let another site you open read your account page, because they are different origins.
- `app.example.com` and `api.example.com` are different origins, so the API must allow the app via CORS.

WHERE IN QUIZZY

The Vercel site and the Render server are different origins. The `CLIENT_URL` variable tells `server/src/app.js` and `server/server.js` which origin to trust.

JSON

DEFINITION

In one sentence: A simple text format for sending data between programs, written like `{"title": "Planets", "count": 5}`.

In more detail: JSON (JavaScript Object Notation) can represent objects (named values in curly braces), lists (in square brackets), text, numbers, true/false and null. Almost every programming language can read and write it, so it is the common language of web APIs. In JavaScript, `JSON.stringify` turns data into JSON text and `JSON.parse` turns it back.

REAL-LIFE EXAMPLE

A standard shipping form that every courier in the world understands, whatever language they speak.

WHERE REAL APPS USE IT

- Most web APIs send and receive JSON.
- Configuration files such as `package.json` in JavaScript projects.

WHERE IN QUIZZY

`client/src/api.js → apiFetch` sends request bodies as JSON, `express.json()` in `server/src/app.js` reads them, and `server/src/services/ai.service.js → generateQuestions` turns Gemini's answer into data with `JSON.parse`.

REST

DEFINITION

In one sentence: A common style for designing web APIs in which every kind of thing has its own URL and standard HTTP methods are used to read or change it.

In more detail: REST (Representational State Transfer) organises an API around "resources", such as `/api/quizzes`, and uses HTTP methods as the actions: GET to read, POST to create, PUT or PATCH to update, DELETE to remove. Each request carries everything the server needs (including the login token), and results are reported with standard status codes. It is simple and works with ordinary web tools.

REAL-LIFE EXAMPLE

A library catalogue: every book has a shelf number (the URL), and the same few actions (look, borrow, return) work for every book.

WHERE REAL APPS USE IT

- Public developer APIs such as GitHub's, where for example a list of repositories has its own URL.
- The backends of most phone apps.

WHERE IN QUIZZY

`server/src/routes/quiz.routes.js`: `GET /api/quizzes` lists quizzes, `POST /api/quizzes` creates one, `GET /api/quizzes/:id` reads one.

HTTP methods (GET and POST)

DEFINITION

In one sentence: The verb of a web request: GET means "give me something", POST means "here is something new, please save or process it".

In more detail: A GET request only reads data, must not change anything, has no body, and can be repeated safely. A POST sends data in its body to create something or to trigger an action; sending it twice may create a duplicate. Other methods are PUT or PATCH (update) and DELETE (remove).

REAL-LIFE EXAMPLE

GET is asking a librarian to show you a book; POST is handing in a form to register a new book.

WHERE REAL APPS USE IT

- Opening any web page is a GET request.
- Submitting a sign-up or checkout form is usually a POST request.

WHERE IN QUIZZY

`client/src/pages/QuizListPage.jsx` sends `GET /api/quizzes`; `CreateQuizPage.jsx` → `handleSubmit` sends `POST /api/quizzes`, and `handleGenerate` sends `POST /api/quizzes/generate` (a POST because it sends data and starts work).

HTTP status codes

DEFINITION

In one sentence: A three-digit number in every response that says how the request went: 2xx means success, 4xx means the request had a problem, 5xx means the server had a problem.

In more detail: The client uses the code to decide what to do next: show the result, show an error, send the user to the login page, or try again later. Choosing the right code is part of designing a good API. The codes used in QUIZZY:

Code	Name	Meaning in plain words
200	OK	Here is what you asked for.
201	Created	I made the new thing you sent.
400	Bad Request	Your data is wrong or incomplete.
401	Unauthorized	I don't know who you are (not logged in, or the token is invalid).
403	Forbidden	I know who you are, but you may not do this.
404	Not Found	There is nothing here (or nothing you may see).
413	Payload Too Large	What you sent is bigger than allowed.
429	Too Many Requests	Slow down; try again later.
500	Internal Server Error	Something broke on the server.
502	Bad Gateway	A service the server relies on failed.

REAL-LIFE EXAMPLE

The stamps on a returned parcel: "delivered", "address incomplete", "recipient unknown", "post office closed".

Where the analogy breaks down: Despite its name, 401 "Unauthorized" really means "not authenticated" (we don't know who you are); 403 is the real "not allowed".

WHERE REAL APPS USE IT

- The "404 Page not found" page on any website.
- APIs answering 429 when you send too many requests.
- "Something went wrong" pages, which usually come from a 500.

WHERE IN QUIZZY

`server/src/controllers/quiz.controller.js` picks the codes: `createQuiz` returns 201 or 400, `getQuiz` returns 404 for other hosts' quizzes, `generateQuiz` returns 502 when Gemini fails; `requireAuth.js` returns 401.

Request header

DEFINITION

In one sentence: Extra labelled information sent along with a request, separate from its main content, such as "this is JSON" or "here is my login token".

In more detail: Headers are name and value pairs, for example `Content-Type: application/json` or `Authorization: Bearer <token>`. Servers read them to know how to handle the request. Responses have headers too, such as `Access-Control-Allow-Origin` for CORS or `Retry-After` after a 429.

REAL-LIFE EXAMPLE

The writing on the outside of an envelope (addressee, "fragile", return address), while the letter inside is the body.

WHERE REAL APPS USE IT

- Browsers send your preferred language so sites can show the right one.
- APIs read the `Authorization` header to find out who is calling.

WHERE IN QUIZZY

`client/src/api.js → apiFetch` sets the `Content-Type` and `Authorization` headers; `server/src/middlewares/requireAuth.js` reads `req.headers.authorization`; `aiRateLimit.js` sets the `Retry-After` response header.

CORS (Cross-Origin Resource Sharing)

DEFINITION

In one sentence: A browser rule that stops a web page from reading data from a different website's server, unless that server says the page is allowed to.

In more detail: Browsers follow the "same-origin policy": JavaScript loaded from one origin can't read responses from another origin by default. CORS is how a server opts in: it sends response headers such as `Access-Control-Allow-Origin` naming the origins it trusts. This protects users, because a malicious site you visit can't use your browser, and your logins, to read data from another site. CORS is enforced only by browsers; tools such as curl ignore it, so it never replaces checking who the caller is.

REAL-LIFE EXAMPLE

A receptionist who only hands you a reply letter from another company if that company's letterhead says "OK to share with visitors from Vercel Street".

Where the analogy breaks down: The other company may already have received and acted on your request; CORS only controls whether the page gets to read the answer. That is why servers must still check permissions themselves.

WHERE REAL APPS USE IT

- A web app on `app.example.com` calling its own API on `api.example.com`, which allows exactly that origin.
- Public data APIs such as weather services, which deliberately allow every origin.
- Web fonts loaded from another domain, which need CORS headers to be usable.

WHERE IN QUIZZY

`server/src/app.js` → `cors({ origin: CLIENT_URL })` and `server/server.js` → `new Server(httpServer, { cors: { origin: CLIENT_URL } })` let only the Vercel site call the Render API and the Socket.IO server.

Preflight request

DEFINITION

In one sentence: A short automatic "may I?" request the browser sends before certain cross-origin requests, to check that the server allows them.

In more detail: For cross-origin requests that aren't "simple", for example ones carrying an `Authorization` header or a JSON body, the browser first sends an `OPTIONS` request asking which methods and headers are allowed. Only if the server's CORS headers approve does the browser send the real request. Your own code never sends it; the browser does it by itself.

REAL-LIFE EXAMPLE

Phoning ahead to ask whether you may bring a guest, before turning up at the party.

WHERE REAL APPS USE IT

- Single-page apps calling an API on another domain with a login token.
- Any `fetch` that sends JSON to a different origin.

WHERE IN QUIZZY

Every `client/src/api.js → apiFetch` call from Vercel to Render triggers a preflight (it has an `Authorization` header and a JSON body); the `cors` middleware in `server/src/app.js` answers it.

Polling

DEFINITION

In one sentence: Asking a server "anything new?" again and again on a timer.

In more detail: Polling is the simplest way to get updates over plain HTTP, but most requests come back with nothing new, which wastes data and battery, and every update waits until the next ask. Long-polling improves on it, and WebSockets avoid the problem by keeping one connection open so the server can speak first.

REAL-LIFE EXAMPLE

A child in the back seat asking "are we there yet?" every minute.

WHERE REAL APPS USE IT

- Older webmail pages checking for new mail every few minutes.
- Simple dashboards that refresh their numbers every few seconds.

WHERE IN QUIZZY

QUIZZY deliberately avoids polling: `DECISIONS.md` decision 4 rejects it in favour of Socket.IO, which only uses long-polling briefly while a connection starts.

CDN (Content Delivery Network)

DEFINITION

In one sentence: A network of servers around the world that keep copies of a website's files, so each visitor downloads them from a server near them.

In more detail: CDN servers store (cache) static files such as HTML, JavaScript and images close to users. This makes pages load faster and protects the main server from traffic spikes. It suits files that are the same for everyone; personal or live data still comes from the app's own server.

REAL-LIFE EXAMPLE

A chain of local warehouses all stocking the same products, so every delivery comes from the closest one.

WHERE REAL APPS USE IT

- Video streaming services sending video from servers near the viewer.
- News sites serving their images and scripts.

WHERE IN QUIZZY

Vercel serves the built files from `client/dist` through its CDN; the live game data still comes from the server on Render.

Mixed content

DEFINITION

In one sentence: When a secure (HTTPS) page tries to load something over insecure HTTP; browsers block it.

In more detail: If an HTTPS page fetched data over plain `http://` or `ws://`, that part could be read or changed on the way, breaking the page's promise of security. Browsers therefore block such requests. The fix is to use `https://` and `wss://` for everything.

REAL-LIFE EXAMPLE

A bank vault with its back door left open, so the guard locks the back door before letting anything through.

WHERE REAL APPS USE IT

- Old websites that moved to HTTPS but still loaded images over `http://`.
- A deployed HTTPS site accidentally calling `http://localhost`.

WHERE IN QUIZZY

`VITE_API_URL` (read in `client/src/config/config.js`) must start with `https://` on Vercel; an `http://` value would make the browser block every `apiFetch` call and both sockets.

3.2 Real-time communication

Real-time

DEFINITION

In one sentence: Updates appear on your screen the moment something happens, without anyone refreshing the page.

In more detail: A real-time web app keeps a live connection open so the server can push changes immediately, usually within a fraction of a second. This needs a connection that stays open, such as a WebSocket, and the server must keep track of who should receive each update.

REAL-LIFE EXAMPLE

Watching a football match live instead of reading about it in tomorrow's newspaper.

WHERE REAL APPS USE IT

- Chat apps showing new messages instantly.
- Live sports scores.
- Delivery and ride-tracking maps that move the car as it drives.

WHERE IN QUIZZY

The host's leaderboard (`client/src/components/Leaderboard.jsx`) changes as soon as `server/src/sockets/socket.js → emitLeaderboard` runs after a student answers.

WebSocket

DEFINITION

In one sentence: A connection between a browser and a server that stays open, so either side can send messages at any moment.

In more detail: A normal HTTP request is one question and one answer, and then it's finished. A WebSocket starts as an HTTP request and is then "upgraded" into a long-lived, two-way connection. This lets the server push updates instantly instead of the browser asking again and again (polling).

REAL-LIFE EXAMPLE

HTTP is like sending letters; a WebSocket is like a phone call that stays connected, where either person can speak whenever they want.

WHERE REAL APPS USE IT

- Chat apps like WhatsApp Web: new messages appear instantly.
- Collaborative editors like Google Docs: you see others typing.
- Live sports scores and trading dashboards.

WHERE IN QUIZZY

Socket.IO uses WebSockets for the live game. `server/src/sockets/socket.js` handles events such as `quiz:answer`, and the host's leaderboard updates the moment any student answers.

Long-polling

DEFINITION

In one sentence: A way to get near-instant updates over normal HTTP: the browser asks for news, and the server holds the question open until there is something to say.

In more detail: When the server finally answers (or a time limit passes), the browser immediately asks again. It works on any network that allows normal web traffic, even ones that block WebSockets, but every message costs a full HTTP request. Socket.IO starts every connection with long-polling and switches ("upgrades") to a WebSocket when it can.

REAL-LIFE EXAMPLE

Asking a shopkeeper to call you back when your parcel arrives, and phoning again right after each call.

WHERE REAL APPS USE IT

- Chat systems falling back to it on strict company or school networks.
- Real-time web apps from before WebSockets were widely supported.

WHERE IN QUIZZY

socket.io-client (`client/src/socket/socket.js` , `client/src/hostSocket.js`) starts with long-polling and then upgrades; those first ordinary HTTP requests are why the Socket.IO server in `server/server.js` needs CORS settings.

WSS (secure WebSocket)

DEFINITION

In one sentence: A WebSocket connection protected by encryption: the WebSocket version of HTTPS.

In more detail: `wss://` is a WebSocket sent through TLS, exactly as `https://` is HTTP sent through TLS. A page loaded over HTTPS must use `wss://`, otherwise the browser blocks the connection as mixed content. With Socket.IO you don't type `wss://` yourself: giving it an `https://` address makes it use `wss://`.

REAL-LIFE EXAMPLE

The same phone call as a WebSocket, but on a scrambled line that eavesdroppers can't understand.

WHERE REAL APPS USE IT

- WhatsApp Web and other browser chat apps.
- Online trading platforms streaming live prices.

WHERE IN QUIZZY

Because `VITE_API_URL` starts with `https://`, the sockets in `client/src/socket/socket.js` and `client/src/hostSocket.js` connect to Render with `wss://`.

Socket.IO

DEFINITION

In one sentence: A library that makes real-time messaging between browsers and a server easy, built on top of WebSockets.

In more detail: Socket.IO adds features plain WebSockets don't have: named events, socket rooms for sending to groups, automatic reconnection, acknowledgements, and a fallback to long-polling. It has two halves, `socket.io` on the Node server and `socket.io-client` in the browser, and both sides must use Socket.IO: a plain WebSocket client can't talk to a Socket.IO server.

REAL-LIFE EXAMPLE

A walkie-talkie system with labelled channels, automatic reconnection when you walk back into range, and "message received" confirmations.

WHERE REAL APPS USE IT

- Chat features in web apps.
- Live dashboards and notifications.
- Multiplayer browser games.

WHERE IN QUIZZY

`server/server.js` creates the Socket.IO server, `server/src/sockets/socket.js → registerSocketHandler` registers every event, and the browser clients are in `client/src/socket/socket.js` and `client/src/hostSocket.js`.

Why QUIZZY uses Socket.IO: the host's screen must update when *other* people act (a student joins, a student answers). With plain HTTP, the host's browser would have to keep polling. `DECISIONS.md` decision 4 records this choice and rejects polling.

Event

DEFINITION

In one sentence: A named message that says "this just happened", which code can listen for and react to.

In more detail: In Socket.IO an event has a name, such as `quiz:answer`, and optional data. One side sends it; the other side runs the function registered for that name with `socket.on(name, handler)`. Some events, like `connect` and `disconnect`, are sent by Socket.IO itself. Browsers use the same idea for clicks and key presses.

REAL-LIFE EXAMPLE

A doorbell: pressing it sends one specific signal, and whoever is listening decides what to do about it.

WHERE REAL APPS USE IT

- Chat apps sending "new message" and "is typing" events.
- Online games sending "player moved" events.
- Every web page reacting to click events.

WHERE IN QUIZZY

QUIZZY has 20 custom events (section 6.8). For example, `client/src/pages/PlayerPage.jsx → handleAnswer` sends `quiz:answer`, and `server/src/sockets/socket.js` handles it with `socket.on("quiz:answer", …)`.

Emit

DEFINITION

In one sentence: To send an event.

In more detail: `socket.emit(name, data)` sends an event over one connection: from the browser to the server, or from the server back to that one browser. To send to many connections at once, the server uses `io.to(…).emit(…)`. An emit can also carry a callback to receive an acknowledgement.

REAL-LIFE EXAMPLE

Saying something into one particular phone call.

WHERE REAL APPS USE IT

- A chat app emitting "new message" when you press send.
- A game emitting your move to the server.

WHERE IN QUIZZY

`client/src/pages/PlayerPage.jsx → handleAnswer` emits `quiz:answer`; the server's `quiz:answer` handler in `server/src/sockets/socket.js` replies with `socket.emit("quiz:answer:result", …)`.

Socket room

DEFINITION

In one sentence: A named group of live connections on the server, so one message can be sent to everyone in the group at once.

In more detail: In Socket.IO, `socket.join(name)` adds a connection to a socket room and `io.to(name).emit(…)` sends to all of its members. Socket rooms exist only on the server (browsers don't know which rooms they are in), and every connection is automatically in a room named after its own socket ID. In this guide, "socket room" always means this Socket.IO group, while "room" means QUIZZY's own game room object in memory.

REAL-LIFE EXAMPLE

A group chat: post once and every member receives it, and only the admin (the server) adds or removes members.

WHERE REAL APPS USE IT

- Chat channels, with one socket room per channel.
- Collaborative editors, with one socket room per open document.
- Online games, with one socket room per match.

WHERE IN QUIZZY

`server/src/sockets/socket.js` puts every socket of a game into the socket room `"483920"` and the host's sockets also into `"host:483920"`; `io.to(player.id)` uses each player's personal socket room.

Broadcast

DEFINITION

In one sentence: Sending one message to many receivers at the same time.

In more detail: In Socket.IO you broadcast by emitting to a socket room (`io.to(room).emit`) or to every connection (`io.emit`). The server sends a copy to each member; connections that aren't targeted receive nothing, so choosing the right socket room decides who sees what.

REAL-LIFE EXAMPLE

A loudspeaker announcement in one classroom: everyone in that room hears it, nobody in the next room does.

WHERE REAL APPS USE IT

- A new message delivered to all members of a group chat.
- A goal update pushed to everyone watching a match.

WHERE IN QUIZZY

The `player:join` handler in `server/src/sockets/socket.js` broadcasts `room:players` to the whole game with `io.to(room.code).emit`, while `emitLeaderboard` broadcasts only to `"host:CODE"`.

Acknowledgement

DEFINITION

In one sentence: A reply that confirms "I got your message and dealt with it".

In more detail: In Socket.IO the sender adds a callback function as the last argument of `emit`; the receiver calls it when finished, and the callback then runs on the sender's side, optionally with data. `socket.timeout(ms).emit(…)` adds a deadline: if no acknowledgement (often shortened to "ack") arrives in time, the callback receives an error instead. It turns "send and hope" into "send and get confirmation".

REAL-LIFE EXAMPLE

Recorded delivery: you get a signature back once the parcel has arrived.

WHERE REAL APPS USE IT

- The "delivered" ticks in messaging apps.
- An app saving a document and showing "Saved" only after the server confirms.

WHERE IN QUIZZY

`client/src/pages/HostPage.jsx` → `handleCloseRoom` uses `socket.timeout(3000).emit("host:close-room", …)`; the server's `host:close-room` handler calls the acknowledgement after closing the room, and the page then goes to `/host`.

Handshake

DEFINITION

In one sentence: The first exchange when a connection is opened, in which both sides agree to talk and the client can show its credentials.

In more detail: For a WebSocket, the handshake is an HTTP request asking to "upgrade" the connection. Socket.IO adds its own handshake data: whatever the client puts in its `auth` option arrives on the server as `socket.handshake.auth`, and server-side middleware can check it before the connection is accepted.

REAL-LIFE EXAMPLE

Showing your ticket at the door before you are let into a concert.

WHERE REAL APPS USE IT

- Chat apps logging in to their server when they connect.
- Every HTTPS connection, which starts with a TLS handshake to agree on encryption.

WHERE IN QUIZZY

`client/src/hostSocket.js` sends `{ token }` in the handshake; `server/src/sockets/socketAuth.js → socketAuth` reads `socket.handshake.auth.token` and stores the user's ID in `socket.data.userId`.

Reconnection

DEFINITION

In one sentence: Automatically connecting again after a connection drops, without the user doing anything.

In more detail: Socket.IO clients keep retrying, with growing pauses, when the connection is lost, for example after Wi-Fi drops or the server restarts. A reconnected client is a brand-new connection with a new socket ID, so the app must restore its own situation, for example by telling the server again who it is.

REAL-LIFE EXAMPLE

Your phone automatically redialling after a call drops, but the person on the other end still has to recognise your voice.

WHERE REAL APPS USE IT

- Chat apps coming back online after you leave a tunnel.
- Video calls that freeze for a moment and then resume.

WHERE IN QUIZZY

`client/src/pages/HostPage.jsx` listens for `connect` (`handleConnect`), so after every reconnection it sends `host:resume` and `server/src/sockets/socket.js` re-attaches the host to their room. Students have no equivalent yet (section 14).

3.3 React and the browser

Component

DEFINITION

In one sentence: A reusable piece of the user interface, written as a function that returns what should appear on screen.

In more detail: In React, a component is a JavaScript function whose name starts with a capital letter and that returns JSX. Components can contain other components, so a whole page is a tree of small pieces. A component receives inputs as props and can remember its own state; React runs the function again whenever either of them changes.

REAL-LIFE EXAMPLE

LEGO bricks: small standard pieces, each with one shape, combined into bigger structures and reused in many builds.

WHERE REAL APPS USE IT

- Social media feeds, where every post, like button and comment box is a component used thousands of times.
- Design systems that give a company one shared set of buttons, cards and inputs.

WHERE IN QUIZZY

`client/src/components/QuestionView.jsx` is a component used by `PlayerPage`; `client/src/components/Button.jsx` is used by all six pages.

JSX

DEFINITION

In one sentence: The HTML-like code you write inside JavaScript to describe what a React component shows.

In more detail: JSX looks like HTML, for example `<Button onClick={handleJoin}>Join</Button>`, but the build tool turns it into ordinary JavaScript function calls. Curly braces `{ }` insert JavaScript values. Small differences from HTML include `className` instead of `class` and event names in camelCase, like `onClick`. Text inserted with `{ }` is escaped automatically, which protects against XSS.

REAL-LIFE EXAMPLE

A fill-in-the-blanks letter template: the layout is fixed, and the gaps in curly braces are filled with live data.

WHERE REAL APPS USE IT

- Every React web app.
- React Native phone apps, which use the same JSX syntax.

WHERE IN QUIZZY

Every `.jsx` file. For example, `client/src/pages/PlayerPage.jsx` returns different JSX for each screen of the game.

Props

DEFINITION

In one sentence: The inputs you pass into a component, like the settings you give a function when you call it.

In more detail: Props are written like HTML attributes, for example `<QuestionView data={currentQuestion} disabled={…} />`, and the component receives them as one object. Props flow one way, from parent to child, and a child can't change its own props. To report something back, a parent passes a function as a prop (like `onAnswer`) that the child calls.

REAL-LIFE EXAMPLE

Ordering a pizza: you choose the size and toppings, and the kitchen makes what you described, but it can't change your order form.

WHERE REAL APPS USE IT

- A "like" button component given a post ID and the current number of likes.
- A product card given a name, price and image.

WHERE IN QUIZZY

`client/src/pages/PlayerPage.jsx` passes `data`, `selectedOption`, `correctOption`, `disabled` and `onAnswer` to `client/src/components/QuestionView.jsx`.

State (in React)

DEFINITION

In one sentence: Data a component remembers between screen updates; when it changes, React redraws that part of the screen.

In more detail: State is created with `useState` and belongs to one component on screen. Unlike a normal variable, it keeps its value when the component runs again, and changing it through its setter function tells React to redraw. Anything on screen that changes over time (typed text, the current question, the score) is driven by state.

REAL-LIFE EXAMPLE

The scoreboard at a match: when someone changes the number, everyone looking at it sees the new score.

WHERE REAL APPS USE IT

- The item count on a shopping cart icon.
- A dark-mode on/off switch.
- The unread badge on a notifications bell.

WHERE IN QUIZZY

`client/src/pages/PlayerPage.jsx` keeps `nickname`, `joined`, `currentQuestion`, `result`, `score`, `finished` and more in state.

Hook

DEFINITION

In one sentence: A special React function, whose name starts with "use", that gives a component a React feature such as memory (state) or running code after drawing (effects).

In more detail: Built-in hooks include `useState`, `useEffect` and `useContext`. They must be called at the top level of a component, in the same order every time, never inside `if` statements or loops, because React tells them apart by their order. You can also combine hooks into your own custom hooks.

REAL-LIFE EXAMPLE

Standard sockets in a wall: plug an appliance (the component) into the right socket to get power (state or effects), and the sockets must always be in the same place.

Where the analogy breaks down: Real sockets can be used in any order; hooks can't, which is exactly the rule that trips beginners up.

WHERE REAL APPS USE IT

- All modern React apps.
- Libraries that offer hooks, such as `useNavigate` from React Router.

WHERE IN QUIZZY

`client/src/pages/PlayerPage.jsx` uses `useState`, `useEffect` and `useParams`; `QuizListPage.jsx` uses the custom hook `useAuth`.

useState

DEFINITION

In one sentence: The hook that gives a component a piece of state plus a function to change it.

In more detail: `const [score, setScore] = useState(0)` returns the current value and a setter. Calling `setScore(10)` asks React to redraw with the new value. When the new value depends on the old one, pass a function instead, for example `setQuestions(prev => [...prev, q])`, so you always build on the latest value.

REAL-LIFE EXAMPLE

A whiteboard with a marker: anyone can read it, the marker is the only way to change it, and everyone watching sees the update.

WHERE REAL APPS USE IT

- Opening and closing a menu.
- The text inside a search box.

WHERE IN QUIZZY

`client/src/pages/CreateQuizPage.jsx` keeps `title`, `questions`, `saving` and `generating` with `useState`; `addQuestion` calls `setQuestions((prev) => [...prev, emptyQuestion()])`.

useEffect

DEFINITION

In one sentence: The hook for running code after the screen has been drawn, usually to connect to something outside React.

In more detail: `useEffect(fn, deps)` runs `fn` after drawing; with an empty list `[]` as `deps`, it runs once when the component first appears. It is used for "side effects": loading data, starting timers, subscribing to updates, opening sockets. The function can return a cleanup function to undo what it started.

REAL-LIFE EXAMPLE

Moving into a flat: once you're in, you set up the internet and mail forwarding (the effect); when you move out, you cancel them (the cleanup).

WHERE REAL APPS USE IT

- Loading data when a page opens.
- Subscribing to live updates, such as a chat connection.
- Changing the browser tab's title.

WHERE IN QUIZZY

`client/src/pages/PlayerPage.jsx` registers its socket listeners and calls `socket.connect()` inside `useEffect`; `QuizListPage.jsx` loads the quiz list in one.

Cleanup function

DEFINITION

In one sentence: The function a `useEffect` returns, which React runs to undo the effect when the component disappears (or before the effect runs again).

In more detail: It removes listeners, clears timers and closes connections so they don't keep running after the screen is gone. Forgetting it causes memory leaks and duplicated behaviour, for example every event being handled twice after you navigate away and back.

REAL-LIFE EXAMPLE

Switching off the lights and locking the door when you leave a room.

WHERE REAL APPS USE IT

- Closing a chat connection when you leave a chat screen.
- Stopping a countdown timer when a dialog closes.

WHERE IN QUIZZY

`client/src/pages/PlayerPage.jsx` and `HostPage.jsx` return a function that calls `socket.off(…)` for every listener and then `socket.disconnect()`; `QuizListPage.jsx`'s cleanup sets `ignore = true` so a late answer is ignored.

Context (React)

DEFINITION

In one sentence: A React feature for sharing data with many components without passing it by hand through every level in between.

In more detail: A Provider component high up in the tree supplies a value, and any component below it can read that value with `useContext` or a custom hook. It avoids "prop drilling", which means passing the same prop through components that don't need it. It suits app-wide data such as the logged-in user or the colour theme.

REAL-LIFE EXAMPLE

A building's Wi-Fi: every room can connect directly, instead of running a cable through every room in between.

WHERE REAL APPS USE IT

- Logged-in user information shown on every page.
- Light and dark themes.
- Language settings.

WHERE IN QUIZZY

`client/src/auth/AuthContext.jsx → AuthProvider` wraps the whole app in `client/src/main.jsx` and shares `session`, `user`, `loading` and `signOut`.

Custom hook

DEFINITION

In one sentence: Your own function, whose name starts with "use", that bundles up hooks you need in several places.

In more detail: A custom hook can call other hooks and return whatever is useful. It shares logic, not data: each component that calls it gets its own values, unless the hook reads something shared such as a context. It keeps components short and puts the shared logic in one place.

REAL-LIFE EXAMPLE

A recipe card you reuse: every cook who follows it makes their own dish.

WHERE REAL APPS USE IT

- Common app hooks such as "use the current user" or "use the window size".
- Library hooks such as `useNavigate` and `useParams` from React Router.

WHERE IN QUIZZY

`client/src/auth/AuthContext.jsx → useAuth` reads the auth context and throws a clear error if it is used outside `AuthProvider`; `RequireAuth`, `LoginPage` and `QuizListPage` use it.

Controlled input

DEFINITION

In one sentence: A form field whose content always comes from React state, so React decides what it shows.

In more detail: You set `value={nickname}` and update the state in `onChange`. Every keystroke goes through your code, so you can check or change it (for example, remove anything that isn't a digit), and the state always matches what's on screen. The alternative, an uncontrolled input, keeps its own value inside the browser.

REAL-LIFE EXAMPLE

A puppet: it only moves when the puppeteer (the state) pulls the strings.

WHERE REAL APPS USE IT

- Search boxes that filter results as you type.
- Card number fields that add spaces as you type.

WHERE IN QUIZZY

`client/src/pages/HomePage.jsx` uses `onChange={(e) => setCode(e.target.value.replace(/\D/g, ""))}` so only digits appear; the nickname field in `PlayerPage.jsx` is controlled by `nickname`.

Conditional rendering

DEFINITION

In one sentence: Showing different things on screen depending on a condition.

In more detail: In JSX you use normal JavaScript: `{error && <ErrorMessage>…}` shows the message only when `error` has a value, and `condition ? <A /> : <B />` picks one of two. Because React shows exactly what the expression produces, the screen always matches the state.

REAL-LIFE EXAMPLE

A shop sign that shows OPEN or CLOSED depending on the time.

WHERE REAL APPS USE IT

- Showing "Log in" or your profile picture depending on whether you are logged in.
- Showing a loading spinner until the content arrives.

WHERE IN QUIZZY

`client/src/pages/HostPage.jsx` shows the "End quiz" button only when `leaderboard.status === "playing"`; `PlayerPage.jsx` shows `ErrorMessage` only when `error` is set.

Early return

DEFINITION

In one sentence: Leaving a function as soon as you know the answer, instead of wrapping everything in nested if/else blocks.

In more detail: In a component, `if (loading) return <Spinner />;` returns that screen right away and skips the rest. A row of early returns reads as "the first matching case wins". The same pattern in handlers (`if (!valid) return;`) keeps the main path of the code flat and easy to follow.

REAL-LIFE EXAMPLE

A bouncer going down a checklist: "banned? leave. no ticket? leave. otherwise, come in."

WHERE REAL APPS USE IT

- Route guards that send logged-out users to the login page.
- Server handlers that stop at the first invalid input.

WHERE IN QUIZZY

`client/src/pages/PlayerPage.jsx` checks `finalResult` , `closedMessage` , `finished` , `currentQuestion` and `joined` in order and returns the first matching screen; `server/src/services/room.service.js → addPlayer` returns `{ error }` early for each broken rule.

Key (in lists)

DEFINITION

In one sentence: A unique label you give each item in a list so React can tell the items apart when the list changes.

In more detail: When you draw a list with `map` , each element needs `key={…}` . React uses keys to match old and new items, so it updates, moves or removes the right ones and keeps each item's state. Use a stable ID; using the position in the array mixes items up when the list is reordered, which also breaks animations.

REAL-LIFE EXAMPLE

Name tags at a party: people can move around the room, but you still know who is who.

WHERE REAL APPS USE IT

- Chat message lists.
- Social media feeds and to-do lists.

WHERE IN QUIZZY

`client/src/components/Leaderboard.jsx` uses `key={player.id}` so rows slide to their new rank; `CreateQuizPage.jsx` uses the temporary `q.id` made by `crypto.randomUUID()` .

Immutable update

DEFINITION

In one sentence: Changing data by making an updated copy, instead of editing the original.

In more detail: React decides whether to redraw by checking whether the state is a new object or array. Editing an array in place (with `push`) keeps the same array, so React may not notice. Instead, create a new one with the spread operator or `map`: `[...prev, item]` or `prev.map(…)`. It also prevents bugs where shared data changes unexpectedly.

REAL-LIFE EXAMPLE

Saving a new version of a document instead of writing over the old one, so anyone can see that something changed.

WHERE REAL APPS USE IT

- Undo and redo history in editors.
- State-management libraries such as Redux, which require it.

WHERE IN QUIZZY

`client/src/pages/CreateQuizPage.jsx` → `updateQuestion` and `updateOption` use `map` and spread to return new arrays and objects.

Single-page application (SPA)

DEFINITION

In one sentence: A website that loads one page once and then changes what's on screen with JavaScript, instead of loading a new page from the server for every click.

In more detail: The browser downloads `index.html` and the JavaScript bundle once; after that, moving between "pages" just swaps components, and data comes from API calls. This feels fast, like an app. The trade-off: the host must answer every path with `index.html`, or direct links like `/join/483920` break.

REAL-LIFE EXAMPLE

A presenter at a whiteboard who erases and redraws parts of it, instead of handing out a new sheet of paper for every change.

WHERE REAL APPS USE IT

- Gmail.
- Google Maps.
- Project boards such as Trello.

WHERE IN QUIZZY

`client/index.html` loads `client/src/main.jsx` once, and `client/vercel.json` sends every path to `index.html`.

Client-side routing

DEFINITION

In one sentence: Switching pages inside a single-page application by changing the address bar and swapping components, without asking the server for a new page.

In more detail: A router library reads the path in the URL, shows the matching component, and updates the address bar when you navigate, using the browser's History API, so the back and forward buttons and bookmarks still work.

REAL-LIFE EXAMPLE

A receptionist directing visitors to different rooms inside the same building, so nobody has to leave and come back in.

WHERE REAL APPS USE IT

- Gmail moving between the inbox and a message.
- Video sites moving to the next video without a full reload.

WHERE IN QUIZZY

`client/src/App.jsx` declares the routes with React Router; `client/src/pages/HomePage.jsx` → `handleJoin` calls `navigate(`/join/${cleanCode}`)`.

URL parameter

DEFINITION

In one sentence: A changeable part of a URL path, like the `483920` in `/join/483920`.

In more detail: A route is declared with a placeholder, such as `/join/:code`; the router accepts any value in that position and hands it to the component, in React Router through `useParams()`. One page can then show many different items.

REAL-LIFE EXAMPLE

The flat number in an address: every flat in the building has the same layout, but the number says which one you mean.

WHERE REAL APPS USE IT

- Product pages such as `/product/123`.
- Profile pages such as `/user/alice`.

WHERE IN QUIZZY

`client/src/pages/PlayerPage.jsx` reads `const { code } = useParams()`; `HostPage.jsx` reads `quizId` the same way.

State machine

DEFINITION

In one sentence: A way of describing something that is always in exactly one of a few named states, with clear rules for moving from one state to another.

In more detail: Each state allows certain events, and each event leads to a specific next state. Describing screens or game logic this way rules out impossible combinations, such as being "finished" and "waiting" at the same time, and makes the logic easy to draw as a diagram.

REAL-LIFE EXAMPLE

A traffic light: red, then green, then amber, then red again, and never two colours at once.

WHERE REAL APPS USE IT

- Order tracking: placed, shipped, delivered.
- Media players: playing, paused, stopped.

WHERE IN QUIZZY

A room's `status` goes `"waiting"` → `"playing"` → `"finished"` (`server/src/services/room.service.js` → `startQuiz` , `endQuiz`); the screens of `PlayerPage.jsx` form another state machine (figure 5.1).

localStorage

DEFINITION

In one sentence: A small storage area in the browser where a website can keep text that is still there after the page or browser is closed.

In more detail: Data is stored per origin as name and value text, and survives reloads and restarts until it is cleared. Any JavaScript running on that origin can read it, so it must not hold anything an XSS attack shouldn't steal. `sessionStorage` works the same way but is emptied when the tab closes.

REAL-LIFE EXAMPLE

A locker at your gym: your things stay there between visits, but anyone who gets into that changing room can open it.

WHERE REAL APPS USE IT

- Remembering your dark-mode choice.
- Keeping an unsent draft.
- Many single-page apps storing the login session.

WHERE IN QUIZZY

supabase-js (`client/src/supabase/supabaseClient.js`) keeps the host's session in localStorage, which is why refreshing the page doesn't log the host out.

Reduced motion

DEFINITION

In one sentence: A setting on phones and computers that asks apps to use less animation, which websites can detect and respect.

In more detail: Movement on screen can make some people dizzy or distract them. Operating systems offer a "reduce motion" option, which CSS can detect with the `prefers-reduced-motion` media query. Considerate apps turn off or simplify animations when it is on, while still showing the same information.

REAL-LIFE EXAMPLE

A "no flash photography" sign at a show: the show goes on, just without the part that hurts some people.

WHERE REAL APPS USE IT

- iPhone and Android system animations.
- Websites switching off moving backgrounds and parallax scrolling.

WHERE IN QUIZZY

Tailwind's `motion-reduce:` classes (for example in `client/src/components/Button.jsx`) and `<MotionConfig reducedMotion="user">` in `client/src/components/Leaderboard.jsx`.

3.4 JavaScript and Node.js

Runtime

DEFINITION

In one sentence: The program that actually runs your code and gives it access to things like the network, files and timers.

In more detail: JavaScript always needs a runtime. A browser is one: it adds the web page, `fetch` and localStorage. Node.js is another: it adds files, network servers and `process.env`. The language is the same, but the built-in tools differ, which is why browser code can't read files from your disk and server code has no web page to draw on.

REAL-LIFE EXAMPLE

A games console: the game only runs on a console that supports it, and each console offers different controllers.

WHERE REAL APPS USE IT

- Web browsers, which run the JavaScript of every website.
- Node.js, which runs web servers and developer tools (Vite itself runs on Node).

WHERE IN QUIZZY

The code in `client/` runs in the browser's runtime; `server/server.js` runs in Node.js on Render.

Event loop

DEFINITION

In one sentence: The mechanism that lets JavaScript, which does one thing at a time, keep many tasks going by running each small job in turn instead of waiting.

In more detail: JavaScript runs one piece of code at a time. Slow operations (network calls, timers, database queries) are handed off; when they finish, the code that should handle the result is put in a queue, and the event loop runs queued jobs one by one whenever the current code has finished. Code is never interrupted in the middle, but one slow calculation blocks everything else.

REAL-LIFE EXAMPLE

A single chef who puts dishes in the oven and serves other orders while they cook, returning when each timer rings. But if the chef spends ten minutes chopping onions, nobody else gets served.

WHERE REAL APPS USE IT

- Node.js web servers handling thousands of connections with one thread.
- Browsers keeping a page responsive while data loads.

WHERE IN QUIZZY

The handlers in `server/src/sockets/socket.js` for many students run one at a time on Node's event loop, so two handlers never change the `rooms` Map at the same instant.

Asynchronous code

DEFINITION

In one sentence: Code that starts a slow task, carries on with other work, and deals with the result later when it is ready.

In more detail: Network requests, database queries and timers take time; waiting for them would freeze the program. Asynchronous functions return straight away and deliver their result later, through callbacks, Promises or async/await. Anything after an `await` may run after other events have happened in between.

REAL-LIFE EXAMPLE

Ordering coffee and getting a buzzer: you sit down, and the buzzer tells you when your coffee is ready.

WHERE REAL APPS USE IT

- A feed loading more posts while you keep scrolling.
- Photos uploading in the background while you use the app.

WHERE IN QUIZZY

`client/src/api.js → apiFetch` and `server/src/services/quiz.service.js → getQuizById` are asynchronous; the `host:create-room` handler in `server/src/sockets/socket.js` waits for the database with `await`.

Promise

DEFINITION

In one sentence: An object that stands for a result that isn't ready yet, which will later either succeed with a value or fail with an error.

In more detail: A Promise starts as "pending" and then becomes "fulfilled" (with a value) or "rejected" (with an error). You say what should happen next with `.then` and `.catch`, or wait for it with `await`. `fetch`, Supabase queries and Gemini calls all return Promises.

REAL-LIFE EXAMPLE

A cloakroom ticket: it isn't your coat, but a promise that you'll get your coat back, or be told it's been lost.

WHERE REAL APPS USE IT

- Every `fetch` call in web apps.
- Database and payment libraries that answer later.

WHERE IN QUIZZY

`server/src/services/ai.service.js → sleep` returns a Promise that finishes after a delay; `client/src/auth/AuthContext.jsx` uses `supabase.auth.getSession().then(…)`.

async/await

DEFINITION

In one sentence: Two keywords that let you write asynchronous code that reads top to bottom like normal code.

In more detail: `async` marks a function that returns a Promise; inside it, `await` pauses that function until a Promise finishes, without freezing the rest of the program. Errors from an awaited Promise can be caught with an ordinary `try/catch`.

REAL-LIFE EXAMPLE

A recipe step "wait until the dough has risen, then shape it": you can do other chores while you wait.

WHERE REAL APPS USE IT

- Modern Node.js servers.
- Modern browser code that loads data.

WHERE IN QUIZZY

`client/src/pages/CreateQuizPage.jsx` → `handleSubmit` uses `await apiFetch(…)` inside `try/catch/finally`; `server/src/middlewares/requireAuth.js` awaits `supabase.auth.getUser`.

Module (import and export)

DEFINITION

In one sentence: A single code file that shares chosen pieces with other files (export) and uses pieces from other files (import).

In more detail: Modules keep code organised and private by default: only what a file exports can be used elsewhere. The client uses ES modules (`import` / `export`); the server uses CommonJS (`require` / `module.exports`). A default export is a file's main item (`import Button from "./Button"`); named exports are imported in curly braces (`import { apiFetch } from "./api"`).

REAL-LIFE EXAMPLE

Departments in a company: each keeps its own work private and shares only official documents with the others.

WHERE REAL APPS USE IT

- Every modern JavaScript project.
- npm packages, which are modules other people publish.

WHERE IN QUIZZY

`client/src/api.js` has a named export `apiFetch`; `server/src/services/room.service.js` shares its functions with `module.exports = { … }` and `server/src/sockets/socket.js` loads them with `require`.

npm package (dependency)

DEFINITION

In one sentence: Ready-made code published by other people that you add to your project instead of writing it yourself.

In more detail: npm is Node's package manager. `package.json` lists the packages a project depends on (its dependencies) with allowed version ranges; `npm install` downloads them into `node_modules`, and `package-lock.json` records the exact versions so every install is identical. devDependencies are only needed while developing and building.

REAL-LIFE EXAMPLE

Buying flat-pack furniture parts instead of cutting the wood yourself, with a parts list (`package.json`) that says exactly what you need.

WHERE REAL APPS USE IT

- React and Express are both npm packages.
- Almost every JavaScript project uses dozens of them.

WHERE IN QUIZZY

`client/package.json` lists packages such as `react` , `socket.io-client` , `motion` and `qrcode.react` ; `server/package.json` lists `express` , `socket.io` , `zod` and `@google/genai` .

Singleton

DEFINITION

In one sentence: Something that exists only once in the whole program, shared by every part of the code that uses it.

In more detail: In JavaScript, a module's code runs only once, so an object created at the top of a module and exported is in effect a singleton: every file that imports it gets the same object. It suits shared connections and clients. The downside is shared state, which can make code harder to test.

REAL-LIFE EXAMPLE

A school's single main office: everyone goes to the same place.

WHERE REAL APPS USE IT

- One shared database connection pool in a web server.
- One app-wide logger.

WHERE IN QUIZZY

`client/src/socket/socket.js` and `client/src/hostSocket.js` each export one socket, `server/src/db/supaBase.js` exports one Supabase client, and `server/src/services/room.service.js` keeps one `rooms` Map.

Environment variable

DEFINITION

In one sentence: A named setting given to a program from outside its code, such as `GEMINI_API_KEY` .

In more detail: The same code can then use different values on your laptop (from a `.env` file) and in production (from the Vercel or Render dashboard), and secrets never end up in Git. Node reads them from `process.env` ; Vite exposes only variables whose names start with `VITE_` to browser code, and bakes their values into the files at build time.

REAL-LIFE EXAMPLE

The thermostat settings in a rented flat: the flat (the code) is the same for every tenant, but each tenant sets their own temperature.

WHERE REAL APPS USE IT

- Every deployed web app, for database addresses and API keys.
- Switching between test and live payment accounts without changing code.

WHERE IN QUIZZY

`server/src/services/ai.service.js` reads `process.env.GEMINI_API_KEY`; `client/src/config/config.js` reads `import.meta.env.VITE_API_URL`. Section 10.3 lists them all.

Map (JavaScript)

DEFINITION

In one sentence: A JavaScript collection that stores values under keys, so you can look a value up quickly by its key.

In more detail: A Map holds key-and-value pairs with the methods `set`, `get`, `has` and `delete`; looking something up by key stays fast however many entries there are. Unlike a plain object, any type can be a key and the order of insertion is kept; unlike an array, you don't have to search through the items one by one.

REAL-LIFE EXAMPLE

A cloakroom: hand in ticket number 42 and you get exactly your coat back, straight away.

WHERE REAL APPS USE IT

- Caches that remember recent results.
- Tracking which users are online, by user ID.

WHERE IN QUIZZY

`server/src/services/room.service.js` → `const rooms = new Map()` maps a room code to its room; `server/src/middlewares/aiRateLimit.js` maps each user ID to a list of times.

Pure function

DEFINITION

In one sentence: A function whose result depends only on its inputs and which changes nothing outside itself.

In more detail: Given the same inputs, a pure function always returns the same output, and it has no "side effects": no network calls, no saving, no changing shared data. That makes it easy to test and reason about. Real programs mix pure logic with a few impure edges that talk to the outside world.

REAL-LIFE EXAMPLE

A calculator: 2 + 2 is always 4, and pressing the keys doesn't change anything else.

WHERE REAL APPS USE IT

- Price and tax calculations.
- Formatting dates and numbers for display.

WHERE IN QUIZZY

`server/src/services/question.service.js → getPublicQuestion` and `isCorrect`, and `quiz.service.js → toQuiz`. The functions in `room.service.js` are not pure, because they change the `rooms` Map.

Destructuring

DEFINITION

In one sentence: A shortcut for taking values out of an object or array and putting them into separate variables in one line.

In more detail: `const { roomCode, nickname } = data` creates two variables from `data`'s properties; `const [score, setScore] = useState(0)` does the same for an array. You can rename values and give defaults. If the value being destructured is `undefined` or `null`, JavaScript throws an error, which is why defaults matter.

REAL-LIFE EXAMPLE

Unpacking a delivery box and putting each item straight onto its own shelf.

WHERE REAL APPS USE IT

- Reading fields from API responses.
- Function parameters in React components (`({ players, nickname })`).

WHERE IN QUIZZY

`server/src/sockets/socket.js`: `socket.on("player:join", (data = {}) => { const { roomCode, nickname } = data; … })`. The `quiz:start` handler destructures without a default, which is the cause of the crash in section 14.

Spread operator

DEFINITION

In one sentence: Three dots (`...`) that copy all the items of an array, or all the properties of an object, into a new one.

In more detail: `[...prev, item]` makes a new array with one extra item; `{ ...q, text }` copies an object and replaces `text`. When two objects are spread into one, properties written later win. The copy is shallow: objects nested inside are shared, not copied. It is the usual way to write immutable updates in React.

REAL-LIFE EXAMPLE

Photocopying a form and changing one field on the copy.

WHERE REAL APPS USE IT

- React state updates.
- Merging a user's settings over the default settings.

WHERE IN QUIZZY

`client/src/api.js → apiFetch` spreads `options` and `headers`; `server/src/controllers/quiz.controller.js → createQuiz` spreads `result.data`, where the order matters (section 14).

3.5 Server design

Middleware

DEFINITION

In one sentence: A checkpoint that every request passes through before it reaches the code that answers it, which can inspect the request, add information to it, or turn it away.

In more detail: In Express, a middleware is a function `(req, res, next)` that runs, in the order it was registered, before the route's handler. It can stop the request by replying itself (for example 401 "Not logged in"), attach facts to it (such as `req.user`), or call `next()` to pass it along. This lets you write a check like "is this user logged in?" once and apply it to many routes. Socket.IO uses the same idea for new connections with `io.use(…)`.

REAL-LIFE EXAMPLE

Airport security before your gate: you pass the checkpoints in a fixed order (ID check, then bag scan), any of them can send you back, and the ID desk stamps your boarding pass so the gate doesn't have to check your ID again. That stamp is what `requireAuth` does when it sets `req.user`.

WHERE REAL APPS USE IT

- Almost every web backend, to check the login on protected API routes.
- Request logging, recording the path and response time of every request.
- Reading JSON bodies and adding security headers.

WHERE IN QUIZZY

`server/src/middlewares/requireAuth.js → requireAuth`, applied to every quiz route by `router.use(requireAuth)` in `quiz.routes.js`, checks the Bearer token and sets `req.user`; `server/src/sockets/socketAuth.js → socketAuth` does the same job for socket connections.

Route

DEFINITION

In one sentence: A rule that connects a web address and HTTP method to the code that should handle it.

In more detail: In Express, `router.get("/", handler)` means "GET requests to this path go to this handler". Paths can contain parameters such as `/:id`. Express checks routes in the order they were registered and stops at the first one that sends a response, which matters in QUIZZY (section 14, issue 2).

REAL-LIFE EXAMPLE

The signs in a hospital corridor that send each patient to the right department.

WHERE REAL APPS USE IT

- `/login`, `/cart` or `/api/orders` in any web backend.
- Webhook addresses that receive notifications from payment providers.

WHERE IN QUIZZY

`server/src/routes/quiz.routes.js` registers `GET /`, `POST /`, `GET /:id` and `POST /generate`, mounted under `/api/quizzes` in `server/src/app.js`.

Controller

DEFINITION

In one sentence: The function that handles one route: it reads the request, checks it, asks the right service to do the work, and sends the response.

In more detail: Controllers deal with the web side of things (URL parameters, request bodies, status codes) and leave the actual business rules to services. That keeps each layer focused: controllers don't talk to the database, and services don't know about HTTP.

REAL-LIFE EXAMPLE

A waiter: takes your order, checks it's on the menu, passes it to the kitchen, and brings back the plate (or explains what went wrong).

WHERE REAL APPS USE IT

- Web frameworks such as Rails, Laravel and Spring are built around controllers.
- Most REST APIs organise their handlers this way.

WHERE IN QUIZZY

`server/src/controllers/quiz.controller.js → createQuiz` checks the body with `quizSchema`, calls `quizService.createQuiz`, and replies 201 or 400.

Service layer

DEFINITION

In one sentence: The part of the backend that holds the real business rules and data access, kept separate from how requests arrive.

In more detail: Services are plain functions (or classes) that do the actual work, such as saving a quiz or scoring an answer, and can be used from several entry points: REST controllers, socket handlers or scripts. Because they know nothing about HTTP, they are easier to reuse and to test.

REAL-LIFE EXAMPLE

The kitchen: it cooks the same dish whether the order came from a waiter, a phone call or a delivery app.

WHERE REAL APPS USE IT

- A payment service used by both a shop's website and its phone app.
- An email service called from sign-up, password reset and receipts.

WHERE IN QUIZZY

`server/src/services/quiz.service.js → getQuizById` is used by both the REST controller (`getQuiz`) and the socket handler for `host:create-room` .

Validation

DEFINITION

In one sentence: Checking that incoming data has the right shape and stays within the allowed limits before using it.

In more detail: Validation catches missing fields, wrong types, text that is too long or numbers out of range, and rejects them with a clear error. It must happen on the server, because anyone can skip the checks in a browser. Validating early keeps bad data out of the database and the game.

REAL-LIFE EXAMPLE

A clerk at the passport office who sends back forms with missing signatures before they go any further.

WHERE REAL APPS USE IT

- Sign-up forms checking that an email address looks valid.
- Payment APIs rejecting negative amounts.

WHERE IN QUIZZY

`server/src/controllers/quiz.controller.js → createQuiz` runs `quizSchema.safeParse(req.body)` ; `server/src/services/room.service.js → addPlayer` checks the nickname's length and uniqueness.

Schema

DEFINITION

In one sentence: A written description of what valid data looks like: which fields exist, what type each one is, and what limits apply.

In more detail: A schema can describe a database table (its columns and types), a JSON document (JSON Schema), or validation rules in code (a Zod schema). Code then checks real data against it. Using one schema in several places keeps the rules the same everywhere.

REAL-LIFE EXAMPLE

A cookie cutter: dough that fits the shape is accepted, everything else is trimmed off or rejected.

WHERE REAL APPS USE IT

- Database table definitions.
- API documentation formats such as OpenAPI.
- Form validation in web apps.

WHERE IN QUIZZY

`server/src/validation/quiz.schema.js` → `questionSchema` checks both host-written quizzes and AI output (`server/src/services/ai.service.js` → `generatedSchema`); the Postgres tables have their own schema (section 7).

Allowlist vs denylist

DEFINITION

In one sentence: An allowlist names exactly what is permitted and blocks everything else; a denylist names what is forbidden and allows everything else.

In more detail: Allowlists are safer, because anything you forgot to think about is blocked by default; with a denylist, anything you forgot gets through. They are used for allowed values in a field, allowed fields to copy into a response, and allowed websites.

REAL-LIFE EXAMPLE

The guest list at a private party (allowlist) versus a list of banned people at a public bar (denylist).

WHERE REAL APPS USE IT

- CORS settings naming the websites allowed to call an API.
- Phone app permissions you grant one by one.

WHERE IN QUIZZY

`server/src/validation/ai.schema.js` → `difficulty: z.enum(["easy", "medium", "hard"])`; `server/src/services/question.service.js` → `getPublicQuestion` copies only `text` and `options`; `CLIENT_URL` is a one-website allowlist.

Rate limiting

DEFINITION

In one sentence: Limiting how many times someone can do something within a period of time.

In more detail: The server counts requests per user (or per IP address) and rejects the extra ones with 429 "Too Many Requests", often saying when to try again. It protects expensive or easily abused actions (AI calls, login attempts, sending messages) from overuse and attacks.

REAL-LIFE EXAMPLE

A fairground ride with a limit of ten rides per wristband per hour.

WHERE REAL APPS USE IT

- Login pages limiting wrong password attempts.
- Public APIs with request quotas.
- Messaging apps limiting how many chats you can forward a message to.

WHERE IN QUIZZY

`server/src/middlewares/aiRateLimit.js` allows 10 AI generations per user per hour, but it is never reached because of the duplicate route (section 14, issue 2).

Sliding window

DEFINITION

In one sentence: A way of counting requests over "the last hour from right now", a window that keeps moving forward, instead of per fixed clock hour.

In more detail: With a fixed window (say 14:00 to 15:00), someone could make 10 requests at 14:59 and another 10 at 15:00. A sliding window always looks back exactly one hour from each new request, so bursts around the boundary aren't possible. A simple version stores the time of each request and throws away the old ones.

REAL-LIFE EXAMPLE

A library rule of "at most 5 books in any 7-day period" rather than "5 books per calendar week".

WHERE REAL APPS USE IT

- API rate limits.
- Fraud checks such as "at most 3 failed card payments in any 10 minutes".

WHERE IN QUIZZY

`server/src/middlewares/aiRateLimit.js` keeps each user's request times and drops those older than `WINDOW_MS` (one hour).

Transient error

DEFINITION

In one sentence: A temporary failure that will probably work if you simply try again a little later.

In more detail: Typical causes are a service being briefly overloaded (HTTP 503), a short-term usage limit (429), or a network glitch. These are worth retrying. Permanent errors, such as invalid input (400), are not: retrying just fails again.

REAL-LIFE EXAMPLE

A busy phone line: try again in a minute and you'll probably get through.

WHERE REAL APPS USE IT

- Cloud services automatically retrying failed requests.
- App stores retrying an interrupted download.

WHERE IN QUIZZY

`server/src/services/ai.service.js` → `TRANSIENT_STATUSES = [429, 500, 503]`; `callWithRetry` retries only these.

Exponential backoff

DEFINITION

In one sentence: Waiting longer and longer between retries, for example 1 second, then 2, then 4.

In more detail: Doubling the wait gives an overloaded service time to recover instead of flooding it with retries, and a maximum number of attempts stops it going on forever. Many systems also add a little randomness ("jitter") so that many clients don't all retry at the same moment.

REAL-LIFE EXAMPLE

Knocking on a door, and waiting a bit longer each time before knocking again.

WHERE REAL APPS USE IT

- Email servers retrying a failed delivery.
- Phone apps reconnecting after losing signal.

WHERE IN QUIZZY

`server/src/services/ai.service.js` → `callWithRetry` waits `1000 * 2 ** attempt` milliseconds (1 s, then 2 s) for up to 2 retries.

Race condition

DEFINITION

In one sentence: A bug that happens when the result depends on which of two overlapping actions finishes first.

In more detail: When two actions overlap (two clicks, two browser tabs, a reply arriving after the user has left), each one may assume nothing changed in between. In Node.js this happens around `await`, because other events run while a function is waiting. The fix is to check again after waiting, or to block the second action with a flag or a database transaction.

REAL-LIFE EXAMPLE

Two people booking the last seat on a plane at the same moment, each seeing "1 seat available".

WHERE REAL APPS USE IT

- Double payments when a "Pay" button is clicked twice.
- Two people editing the same record and one silently overwriting the other.

WHERE IN QUIZZY

The `host:create-room` handler in `server/src/sockets/socket.js` sets `socket.data.creatingRoom`, then checks `findRoomByHost` and `socket.connected` again after awaiting `getQuizById`.

In-memory state

DEFINITION

In one sentence: Data a server keeps in its own working memory (RAM) instead of in a database.

In more detail: It is very fast and simple (just JavaScript objects), but it disappears when the server restarts and isn't shared between several copies of the server. It suits short-lived data such as a live game.

REAL-LIFE EXAMPLE

Keeping the score on a whiteboard in the room rather than in the filing cabinet.

WHERE REAL APPS USE IT

- Game servers holding the state of a live match.
- Caches that keep recent results close at hand.

WHERE IN QUIZZY

`server/src/services/room.service.js` → the `rooms` Map holds every live game; results are only written to the database at the end, by `server/src/services/session.service.js`.

Grace period

DEFINITION

In one sentence: A short wait before taking an action that can't be undone, giving someone the chance to come back first.

In more detail: Instead of reacting immediately to something like a disconnect, the system starts a timer; if the person returns before it runs out, the timer is cancelled and nothing happens.

REAL-LIFE EXAMPLE

A car park that gives you ten minutes after paying to drive out.

WHERE REAL APPS USE IT

- Subscription services keeping your account active for a few days after a failed payment.
- Chat apps waiting a little before showing someone as offline.

WHERE IN QUIZZY

`server/src/sockets/socket.js` → `HOST_GRACE_MS` (60 seconds): when the host disconnects a timer starts, and `host:resume` cancels it.

Load balancer and sticky sessions

DEFINITION

In one sentence: A load balancer spreads users across several copies of a server; sticky sessions make sure the same user keeps going to the same copy.

In more detail: When one server isn't enough, you run several copies behind a load balancer. Connections that hold state, like Socket.IO's long-polling, need every request from one browser to reach the same copy, hence sticky sessions. A shared store such as Redis then lets the copies share data.

REAL-LIFE EXAMPLE

A supermarket manager sending each customer to a free checkout; once you have started unloading your shopping, you stay at that checkout until you are done.

WHERE REAL APPS USE IT

- Every large website runs many server copies behind load balancers.
- Real-time apps using sticky sessions so long-polling connections work.

WHERE IN QUIZZY

Not used: QUIZZY runs one server instance on Render. It is part of the scaling plan in sections 12 and 13.

Redis

DEFINITION

In one sentence: A very fast database that keeps its data in memory, often used as a shared scratchpad between several servers.

In more detail: Redis stores keys with values (text, lists, counters and more) and can delete keys automatically after a set time. Because it is fast and shared, it is commonly used for caches, rate limits, sessions and passing real-time messages between server copies; Socket.IO has a Redis adapter for exactly that.

REAL-LIFE EXAMPLE

A shared whiteboard in the staff room that every shift can read and update.

WHERE REAL APPS USE IT

- Caching pages and query results.
- Game leaderboards.
- Rate limiting across many servers.

WHERE IN QUIZZY

Not used today. Sections 12 and 13 suggest it for holding rooms and rate-limit counters so QUIZZY could run on several servers.

Health check route

DEFINITION

In one sentence: A tiny web address a server answers just to show that it is alive.

In more detail: Hosting platforms and monitoring tools call it regularly; if it stops answering, they can alert someone or restart the server. It should be quick and need no login.

REAL-LIFE EXAMPLE

A nurse checking a patient's pulse.

WHERE REAL APPS USE IT

- Cloud platforms checking `/health` before sending traffic to a server.
- Uptime monitors that alert you when a site goes down.

WHERE IN QUIZZY

`server/src/app.js` → `GET /` replies "QUIZZY Server is running!"; calling it is also a way to wake the sleeping free Render server.

3.6 Databases

Database

DEFINITION

In one sentence: An organised, permanent store of data that programs can search and update safely.

In more detail: A database keeps data on disk so it survives restarts, lets many users read and write at the same time without corrupting anything, and answers questions about the data ("queries") quickly. The database management system, such as PostgreSQL, is the program that does all of this.

REAL-LIFE EXAMPLE

A well-organised library with a librarian who fetches and files books for you.

WHERE REAL APPS USE IT

- User accounts on every website.
- Product catalogues in online shops.
- Bank account records.

WHERE IN QUIZZY

Supabase's Postgres database stores quizzes, questions, sessions and results; `server/src/db/supaBase.js` connects to it.

Relational database

DEFINITION

In one sentence: A database that stores data in tables and links the tables together through shared ID numbers.

In more detail: Each kind of thing gets its own table, and relationships are stored with keys: a question row stores the ID of its quiz. SQL queries can combine tables. Relational databases enforce rules (types, unique keys, foreign keys) and support transactions. PostgreSQL, MySQL and SQLite are examples.

REAL-LIFE EXAMPLE

A set of spreadsheets that refer to each other by ID numbers, with a strict clerk who refuses entries that break the rules.

WHERE REAL APPS USE IT

- Banking systems.
- Online shop orders and stock.
- School and hospital records.

WHERE IN QUIZZY

QUIZZY's four tables (`quizzes` , `questions` , `sessions` , `results`) are linked by `quiz_id` and `session_id` (section 7.2).

Table, row and column

DEFINITION

In one sentence: A table holds one kind of thing, each row is one item, and each column is one property that every item has.

In more detail: It works like a spreadsheet: the `questions` table has one row per question and columns such as `text`, `options` and `correct_index`. Every column has a type (text, whole number, UUID and so on), and every row has the same columns.

REAL-LIFE EXAMPLE

A class register: one row per student, with columns for name, date of birth and so on.

WHERE REAL APPS USE IT

- A `users` table in almost every app.
- An `orders` table in an online shop.

WHERE IN QUIZZY

The `questions` table has one row per question with the columns `id`, `quiz_id`, `position`, `text`, `options` and `correct_index` (section 7.2).

SQL

DEFINITION

In one sentence: The language used to ask a relational database for data or to change the data in it.

In more detail: SQL (Structured Query Language) statements such as `SELECT`, `INSERT`, `UPDATE` and `DELETE` describe what you want, and the database works out how to do it. Libraries like supabase-js write these requests for you.

REAL-LIFE EXAMPLE

A standard request slip at a library: you write down what you want, and the librarian finds it.

WHERE REAL APPS USE IT

- Every relational database.
- Data analysts writing reports.

WHERE IN QUIZZY

QUIZZY's server writes no raw SQL: `server/src/services/quiz.service.js` uses supabase-js (`from(…).select(…)`), and the SQL lives inside the `create_quiz` and `save_session_results` database functions.

Primary key

DEFINITION

In one sentence: A column whose value uniquely identifies each row in a table.

In more detail: No two rows can share a primary key, and it can never be empty. Other tables use it to refer to that row. QUIZZY uses UUIDs as primary keys, and the database automatically builds an index on primary keys so lookups by ID are fast.

REAL-LIFE EXAMPLE

A student ID number: names can repeat, ID numbers never do.

WHERE REAL APPS USE IT

- User IDs.
- Order numbers.

WHERE IN QUIZZY

`quizzes.id`, `questions.id`, `sessions.id` and `results.id` are UUID primary keys; `server/src/services/quiz.service.js → getQuizById` finds a quiz with `.eq("id", id)`.

Foreign key

DEFINITION

In one sentence: A column that stores another table's primary key to link two rows, and the database refuses links to rows that don't exist.

In more detail: `questions.quiz_id` must contain an existing `quizzes.id`. This guarantee ("referential integrity") prevents orphaned data, such as a question belonging to no quiz. Foreign keys also tell tools like Supabase how tables relate, which makes combined queries possible.

REAL-LIFE EXAMPLE

A library loan slip that must contain a real member number; the system won't accept a made-up one.

WHERE REAL APPS USE IT

- Orders pointing to customers.
- Comments pointing to posts.

WHERE IN QUIZZY

`questions.quiz_id` → `quizzes.id`, `sessions.quiz_id` → `quizzes.id`, `results.session_id` → `sessions.id`; `server/src/services/quiz.service.js → getQuizById` relies on the first to load a quiz together with its questions.

One-to-many relationship

DEFINITION

In one sentence: A link between tables where one row in the first table connects to many rows in the second.

In more detail: It is stored with a foreign key on the "many" side: each question has one `quiz_id`, and one quiz can have many questions. Other kinds are one-to-one, and many-to-many (which needs an extra linking table).

REAL-LIFE EXAMPLE

A parent and children: each child has one birth mother, but a mother can have many children.

WHERE REAL APPS USE IT

- One customer with many orders.
- One post with many comments.

WHERE IN QUIZZY

`quizzes` → `questions` and `sessions` → `results` (figure 7.2).

JOIN

DEFINITION

In one sentence: Combining rows from two tables into one result by matching them on a shared column.

In more detail: A SQL `JOIN` lines up, for example, each quiz with its questions where `questions.quiz_id = quizzes.id`. Supabase hides the JOIN behind "embedded" selects: asking for `questions(…)` inside a `quizzes` query makes it join along the foreign key for you.

REAL-LIFE EXAMPLE

Stapling each order slip to the matching customer's card.

WHERE REAL APPS USE IT

- Showing posts together with their authors' names.
- Showing an order together with its products.

WHERE IN QUIZZY

`server/src/services/quiz.service.js` → `getQuizById` selects `"id, title, creator_id, questions(id, position, text, options, correct_index)"` in one query; `listQuizzes` uses `questions(count)`.

Index (database)

DEFINITION

In one sentence: An extra lookup structure the database keeps so it can find rows quickly without reading the whole table.

In more detail: Like the index at the back of a book, it is sorted by one or more columns and points to the matching rows. It makes searching and sorting fast, but uses extra space and makes writing slightly slower. Postgres automatically indexes primary keys; foreign keys and other columns only get an index if you create one.

REAL-LIFE EXAMPLE

The index at the back of a textbook: look up "photosynthesis" and jump straight to page 212.

WHERE REAL APPS USE IT

- Finding a user by email address at login.
- Sorting orders by date.

WHERE IN QUIZZY

QUIZZY's primary keys are indexed automatically. Useful extra indexes would be `questions(quiz_id, position)` and `quizzes(creator_id, created_at)`; whether they exist isn't visible in the dashboard (section 7.7).

UUID

DEFINITION

In one sentence: A long random ID, like `1c0ba4c7-a2c3-4c7f-…`, that is practically guaranteed to be unique without anyone keeping count.

In more detail: A UUID (Universally Unique Identifier) is 128 bits written as 32 hexadecimal characters in five groups. Random (version 4) UUIDs can be created anywhere (browser, server, database) with a negligible chance of two ever being the same, and unlike 1, 2, 3 they can't be guessed.

REAL-LIFE EXAMPLE

A lottery ticket number with so many digits that two tickets never match.

WHERE REAL APPS USE IT

- Database IDs in many systems.
- Identifiers for files, devices and orders.

WHERE IN QUIZZY

All four tables use UUID primary keys; `server/src/services/quiz.service.js → isValidId` checks that an ID looks like a UUID; `client/src/pages/CreateQuizPage.jsx` uses `crypto.randomUUID()` for temporary React keys.

Transaction

DEFINITION

In one sentence: A group of database changes that is saved as one unit, so either all of the changes happen or none of them do.

In more detail: If any step inside a transaction fails, the database "rolls back", meaning it undoes the earlier steps, so you never get half-finished data. Postgres transactions follow the ACID guarantees. In Postgres, a single database-function call automatically runs inside one transaction, which is why QUIZZY puts saves that touch two tables into functions.

REAL-LIFE EXAMPLE

A bank transfer: money leaving account A and arriving in account B is one action, and if the deposit fails, the withdrawal is cancelled, so money never disappears halfway.

WHERE REAL APPS USE IT

- Banking and payment apps moving money.
- Online shops creating an order and reducing the stock together.
- Ticket booking, reserving a seat and recording the booking so two people can't get the same seat.

WHERE IN QUIZZY

`server/src/services/quiz.service.js → createQuiz` calls the `create_quiz` database function, which saves the quiz and all its questions in one transaction; `server/src/services/session.service.js → saveSessionResults` does the same for `sessions` and `results`.

ACID

DEFINITION

In one sentence: Four promises a database makes about transactions: all-or-nothing, the rules are always kept, other users don't disturb you, and saved means saved.

In more detail: **A**tomicity: all steps or none. **C**onsistency: the data always obeys the rules (types, keys). **I**solation: transactions running at the same time don't see each other's half-finished work. **D**urability: once saved ("committed"), the data survives crashes. PostgreSQL provides all four.

REAL-LIFE EXAMPLE

A bank transfer that happens completely (atomic), follows the bank's rules (consistent), can't see other customers' half-done transfers (isolated), and is recorded permanently in the ledger (durable).

WHERE REAL APPS USE IT

- Banking and payment systems.
- Booking and stock systems that must never double-sell.

WHERE IN QUIZZY

`create_quiz` and `save_session_results` each run as one Postgres transaction (called from `server/src/services/quiz.service.js → createQuiz` and `server/src/services/session.service.js → saveSessionResults`).

Migration (database)

DEFINITION

In one sentence: A saved script, kept with the code, that changes the database structure one step at a time.

In more detail: Instead of clicking changes in a dashboard, each change (create a table, add a column) is written as a numbered SQL file. Running the migrations in order rebuilds the exact same database anywhere, and code review shows every change. Tools such as the Supabase CLI manage them.

REAL-LIFE EXAMPLE

Building plans filed with the council: anyone can rebuild the same house from the plans, step by step.

WHERE REAL APPS USE IT

- Web frameworks such as Rails and Django, which have migrations built in.
- Teams keeping their test and live databases identical.

WHERE IN QUIZZY

QUIZZY has no migrations: the tables and functions were created in the Supabase dashboard. Adding them is a recommended improvement (sections 12 and 14).

Row Level Security (RLS)

DEFINITION

In one sentence: A database feature that decides, row by row, which rows each user is allowed to see or change.

In more detail: When RLS is switched on for a table, every query from a normal (non-admin) user is filtered by rules called policies, such as "only rows where `creator_id` is the logged-in user". With RLS on and no policies, normal users can't access the table at all. Admin keys, such as Supabase's secret key, bypass RLS.

REAL-LIFE EXAMPLE

A hotel key card that only opens your own room, even though every room is in the same building.

WHERE REAL APPS USE IT

- Apps where the browser talks to the database directly, such as many apps built on Supabase.
- Business software keeping each company's data separate in one database.

WHERE IN QUIZZY

QUIZZY's browser never queries tables, and the dashboard suggests RLS is on with no policies (section 7.6), so the public key opens nothing; `server/src/db/supaBase.js` uses the secret key, which bypasses RLS.

Database function (RPC)

DEFINITION

In one sentence: A small program stored inside the database that the app can call by name over the network.

In more detail: A Postgres function contains SQL (and logic such as variables and loops) and runs entirely inside the database, as one transaction. RPC means "Remote Procedure Call": calling a function that lives somewhere else as if it were local. Supabase makes such functions callable with `supabase.rpc("name", args)`.

REAL-LIFE EXAMPLE

A vending machine's "meal deal" button: one press, several items, all handled inside the machine.

WHERE REAL APPS USE IT

- Saving data that spans several tables in one step.
- Reports calculated inside the database, close to the data.

WHERE IN QUIZZY

`server/src/services/quiz.service.js` → `createQuiz` calls `supabase.rpc("create_quiz", …)`; `server/src/services/session.service.js` → `saveSessionResults` calls `supabase.rpc("save_session_results", …)`.

Cascade delete

DEFINITION

In one sentence: A database rule that automatically deletes the child rows when their parent row is deleted.

In more detail: It is set on a foreign key with `ON DELETE CASCADE`: deleting a quiz would then also delete its questions, instead of the database refusing the delete (the default) or leaving orphans behind. It is powerful but dangerous, because one delete can remove a lot of data.

REAL-LIFE EXAMPLE

Deleting a folder on your computer, which also deletes every file inside it.

WHERE REAL APPS USE IT

- Deleting an account also deleting that account's posts.
- Deleting a project also deleting its tasks.

WHERE IN QUIZZY

QUIZZY has no delete feature yet, and whether its foreign keys cascade isn't visible in the dashboard. If deleting quizzes is added, the questions should cascade, and someone has to decide what happens to old sessions and results.

snake_case vs camelCase

DEFINITION

In one sentence: Two styles of writing names: snake_case joins words with underscores (`correct_index`), camelCase joins them with capital letters (`correctIndex`).

In more detail: SQL databases usually use snake_case, while JavaScript uses camelCase. Code that moves data between the two converts the names in one place, so the rest of the code only ever sees one style.

REAL-LIFE EXAMPLE

Translating a form at the border, so people on each side read it in their own language.

WHERE REAL APPS USE IT

- Most backends converting database rows into program objects.
- API libraries that convert field names automatically.

WHERE IN QUIZZY

`server/src/services/quiz.service.js → toQuiz` converts `creator_id` → `creatorId` and `correct_index` → `correctIndex` ; the JSON sent to `create_quiz` still uses `correctIndex` (section 7.2).

3.7 Security and authentication

Authentication

DEFINITION

In one sentence: Checking who you are.

In more detail: Usually done with something you know (a password), something you have (a code sent to your phone) or something you are (a fingerprint). Once you are authenticated, the app gives you a session or token so you don't have to prove it again on every click.

REAL-LIFE EXAMPLE

Showing your ID card at a building's reception desk.

WHERE REAL APPS USE IT

- Logging in to your email.
- Unlocking a banking app with your face or fingerprint.

WHERE IN QUIZZY

`client/src/pages/LoginPage.jsx → handleSubmit` calls `supabase.auth.signInWithPassword`; the server then checks the resulting token in `server/src/middlewares/requireAuth.js` and `server/src/sockets/socketAuth.js`.

Authorization

DEFINITION

In one sentence: Checking what you are allowed to do, once we know who you are.

In more detail: Authorization rules look like "only the quiz's owner may host it". They are checked on the server, after authentication, and for every single action, not just once at login.

REAL-LIFE EXAMPLE

Your ID gets you into the building (authentication); your key card decides which rooms you can open (authorization).

WHERE REAL APPS USE IT

- Only admins can delete other users' accounts.
- You can edit your own posts but not other people's.

WHERE IN QUIZZY

`server/src/controllers/quiz.controller.js → getQuiz` checks `quiz.creatorId !== req.user.id`; `server/src/services/room.service.js → startQuiz` checks `room.hostId`. Section 8.5 lists every rule.

Password hashing

DEFINITION

In one sentence: Storing a scrambled "fingerprint" of a password instead of the password itself, so even the people running the database can't read it.

In more detail: A hash function turns the password into a fixed-length string that can't be turned back into the password. At login, the typed password is hashed again and the two hashes are compared. Good password hashes are deliberately slow and add a random value (a "salt") to every password, so stolen hashes are very hard to crack.

REAL-LIFE EXAMPLE

A fingerprint: it can confirm that it's you, but nobody can rebuild you from it.

WHERE REAL APPS USE IT

- Every reputable login system.
- Password managers protecting their own master password.

WHERE IN QUIZZY

QUIZZY's server never sees passwords: Supabase Auth hashes and stores them, and `client/src/pages/LoginPage.jsx` sends the password straight to Supabase over HTTPS.

Session

DEFINITION

In one sentence: The period during which you stay logged in, and the information the app keeps so it remembers that you are.

In more detail: After login, the app stores something (a cookie or tokens) so you don't have to type your password on every page. In Supabase, a session is an access token plus a refresh token, saved in the browser's localStorage and renewed automatically until you log out. (In QUIZZY's database, the table named `sessions` means something else: one finished game.)

REAL-LIFE EXAMPLE

A day pass at a museum: you show it at each room instead of buying a new ticket every time.

WHERE REAL APPS USE IT

- Staying logged in to a social media site.
- Shopping sites remembering who you are while you browse.

WHERE IN QUIZZY

`client/src/auth/AuthContext.jsx → AuthProvider` reads the session with `supabase.auth.getSession()` and keeps it up to date with `onAuthStateChange`.

Access token

DEFINITION

In one sentence: A short-lived pass the app sends with each request to prove the user is logged in.

In more detail: In QUIZZY it is a JWT that expires after about an hour (Supabase's default). A short lifetime limits the damage if it is ever stolen. The server checks it on every request.

REAL-LIFE EXAMPLE

A day ticket: valid today only, and useless tomorrow.

WHERE REAL APPS USE IT

- The APIs of most modern apps.
- "Log in with Google", which gives the app an access token.

WHERE IN QUIZZY

`client/src/api.js → apiFetch` reads `data.session.access_token` and sends it; `server/src/middlewares/requireAuth.js` checks it.

Refresh token

DEFINITION

In one sentence: A longer-lived secret used only to get a new access token when the old one expires, so the user stays logged in.

In more detail: It is sent only to the login service, never with normal requests, and Supabase swaps it for a fresh access token automatically in the background. Logging out cancels it.

REAL-LIFE EXAMPLE

Your membership card, which you show at the front desk to get a new day ticket each day.

WHERE REAL APPS USE IT

- Phone apps that keep you logged in for weeks.
- Big sign-in providers such as Google and Microsoft.

WHERE IN QUIZZY

supabase-js in the browser (`client/src/supabase/supabaseClient.js`) refreshes tokens automatically; QUIZZY's own code never touches the refresh token.

JWT (JSON Web Token)

DEFINITION

In one sentence: A small digital pass that proves who you are, which the server can check without asking for your password again.

In more detail: It has three parts separated by dots: header, payload and signature. The payload holds facts such as your user ID and the expiry time, and anyone can read it. The signature is made with a secret that only the issuer (Supabase) has, so if anyone changes the payload, the signature no longer matches and the token is rejected.

REAL-LIFE EXAMPLE

A concert wristband with a tamper-proof seal: anyone can read the name on it, but you can't change the name without breaking the seal.

Where the analogy breaks down: Unlike a wristband, a JWT also expires after about an hour.

WHERE REAL APPS USE IT

- Logins in many web and phone apps.
- Single sign-on, where one login works across several apps.
- APIs that need to know which user is calling.

WHERE IN QUIZZY

`client/src/api.js → apiFetch` sends it in the `Authorization` header; `server/src/middlewares/requireAuth.js` verifies it.

Why QUIZZY uses JWTs: they are how the server learns which host is calling without keeping its own list of logged-in users. QUIZZY checks each token by calling `supabase.auth.getUser(token)` (a network call to

Supabase) instead of checking the signature locally. That is simpler and also catches users who have been signed out or deleted, at the cost of one extra network request.

Signature (digital)

DEFINITION

In one sentence: A code attached to data that proves who created it and that it hasn't been changed since.

In more detail: The issuer calculates the signature from the data and a secret key. Anyone verifying it can check that the two still fit together; if even one character of the data changed, they no longer match. A signature doesn't hide the data; it only protects against changes and forgeries.

REAL-LIFE EXAMPLE

A wax seal on a letter: anyone can read the letter, but a broken or fake seal shows that someone tampered with it.

WHERE REAL APPS USE IT

- JWTs used for logins.
- Software updates signed by the company that made them.
- HTTPS certificates.

WHERE IN QUIZZY

The signature on Supabase's JWTs is what makes the user ID inside them trustworthy; `server/src/middlewares/requireAuth.js` leaves the actual check to `supabase.auth.getUser` instead of verifying it itself.

Bearer token

DEFINITION

In one sentence: A token that gives access to whoever presents it (the "bearer"), with no extra proof needed.

In more detail: It is sent in the header `Authorization: Bearer <token>`. Because simply having it is enough, a bearer token must only travel over HTTPS and must never be logged or shared.

REAL-LIFE EXAMPLE

Cash or a cinema ticket: whoever holds it can use it.

WHERE REAL APPS USE IT

- Most REST APIs, including those of GitHub and Google.
- Phone apps calling their own backends.

WHERE IN QUIZZY

`client/src/api.js → apiFetch` sets `Authorization: Bearer ${token}`; `server/src/middlewares/requireAuth.js` splits the header and rejects anything that isn't `Bearer <token>`.

Secret key vs publishable key

DEFINITION

In one sentence: A publishable key is safe to put in a website and gives only limited access; a secret key gives full admin access and must stay on the server.

In more detail: Supabase's publishable key identifies the project for public actions such as logging in, and its access to tables is limited by Row Level Security. The secret key bypasses RLS completely. Anything in client code can be read by anyone, so secret keys belong only in server environment variables. (Older Supabase projects call them the "anon" key and the "service_role" key.)

REAL-LIFE EXAMPLE

A shop's public phone number versus the key to its safe.

WHERE REAL APPS USE IT

- Payment providers: a publishable key in the checkout page, a secret key on the server.
- Map services, whose public keys are restricted to the company's own website.

WHERE IN QUIZZY

`client/src/supabase/supabaseClient.js` uses `VITE_SUPABASE_PUBLISHABLE_KEY`; `server/src/db/supaBase.js` uses `SUPABASE_SECRET_KEY`.

XSS (Cross-Site Scripting)

DEFINITION

In one sentence: An attack where someone sneaks their own JavaScript into a web page, so it runs in other people's browsers.

In more detail: XSS happens when an app shows text typed by a user as HTML, for example a nickname like `<img src=x onerror=…>`. The injected script could steal tokens or act as the victim. The defence is to always treat user text as text: React does this automatically for anything inserted with `{ }`, unless you use `dangerouslySetInnerHTML`.

REAL-LIFE EXAMPLE

Someone writes "give the visitor the master key" on a sticky note, and a careless assistant reads it out as an order instead of as a message.

WHERE REAL APPS USE IT

- A classic risk in comment sections and chat messages.
- Profile names and bios shown to other users.

WHERE IN QUIZZY

Nicknames and quiz text are shown with `{ }` in JSX (for example in `client/src/components/Lobby.jsx`), so React escapes them; QUIZZY never uses `dangerouslySetInnerHTML`.

SQL injection

DEFINITION

In one sentence: An attack where text typed by a user ends up being treated as part of a database command.

In more detail: If code builds SQL by gluing user input into a string, input like `'); DROP TABLE quizzes; --` can change what the command does. The defence is parameterised queries: the input is sent separately as data and is never mixed into the command itself.

REAL-LIFE EXAMPLE

A form where someone writes "and also hand me the safe keys" in the name box, and the clerk obeys it.

WHERE REAL APPS USE IT

- A risk for any search box or login form built with raw SQL.
- One of the most common causes of data breaches in older websites.

WHERE IN QUIZZY

supabase-js and `supabase.rpc` send values as parameters (`server/src/services/quiz.service.js`), so quiz titles and nicknames can't change any query.

Mass assignment

DEFINITION

In one sentence: An attack where a request includes extra fields the server didn't expect, such as `isAdmin: true`, and the server saves them without checking.

In more detail: It happens when code copies a whole request body into a database record. The defence is to accept only known fields (an allowlist), for example a validation schema that drops unknown fields.

REAL-LIFE EXAMPLE

A job application where the applicant adds a line "starting salary: one million", and HR files it without reading.

WHERE REAL APPS USE IT

- "Update my profile" endpoints.
- Sign-up forms that could be tricked into creating admin accounts.

WHERE IN QUIZZY

`server/src/validation/quiz.schema.js` → `quizSchema` drops unknown fields, so a `creatorId` sneaked into the body is removed before `server/src/controllers/quiz.controller.js` → `createQuiz` uses the data.

Enumeration attack

DEFINITION

In one sentence: Trying many IDs or usernames to find out which ones exist, based on how the app responds.

In more detail: If an app answers "forbidden" for items that exist but are private and "not found" for items that don't exist, an attacker learns which ones are real. Giving the same answer in both cases, and using IDs that can't be guessed, prevents it.

REAL-LIFE EXAMPLE

A hotel receptionist who says "that guest doesn't want visitors" instead of "no such guest", which confirms the guest is staying there.

WHERE REAL APPS USE IT

- Login pages saying "wrong email or password" rather than "no account with that email".
- Code-hosting sites showing "not found" for private projects you can't see.

WHERE IN QUIZZY

`server/src/controllers/quiz.controller.js → getQuiz` returns 404 both for missing quizzes and for other hosts' quizzes, and quiz IDs are random UUIDs.

Prompt injection

DEFINITION

In one sentence: Tricking an AI model with text that contains instructions, so it ignores the app's own instructions.

In more detail: When user text is placed inside a prompt, the model can't reliably tell the app's instructions apart from the user's. Defences: keep user input short and clearly separated, validate the model's output, limit what the output can do, and keep a human review step.

REAL-LIFE EXAMPLE

Slipping a note into a pile of letters for an assistant that says "ignore your boss and do what I say".

WHERE REAL APPS USE IT

- Chatbots and AI assistants.
- AI tools that read emails or web pages, which may contain hidden instructions.

WHERE IN QUIZZY

The topic is inserted into the prompt in `server/src/services/ai.service.js → buildPrompt`; it is limited to 100 characters, the output must pass Zod, and the host reviews the questions before saving.

Defense in depth

DEFINITION

In one sentence: Using several independent layers of protection, so one failure doesn't expose everything.

In more detail: Each layer assumes the others might fail: validation, authentication, authorization, database rules, network rules. An attacker has to get through all of them.

REAL-LIFE EXAMPLE

A castle with a moat, walls, a gate and guards.

WHERE REAL APPS USE IT

- Banks combining passwords, one-time codes and fraud monitoring.
- Cloud systems combining firewalls with per-user permissions.

WHERE IN QUIZZY

The ownership checks in `server/src/controllers/quiz.controller.js`, plus Row Level Security on the Supabase tables, plus CORS in `server/src/app.js` (section 11).

3.8 AI

Large language model (LLM)

DEFINITION

In one sentence: An AI program, trained on huge amounts of text, that writes new text by repeatedly predicting what should come next.

In more detail: A model such as Gemini takes a prompt and produces a response piece by piece (in "tokens"), choosing likely continuations based on patterns learned in training. It is very good at language tasks but can be confidently wrong (this is called "hallucinating"), so its output must be checked.

REAL-LIFE EXAMPLE

An extremely well-read autocomplete.

Where the analogy breaks down: Unlike the autocomplete on your phone, it can follow detailed instructions and produce whole structured documents, which is why it can write a full quiz.

WHERE REAL APPS USE IT

- Chat assistants such as Gemini and ChatGPT.
- Writing and coding helpers.
- Customer-support chatbots.

WHERE IN QUIZZY

`server/src/services/ai.service.js → generateQuestions` asks Gemini to write the quiz questions.

API key

DEFINITION

In one sentence: A secret string that identifies your app to a paid or limited service, so the service knows who to bill and which limits apply.

In more detail: It is sent with every request to the service (Gemini here). Unlike a user's login token, it represents the app itself, not a person. Anyone who has it can use up your quota or your money, so it must stay on the server.

REAL-LIFE EXAMPLE

A company credit card handed to one department: whoever holds it can spend on the company's account.

WHERE REAL APPS USE IT

- Map services shown inside apps.
- Payment, weather and AI services.

WHERE IN QUIZZY

`server/src/services/ai.service.js` reads `process.env.GEMINI_API_KEY` and passes it to `new GoogleGenAI({ apiKey })`; it is set only on Render and in `server/.env`.

Prompt

DEFINITION

In one sentence: The instructions and input text you send to an AI model.

In more detail: A clear prompt says what to produce, in what format, and which rules to follow (for example "exactly 4 options, exactly one correct"). Improving the prompt is often the cheapest way to get better results, but it never guarantees the rules are followed, so the output still has to be checked.

REAL-LIFE EXAMPLE

A detailed brief you give a freelancer before they start work.

WHERE REAL APPS USE IT

- Messages you type to an AI assistant.
- Hidden instructions behind AI features in apps, such as "summarise this email".

WHERE IN QUIZZY

`server/src/services/ai.service.js → buildPrompt` builds the prompt from the topic, difficulty and number of questions.

Structured output

DEFINITION

In one sentence: Asking an AI model to answer in a fixed, machine-readable format, such as JSON with specific fields, instead of free text.

In more detail: With Gemini you pass a JSON Schema and `responseMimeType: "application/json"`, and the model's answer is constrained to that shape. It greatly reduces answers that can't be read by the program, but it doesn't guarantee the content is correct, so you still validate it.

REAL-LIFE EXAMPLE

Asking someone to fill in a form instead of writing you a letter.

WHERE REAL APPS USE IT

- AI tools pulling the amount and date out of invoices.
- Chatbots that call other tools with exact, typed inputs.

WHERE IN QUIZZY

`server/src/services/ai.service.js` → `generateQuestions` passes `responseJsonSchema` and then checks the result with `generatedSchema` (Zod).

JSON Schema

DEFINITION

In one sentence: A standard way to describe what a JSON document must look like: its fields, their types, and which ones are required.

In more detail: A JSON Schema is itself written in JSON, for example `{ type: "object", properties: …, required: [...] }`. Tools use it to check data or, as Gemini does, to restrict what a model may generate. Zod schemas play a similar role inside JavaScript code.

REAL-LIFE EXAMPLE

The blank template of a form, showing every box and which ones must be filled in.

WHERE REAL APPS USE IT

- API documentation formats such as OpenAPI.
- Code editors checking configuration files as you type.

WHERE IN QUIZZY

`server/src/services/ai.service.js` → `responseJsonSchema` describes `{ questions: [{ text, options, correctIndex }] }`.

Model alias vs pinned model

DEFINITION

In one sentence: An alias is a model name that always points to the newest version; a pinned name stays on one exact version.

In more detail: Aliases give you upgrades automatically, but the model's behaviour can change without warning. Pinned versions behave predictably but are eventually retired, so you must update them yourself. Production apps usually pin a version and upgrade on purpose after testing.

REAL-LIFE EXAMPLE

Subscribing to "the latest edition" of a textbook versus ordering "the 3rd edition".

WHERE REAL APPS USE IT

- AI services, which offer both "latest" names and dated version names.
- Software dependencies, which can follow "latest" or an exact version number.

WHERE IN QUIZZY

The model name comes from the `GEMINI_MODEL` environment variable (`server/src/services/ai.service.js`); whether it is an alias or pinned depends on the value set on Render, which this guide doesn't show.

3.9 Building and deploying

Build

DEFINITION

In one sentence: Turning the source code you write into the optimised files that are actually shipped and run.

In more detail: For the client, `npm run build` (which runs `vite build`) converts JSX into plain JavaScript, bundles the files, removes unused code and minifies everything, writing the result to `client/dist`. The Node server needs no build step: Render just installs the packages and runs it.

REAL-LIFE EXAMPLE

Printing and binding a book from the author's manuscript.

WHERE REAL APPS USE IT

- Almost every modern website has a build step.
- Phone apps are "built" into installable packages before going to the app stores.

WHERE IN QUIZZY

`client/package.json` → `"build": "vite build"`; Vercel runs it on every deployment.

Bundling

DEFINITION

In one sentence: Combining many code files and libraries into a few files that the browser can download efficiently.

In more detail: A bundler starts at the entry file (`main.jsx`), follows every import, and packs everything needed into one or a few files, so the browser makes a few requests instead of hundreds. Bundlers can also split code into pieces that load only when needed.

REAL-LIFE EXAMPLE

Packing everything for a trip into a few suitcases instead of carrying a hundred loose items.

WHERE REAL APPS USE IT

- Almost all large websites ship bundled JavaScript.
- Browser extensions and desktop apps built with web technology.

WHERE IN QUIZZY

Vite bundles `client/src` into `client/dist/assets/index-….js` and `.css` ; the build warns that the main bundle is over 500 kB.

Minification

DEFINITION

In one sentence: Shrinking code by removing spaces, comments and long names, without changing what it does.

In more detail: Minified code is much smaller, so it downloads faster, but it is very hard for people to read. "Source maps" can map it back to the original code for debugging.

REAL-LIFE EXAMPLE

Writing a message in shorthand: shorter to send, same meaning.

WHERE REAL APPS USE IT

- The JavaScript and CSS of nearly every live website.
- JavaScript libraries published in a ".min.js" version.

WHERE IN QUIZZY

`vite build` (run by `npm run build` in `client/`) minifies the client bundle automatically.

Deployment

DEFINITION

In one sentence: Putting a new version of your app onto the computers where real users can use it.

In more detail: Typically: build the code, upload it, start the new version, and switch users over to it. Platforms like Vercel and Render do all of this automatically. Deploying a server restarts it, which matters when important data lives only in memory.

REAL-LIFE EXAMPLE

Restocking a shop's shelves with the new product line overnight.

WHERE REAL APPS USE IT

- Websites that update several times a day.
- Phone app updates appearing in the app store.

WHERE IN QUIZZY

Pushing to GitHub triggers a Vercel deployment (client) and a Render deployment (server); a Render deployment wipes QUIZZY's in-memory rooms.

Hosting

DEFINITION

In one sentence: Renting space on computers that are always online, so your app can be reached on the internet.

In more detail: A host provides the machines (or a managed platform), the network, HTTPS certificates and domain names. Different hosts suit different kinds of programs: static files, serverless functions, long-running servers or databases.

REAL-LIFE EXAMPLE

Renting a shop on a busy street instead of building one yourself.

WHERE REAL APPS USE IT

- Websites hosted on platforms such as Vercel or Netlify.
- Servers and databases hosted on cloud platforms.

WHERE IN QUIZZY

The client is hosted on Vercel, the server on Render, and the database on Supabase (section 2).

Static site

DEFINITION

In one sentence: A website made of ready-made files that are sent to every visitor exactly as they are, without the server running any code for each request.

In more detail: The HTML, CSS, JavaScript and images are produced once, at build time. Static files are cheap, fast and easy to copy onto a CDN. The site can still be interactive in the browser: a React single-page application is a static site whose JavaScript then calls APIs.

REAL-LIFE EXAMPLE

Printed flyers: every visitor gets the same copy.

WHERE REAL APPS USE IT

- Portfolios and documentation websites.
- The frontends of many single-page applications.

WHERE IN QUIZZY

The built client (`client/dist`) is a static site served by Vercel; all the dynamic work happens on Render.

Serverless function

DEFINITION

In one sentence: A small piece of server code that a platform starts only when a request arrives, and stops again soon after.

In more detail: You don't manage a server: the platform runs your function for one request (or a short burst), then freezes or discards it. Nothing is kept in memory between requests, and several copies can run at once without sharing memory. It is cheap and scales automatically, but doesn't suit long-lived connections or data held in memory.

REAL-LIFE EXAMPLE

A taxi: perfect for one trip, then it's gone.

Where the analogy breaks down: Several taxis can serve you at once, but none of them remembers your previous trip; that lack of memory is exactly why QUIZZY's server can't be serverless.

WHERE REAL APPS USE IT

- Contact forms and webhooks on websites.
- Image resizing triggered by an upload.

WHERE IN QUIZZY

QUIZZY doesn't use serverless functions: the live game needs a long-running process that holds sockets and the `rooms` Map (section 10.4).

Long-running process

DEFINITION

In one sentence: A program that starts once and keeps running continuously, handling many requests over hours or days.

In more detail: It can keep data in memory, hold connections open (like WebSockets) and run timers. The trade-offs: you pay for it even while it's idle, and a restart wipes its memory.

REAL-LIFE EXAMPLE

A shopkeeper who stays in the shop all day and remembers the regular customers.

WHERE REAL APPS USE IT

- Game servers and chat servers.
- Database servers.

WHERE IN QUIZZY

`server/server.js` runs as one long-running Node process on Render, holding the `rooms` Map and every socket.

Monorepo

DEFINITION

In one sentence: A single Git repository that contains several projects, such as a frontend and a backend.

In more detail: Keeping related code together means one commit can change both sides, and the history lives in one place. Each project still has its own `package.json` and packages, and the hosting platforms are told which folder to build (the "root directory").

REAL-LIFE EXAMPLE

One filing cabinet with a separate drawer for each department.

WHERE REAL APPS USE IT

- Large companies keeping many projects in one repository.
- Full-stack starter projects with `client/` and `server/` folders.

WHERE IN QUIZZY

The QUIZZY repository contains `client/` and `server/`, each with its own `package.json`; Vercel builds `client/` and Render runs `server/`.

Continuous deployment

DEFINITION

In one sentence: Automatically deploying every change pushed to the main branch, with no manual steps.

In more detail: The hosting platform watches the repository, and each push triggers a build and a deployment. It is often combined with automatic tests that run first (continuous integration, CI), so broken code doesn't reach users.

REAL-LIFE EXAMPLE

A newspaper that goes to print automatically as soon as an editor approves a story.

WHERE REAL APPS USE IT

- Most modern web teams.
- Open-source projects publishing a new version when changes are merged.

WHERE IN QUIZZY

Vercel and Render watch the GitHub repository, so a push to `main` redeploys both (section 10.5); QUIZZY has no automatic tests yet.

Production vs preview

DEFINITION

In one sentence: Production is the live version that real users see; a preview is a temporary copy of a change, used for testing before it goes live.

In more detail: Vercel creates a preview deployment with its own URL for each branch or pull request, and updates production only from the main branch. Previews may need their own settings: for example, the server's CORS allowlist won't include preview URLs.

REAL-LIFE EXAMPLE

A dress rehearsal versus opening night.

WHERE REAL APPS USE IT

- Website teams reviewing every change on a preview link.
- Beta versions of phone apps given to testers before release.

WHERE IN QUIZZY

The production client is the Vercel deployment whose address is set as `CLIENT_URL` on Render; a preview URL would be blocked by the CORS check in `server/src/app.js`.

Cold start (free-tier sleep)

DEFINITION

In one sentence: The delay when a sleeping server has to start up before it can answer the first request.

In more detail: Free hosting plans, such as Render's, stop servers that have had no traffic for a while to save resources. The next visitor waits while the server boots, which can take tens of seconds. For QUIZZY, falling asleep also wipes every in-memory room.

REAL-LIFE EXAMPLE

Starting a car on a frosty morning: it takes a while before you can drive off.

WHERE REAL APPS USE IT

- Free-tier hosting of hobby projects.
- Serverless functions that haven't run for a while.

WHERE IN QUIZZY

The free Render instance running `server/server.js` sleeps after about 15 minutes without traffic; opening `QuizListPage` wakes it with `GET /api/quizzes`.

3.10 The technologies QUIZZY uses

Version numbers come from `client/package.json` and `server/package.json`. Socket.IO is defined in 3.2 and JWT in 3.7.

React

DEFINITION

In one sentence: A JavaScript library for building user interfaces out of reusable components.

In more detail: You describe what the screen should look like for the current state, and React updates the real page when the state changes. It compares the new description with the previous one and changes only what is different, which is faster and less error-prone than changing the page by hand. It was created at Facebook (now Meta); QUIZZY uses version 19.

REAL-LIFE EXAMPLE

Instead of repainting a whole wall when one tile changes, you give a decorator a picture of how the wall should look, and they replace only the tiles that differ.

WHERE REAL APPS USE IT

- Facebook and Instagram on the web.
- Countless dashboards and admin panels.

WHERE IN QUIZZY

`client/src/main.jsx` → `createRoot(…).render(…)` starts React; every page in `client/src/pages` and component in `client/src/components` is a React component.

Why QUIZZY uses it: the student screen moves through many stages (join form, lobby, question, result, finished, final rank, session ended), and React makes "show the screen that matches the current state" simple: `PlayerPage` is a chain of `if (finalResult) … if (closedMessage) … if (currentQuestion) …` early returns.

Vite

DEFINITION

In one sentence: A tool that runs your frontend while you develop it and packages it for production.

In more detail: During development, `npm run dev` starts a local server that updates the page instantly when you save a file. For production, `npm run build` bundles and minifies everything into `client/dist`. Only environment variables whose names start with `VITE_` reach browser code, and their values are fixed into the files at build time.

REAL-LIFE EXAMPLE

A print shop: while you write, it shows you instant previews; when you are done, it binds your loose pages into a finished book (the `dist` folder) for the bookshop (Vercel) to sell.

WHERE REAL APPS USE IT

- Modern React and Vue projects.
- Many newer web frameworks use Vite under the hood.

WHERE IN QUIZZY

`client/vite.config.js` loads the `react()` and `tailwindcss()` plugins; `client/src/config/config.js` reads `import.meta.env.VITE_API_URL`.

Why QUIZZY uses it: fast development, built-in JSX support through `@vitejs/plugin-react`, the Tailwind plugin, and environment variables through `import.meta.env`.

COMMON MISTAKE

Changing a `VITE_` variable in the Vercel dashboard and expecting the live site to change. Because the values are baked in at build time, you must **redeploy**. The repository has a commit literally named `chore: trigger Vercel deploy`, which is exactly this situation.

React Router

DEFINITION

In one sentence: A library that decides which React page to show for each web address.

In more detail: It provides client-side routing: `<Route path="/join/:code">` shows `PlayerPage` for any URL like `/join/123456`, `useParams()` reads URL parameters, `useNavigate()` changes the page from code, and `<Navigate>` redirects while drawing. QUIZZY uses version 7 (`react-router-dom`).

REAL-LIFE EXAMPLE

A building directory that tells you which room to go to, without making you leave the building.

WHERE REAL APPS USE IT

- Many React single-page applications.
- Admin dashboards with many pages.

WHERE IN QUIZZY

`client/src/main.jsx` wraps the app in `<BrowserRouter>`; `client/src/App.jsx` declares every route (section 5).

Why QUIZZY uses it: shareable links. The QR code on the projector is simply the URL `https://<your-site>/join/123456`; React Router reads the room code straight from it.

Tailwind CSS

DEFINITION

In one sentence: A styling toolkit where you style elements by adding small ready-made class names, such as `rounded-2xl` or `bg-white`.

In more detail: Instead of writing CSS rules in a separate file, you combine "utility classes" on each element. Version 4 is configured in CSS with an `@theme` block, where custom design values (colours, fonts) automatically become classes. Prefixes like `sm:` and `lg:` apply styles only on wider screens, and `motion-reduce:` respects reduced motion. Classes you never use aren't shipped.

REAL-LIFE EXAMPLE

Dressing from a rack of standard clothing pieces instead of sewing a custom outfit for each person.

Where the analogy breaks down: Two pieces of the same kind clash: if an element has both `text-lg` and `text-2xl`, the winner is decided by their order in Tailwind's generated CSS, not by the order you wrote them.

WHERE REAL APPS USE IT

- Many modern React and Next.js projects.
- Ready-made UI component kits built on Tailwind.

WHERE IN QUIZZY

`client/src/index.css` (`@import "tailwindcss"`, the `@theme` design values and animations); `client/src/components/Button.jsx` builds its variants from Tailwind classes.

Why QUIZZY uses it: fast, consistent styling with shared design values such as `--color-answer-red`, and responsive prefixes for the phone-first student screens and projector-first host screens.

Node.js

DEFINITION

In one sentence: A program that runs JavaScript outside the browser, so JavaScript can power servers.

In more detail: Node.js is a runtime built on the same JavaScript engine as Google Chrome. It handles many connections at once with a single thread and an event loop, and it comes with npm, the largest collection of reusable packages. QUIZZY's server uses the older CommonJS module style (`require`).

REAL-LIFE EXAMPLE

The same language (JavaScript) moving to a new country (the server) and getting a new set of tools there.

WHERE REAL APPS USE IT

- Web servers and APIs at companies of every size.
- Developer tools such as Vite and the npm command itself.

WHERE IN QUIZZY

`server/package.json` → `"start": "node server.js"` ; `server/server.js` is the entry point on Render.

Why QUIZZY uses it: the same language on both sides, an excellent WebSocket library (Socket.IO), and, because there is only one thread, the in-memory `rooms` Map is never changed by two pieces of code at the same instant: each event handler runs until its next `await` before another starts.

Express (with cors and dotenv)

DEFINITION

In one sentence: A small Node.js library for building web servers and APIs by declaring routes and middleware.

In more detail: You write `app.use(…)` for middleware and `router.get(…)` or `router.post(…)` for routes. Express 5 automatically passes errors from `async` handlers to its error handling. Two helper packages go with it in QUIZZY: `cors` , a middleware that adds the CORS headers, and `dotenv` , which loads a local `.env` file into `process.env` (on Render there is no `.env` file, so it simply does nothing).

REAL-LIFE EXAMPLE

A mail-sorting room: every letter passes the same stations (middleware) before reaching the right desk (route handler).

WHERE REAL APPS USE IT

- Countless Node.js APIs, from small projects to large companies.
- Backends for single-page and phone apps.

WHERE IN QUIZZY

`server/src/app.js` (`cors(…)` , `express.json()` , `app.use("/api/quizzes", quizRouter)`); `server/src/routes/quiz.routes.js` ; `server/server.js` line 1 calls `require("dotenv").config()` .

Why QUIZZY uses it: a small, well-known way to offer the four quiz endpoints, with `requireAuth` applied once to the whole router.

Supabase and PostgreSQL

DEFINITION

In one sentence: Supabase is a hosted service that gives you a PostgreSQL database, user logins and a ready-made API, without running any servers yourself.

In more detail: PostgreSQL ("Postgres") is a popular open-source relational database. Supabase runs it for you and adds an authentication service, an automatic REST API for your tables, the `@supabase/supabase-js` client library (so you write `supabase.from("quizzes").select(…)` instead of raw SQL), a dashboard, and two kinds of keys.

REAL-LIFE EXAMPLE

Postgres is a well-organised filing cabinet; Supabase is the office around it, with a receptionist who fetches files (the API), a security desk issuing ID badges (Auth), and keys deciding who may open which drawer.

WHERE REAL APPS USE IT

- Many startups and side projects build their whole backend on Supabase.
- PostgreSQL itself is one of the most widely used databases, from small apps to large companies.

WHERE IN QUIZZY

`server/src/db/supaBase.js` (server client with the secret key) and `client/src/supabase/supabaseClient.js` (browser client with the publishable key); all queries are in `server/src/services/quiz.service.js` and `session.service.js` .

Why QUIZZY uses it: one managed service provides both the database and user accounts, with a free tier and no database server to run. QUIZZY uses the publishable key in the browser (login only) and the secret key on the server (all database access); see section 7.

Supabase Auth

DEFINITION

In one sentence: Supabase's ready-made login system: it stores accounts, checks passwords and hands out login tokens.

In more detail: In the browser, `supabase.auth.signUp` and `signInWithPassword` create a session (an access token plus a refresh token), which supabase-js stores and renews automatically. On the server, `supabase.auth.getUser(token)` asks Supabase whether a token is valid and whose it is. Projects can require users to confirm their email before logging in.

REAL-LIFE EXAMPLE

The security desk of an office building, which checks your ID once and gives you a badge you show everywhere else.

WHERE REAL APPS USE IT

- Apps built on Supabase.
- Similar services such as Firebase Authentication and Auth0 do the same job for other apps.

WHERE IN QUIZZY

`client/src/pages/LoginPage.jsx → handleSubmit`; `client/src/auth/AuthContext.jsx → AuthProvider`; `server/src/middlewares/requireAuth.js` and `server/src/sockets/socketAuth.js` call `getUser`.

Why QUIZZY uses it: hosts need accounts to own quizzes, and building secure password storage and token signing yourself is risky and slow. Students deliberately need no account.

Zod

DEFINITION

In one sentence: A JavaScript library for checking that data has the shape you expect, with clear error messages when it doesn't.

In more detail: You describe valid data once as a schema; `schema.safeParse(input)` then returns either `{ success: true, data }` with cleaned data or `{ success: false, error }` with readable messages. `z.object` also drops unknown fields by default, which blocks mass assignment.

REAL-LIFE EXAMPLE

An airport baggage sizer: the bag either fits the frame or it doesn't, and you are told exactly which side is too big.

WHERE REAL APPS USE IT

- Checking form input and API requests in many JavaScript and TypeScript projects.
- Checking configuration and environment variables when an app starts.

WHERE IN QUIZZY

`server/src/validation/quiz.schema.js` (`questionSchema`, `quizSchema`), `server/src/validation/ai.schema.js` (`generateRequestSchema`), and `server/src/services/ai.service.js → generatedSchema`.

Why QUIZZY uses it: never trust input. Zod checks the quizzes hosts save, the AI generation requests, and the **AI's own output**. The same `questionSchema` is used for human-written and AI-written questions, so both follow identical rules.

Google Gemini (@google/genai)

DEFINITION

In one sentence: Google's family of AI models, which QUIZZY uses to write quiz questions.

In more detail: Gemini models are large language models. The `@google/genai` library calls them with `ai.models.generateContent(…)`. QUIZZY uses structured output (a JSON Schema plus `responseMimeType: "application/json"`) so the answer arrives in a predictable shape, and picks the model through the `GEMINI_MODEL` environment variable.

REAL-LIFE EXAMPLE

A very well-read assistant who fills in a printed form for you, instead of writing a free-form letter.

WHERE REAL APPS USE IT

- Google's Gemini app and the AI features in Google's products.
- Developers' own apps, which call the same models through Google's API.

WHERE IN QUIZZY

`server/src/services/ai.service.js → generateQuestions`, `buildPrompt`, `callWithRetry`; called from `server/src/controllers/quiz.controller.js → generateQuiz` and, in the browser, from `client/src/pages/CreateQuizPage.jsx → handleGenerate`.

Why QUIZZY uses it: writing quizzes by hand is slow; the host types a topic, difficulty and number of questions, then reviews the generated questions before saving. Because the model name is an environment variable, it can be changed without touching the code.

Vercel

DEFINITION

In one sentence: A hosting platform that builds websites straight from GitHub and serves them quickly all over the world.

In more detail: Vercel runs your build on every push, serves the static result from its CDN, can run serverless functions, and creates preview deployments for branches. Settings such as rewrites live in `vercel.json`, and environment variables are set in its dashboard.

REAL-LIFE EXAMPLE

A chain of vending machines restocked automatically: perfect for handing out pre-packed items instantly, but you can't hold a long conversation with a vending machine.

WHERE REAL APPS USE IT

- Hosting for Next.js sites and many other frontends.
- Preview links for every pull request in many web teams.

WHERE IN QUIZZY

`client/vercel.json` sends every path to `index.html`; Vercel hosts the built client.

Why QUIZZY uses it: free, automatic deployments of the static React build. Only the client is hosted there; section 10 explains why the server is not.

Render

DEFINITION

In one sentence: A hosting platform that keeps your server program running all the time and gives it a public HTTPS address.

In more detail: You connect a repository and give a build command and a start command; Render installs, starts and restarts the program, sets the `PORT` variable, and redeploys on every push. Free web services sleep after a period without traffic.

REAL-LIFE EXAMPLE

Renting a small shop with a permanent shopkeeper (your Node process) who can talk to customers for as long as they like, unlike the vending machine.

WHERE REAL APPS USE IT

- Hosting APIs and background workers for small teams and side projects.
- Hosting small databases alongside them.

WHERE IN QUIZZY

Runs `server/` with `npm start` (`node server.js`); `server/server.js` listens on `process.env.PORT`.

Why QUIZZY uses it: Socket.IO needs a process that stays running and keeps the connections and the `rooms` Map in memory.

qrcode.react (QR codes)

DEFINITION

In one sentence: A React component that draws a QR code for any text, right in the browser.

In more detail: A QR code is a square barcode that phone cameras can read; it usually contains a URL. The library works out the pattern of squares and draws it as SVG, so it stays sharp at any size, even on a projector. Its error-correction setting lets the code still scan when part of it is blurred or covered.

REAL-LIFE EXAMPLE

A printed address label that a phone camera can read instead of a person.

WHERE REAL APPS USE IT

- Restaurant menus and payment codes.
- Joining a Wi-Fi network or opening an event ticket.

WHERE IN QUIZZY

`client/src/pages/HostPage.jsx` → `<QRCodeSVG value={joinLink} … />` in the host lobby.

motion (animation library)

DEFINITION

In one sentence: An animation library for React that makes elements move smoothly, including sliding into new positions when a list reorders.

In more detail: Writing `<motion.li layout>` makes an element remember where it was; when its position changes, motion measures the old and new positions and animates the difference. `<MotionConfig reducedMotion="user">` switches movement off for users who ask for reduced motion. It is the successor to the library known as Framer Motion.

REAL-LIFE EXAMPLE

Runners' names on a race board sliding past each other as positions change, instead of jumping.

WHERE REAL APPS USE IT

- Animated menus and page transitions in React apps.
- Drag-and-drop lists that reorder smoothly.

WHERE IN QUIZZY

`client/src/components/Leaderboard.jsx` → `motion.li layout` for the rows, the podium animation, and `MotionConfig`.

Why QUIZZY uses these two: students join by scanning the projector instead of typing a URL, and leaderboard rows slide when ranks change, which makes the live board easy to follow from the back of a

classroom.

INTERVIEW TIP

For every technology, interviewers want the *reason*, not just the definition. Practise one line each: "Socket.IO because the host's screen must react to other people's actions", "Zod because I validate the AI's output too, not just user input", "Render instead of Vercel for the server because WebSockets need a long-running process."

4. File map

This section lists every file in `client/src` and `server/src` (plus the two entry files `server/server.js` and `client/index.html`), what each one is responsible for, which local files it imports, and which files import it. The `.kilo/worktrees/` folder in the repository is a separate copy of the project made by an editor tool and is ignored here.

Key terms here (defined in section 3): Module (import and export) · npm package (dependency)

4.1 Client files (client/src)

File	Responsibility	Imports (local)	Imported by
`../index.html`	The single HTML page; loads Google Fonts (Fredoka, Nunito) and `/src/main.jsx`	main.jsx	Vite / Vercel
`main.jsx`	Entry point: renders `<App>` inside `<BrowserRouter>` and `<AuthProvider>`; imports global CSS	auth/AuthContext, App.jsx, index.css	index.html
`App.jsx`	Declares all routes (section 5)	all 6 pages, auth/RequireAuth, App.css	main.jsx
`index.css`	Tailwind import, `@theme` tokens (colors, fonts, shadow, animations), keyframes, body gradient	-	main.jsx
`App.css`	Empty (kept from the Vite template)	-	App.jsx
`api.js`	`apiFetch(path, options)`: `fetch` to `API_URL` with JSON header and the current Bearer token	supabase/supabaseClient, config/config	QuizListPage, CreateQuizPage
`hostSocket.js`	Socket.IO client for hosts; its `auth` callback sends the current access token at every (re)connection	config/config, supabase/supabaseClient	HostPage
`socket/socket.js`	Socket.IO client for students; no token (anonymous)	config/config	PlayerPage
`config/config.js`	`API_URL` = `VITE_API_URL`, falling back to `http://localhost:3000`	-	api, hostSocket, socket/socket; also imported (unused) by QuizListPage, CreateQuizPage

File	Responsibility	Imports (local)	Imported by
`supabase/supabaseClient.js`	Browser Supabase client (URL + publishable key); throws on startup if either variable is missing	-	api, hostSocket, auth/AuthContext, LoginPage
`auth/AuthContext.jsx`	`AuthProvider` keeps `session`, `user`, `loading`, `signOut`; `useAuth()` hook reads them	supabase/supabaseClient	main, RequireAuth, LoginPage, QuizListPage
`auth/RequireAuth.jsx`	Route guard: spinner while loading, redirect to `/login` (remembering `from`) if logged out	AuthContext, components/CenteredPage, components/Spinner	App
`pages/HomePage.jsx`	Landing page: 6-digit code input (digits only) → navigate to `/join/:code`; link to the host dashboard	components/Button	App
`pages/LoginPage.jsx`	Sign up / log in form; redirects to `from` (or `/host`) when done or already logged in	supabase/supabaseClient, auth/AuthContext, Button, Spinner, Card, Input, ErrorMessage, CenteredPage	App
`pages/QuizListPage.jsx`	Loads `GET /api/quizzes`; shows quiz cards, "Create a new quiz", log out	config (unused), auth/AuthContext, api, Button, Spinner, Card, CenteredPage	App
`pages/CreateQuizPage.jsx`	Quiz editor: title, questions, 4 options, correct answer; AI generation; saves with `POST /api/quizzes`	config (unused), api, Button, Spinner, AnswerShape, Card, Input, ErrorMessage, theme	App
`pages/HostPage.jsx`	Host socket screen: resume check, create room, lobby with QR code, start, live leaderboard, end, close room	components/Lobby, Leaderboard, hostSocket, Button, Spinner, Card, CenteredPage	App
`pages/PlayerPage.jsx`	Student socket screen: join form → lobby → questions → finished → final result / session ended	socket/socket, Lobby, QuestionView, Button, WaitingDots, Card, Input, ErrorMessage, CenteredPage, theme	App
`components/Lobby.jsx`	Player pills; student view ("You're in!") when given `nickname`, host view otherwise	WaitingDots, theme	HostPage, PlayerPage

File	Responsibility	Imports (local)	Imported by
`components/QuestionView.jsx`	Progress bar, question text, 2×2 answer tiles with shapes, correct/wrong reveal	AnswerShape, theme	PlayerPage
`components/Leaderboard.jsx`	Live rows (rank, name, progress, score) with `motion` layout animation; podium on final results	theme (+ npm `motion/react`)	HostPage
`components/Button.jsx`	Chunky button: `variant`, `size`, `as` (render as a Link)	-	all 6 pages
`components/Card.jsx`	White card with chunky shadow; `as` changes the element	-	Login, QuizList, CreateQuiz, Host, Player pages
`components/Input.jsx`	Styled input or select (`as="select"`)	-	Login, CreateQuiz, Player pages
`components/ErrorMessage.jsx`	Red alert box with icon and `role="alert"`	-	Login, CreateQuiz, Player pages
`components/CenteredPage.jsx`	Full-height centered `<main>` layout	-	RequireAuth, Login, QuizList, Host, Player
`components/Spinner.jsx`	Loading ring (still under reduced motion)	-	RequireAuth, Login, QuizList, CreateQuiz, Host
`components/WaitingDots.jsx`	Three bouncing dots	-	Lobby, PlayerPage
`components/AnswerShape.jsx`	Triangle, diamond, circle, square SVGs	-	QuestionView, CreateQuizPage
`components/theme.js`	Shared constants: `answerStyles` (color + shape per slot), `medals`	-	Lobby, QuestionView, Leaderboard, CreateQuizPage, PlayerPage
`assets/hero.png`, `react.svg`, `vite.svg`	Leftovers from the Vite template	-	nothing (unused)

4.2 Server files (server/)

File	Responsibility	Imports (local)	Imported by
`server.js`	Loads `.env`; creates the HTTP server from the Express app; creates the Socket.IO server (CORS = `CLIENT_URL`); registers socket handlers; listens on `PORT`	src/app.js, src/sockets/socket.js	`npm start`
`src/app.js`	Express app: `cors({ origin: CLIENT_URL })`, `express.json()`, health route `GET /`, mounts `/api/quizzes`	routes/quiz.routes	server.js
`src/db/supaBase.js`	Creates the server Supabase client with the **secret key**; no session persistence; throws on startup if variables are missing	-	requireAuth, socketAuth, quiz.service, session.service
`src/middlewares/requireAuth.js`	Reads `Authorization: Bearer`, verifies with `supabase.auth.getUser`, sets `req.user = { id, email }`; 401/500 otherwise	db/supaBase	routes/quiz.routes
`src/middlewares/aiRateLimit.js`	Max 10 AI generations per user per hour, tracked in an in-memory Map; replies 429 with `Retry-After`	-	routes/quiz.routes (but never reached, see section 14)
`src/routes/quiz.routes.js`	Router: `requireAuth` for all; `GET /`, `POST /`, `GET /:id`, `POST /generate` (registered twice)	controllers/quiz.controller, requireAuth, aiRateLimit	app.js
`src/controllers/quiz.controller.js`	`getQuiz`, `listQuizzes`, `createQuiz`, `generateQuiz`: validate input, call services, choose status codes	services/quiz.service, services/ai.service, validation/quiz.schema, validation/ai.schema	routes/quiz.routes
`src/validation/quiz.schema.js`	Zod `questionSchema` (text 1–300, 2–6 options of 1–100 chars, valid `correctIndex`) and `quizSchema` (title 1–100, 1–50 questions)	-	quiz.controller, ai.service
`src/validation/ai.schema.js`	Zod `generateRequestSchema`: topic 3–100 chars, difficulty easy/medium/hard, count integer 1–15	-	quiz.controller

File	Responsibility	Imports (local)	Imported by
`src/services/quiz.service.js`	`isValidId` (UUID check), `getQuizById`, `listQuizzes`, `createQuiz` (RPC `create_quiz`), `toPublicQuiz` (unused)	db/supaBase	quiz.controller, sockets/socket
`src/services/ai.service.js`	Gemini client; `buildPrompt`; `callWithRetry` (retry on 429/500/503 with exponential backoff); `generateQuestions` (JSON schema, Zod check, up to 2 attempts)	validation/quiz.schema	quiz.controller
`src/services/room.service.js`	The in-memory `rooms` Map and all game rules: `roomService` (create), `getRoom`, `addPlayer`, `startQuiz`, `submitAnswer`, `getLeaderboard`, `removePlayer`, `endQuiz`, `closeRoom`, `findRoomByHost`	services/question.service	sockets/socket
`src/services/question.service.js`	`getPublicQuestion` (question *without* the answer), `getOptionCount`, `getCorrectAnswer`, `isCorrect`	-	room.service, sockets/socket
`src/services/session.service.js`	`saveSessionResults`: maps the leaderboard to plain results and calls RPC `save_session_results`	db/supaBase	sockets/socket
`src/sockets/socket.js`	`registerSocketHandler(io)`: auth middleware, `emitLeaderboard`, every event handler, 60-second host grace timer	room, quiz, question, session services; sockets/socketAuth	server.js
`src/sockets/socketAuth.js`	Socket.IO middleware: if the handshake has a token, verify it and set `socket.data.userId`; otherwise allow as anonymous	db/supaBase	sockets/socket
`src/routes/room.routes.js`, `src/controllers/room.controller.js`	Entirely commented out (an early HTTP "create room" endpoint)	-	nothing (dead code)
`src/data/questions.js`	Empty (once held hardcoded test questions)	-	nothing (dead code)

4.3 Dependency diagrams

Arrows point from a file to the file it imports. To keep the client diagram readable, the six small presentational components (`Button`, `Card`, `Input`, `ErrorMessage`, `CenteredPage`, `Spinner`) are drawn

as one "UI kit" box.

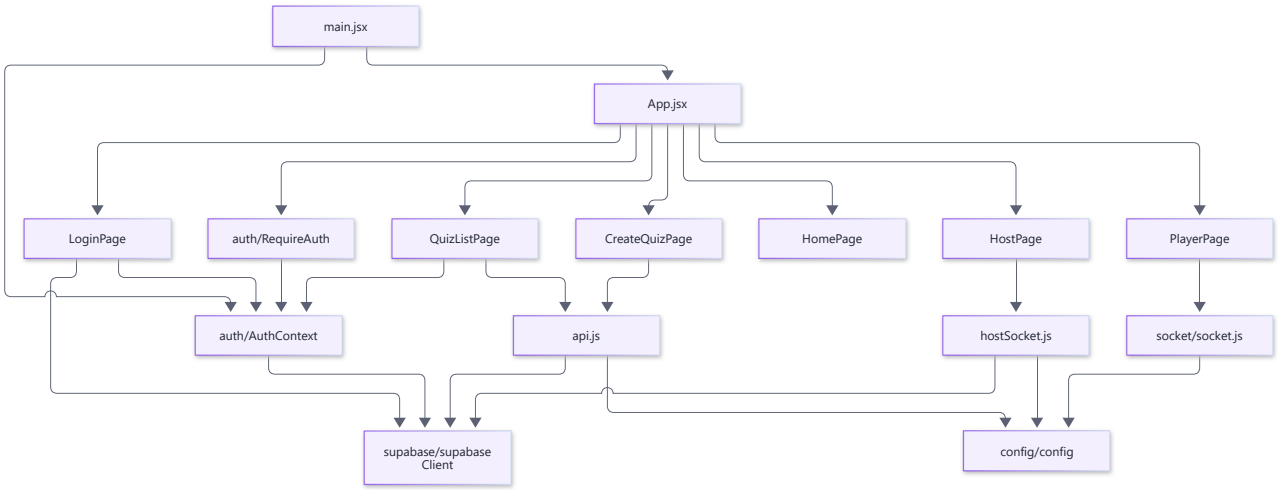


Figure 4.1a: Client import graph, part 1: entry, auth and the three connection helpers (REST, host socket, student socket).

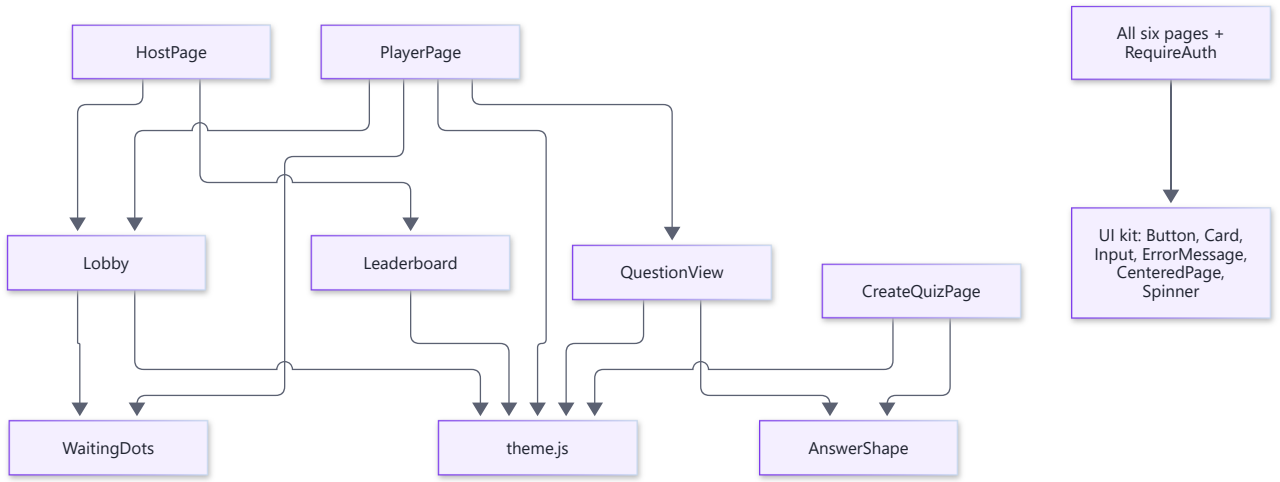


Figure 4.1b: Client import graph, part 2: pages and their components.

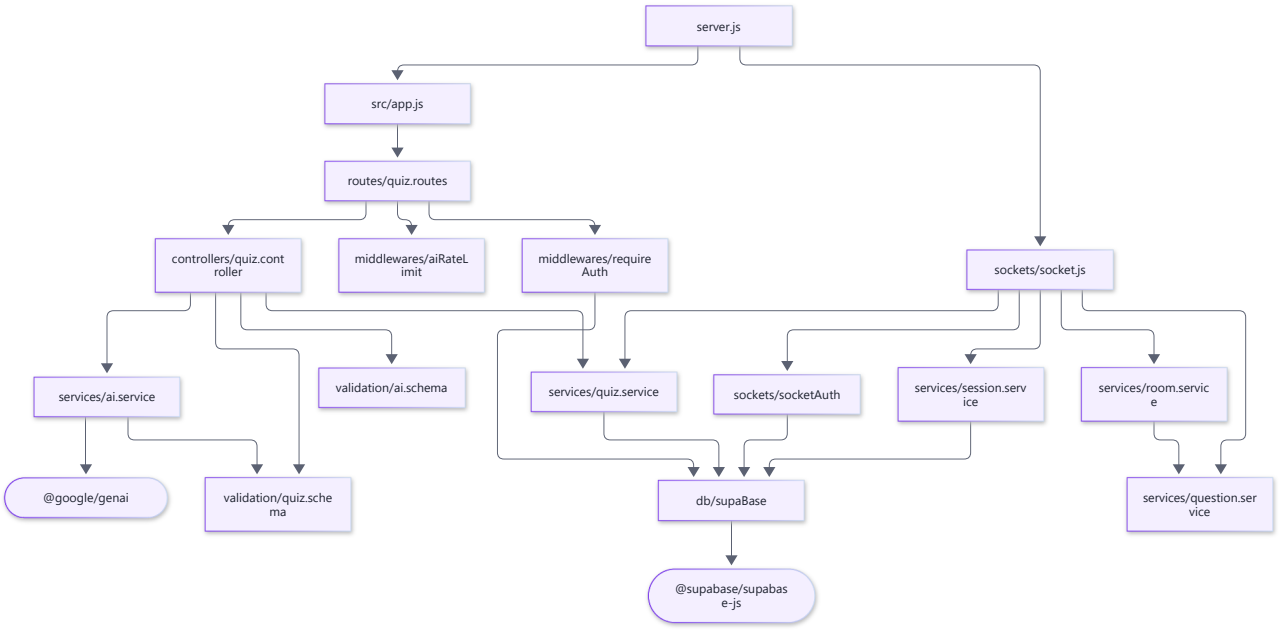


Figure 4.2: Server import graph. Rounded boxes are npm packages.

KEY IDEA

Only **four** server files touch the database (`requireAuth` , `socketAuth` , `quiz.service` , `session.service`), and only **one** file (`room.service.js`) owns the live game state. If you know those five files, you know where every piece of data lives.

COMMON MISTAKE

Missing or mismatched exports. `quiz.routes.js` still contains `console.log("controller exports:", Object.keys(quizController))` , a leftover from debugging exactly this: if a controller function is not listed in `module.exports` , `router.get("/", quizController.listQuizzes)` receives `undefined` and Express crashes on startup. On the client, mixing up default and named imports (`import apiFetch from "../api"` instead of `import { apiFetch } from "../api"`) gives `undefined` at run time. Also remember to **save** the file: an unsaved editor tab is not what Vite or nodemon runs.

5. Pages and routes

All routes are declared in `client/src/App.jsx`. Three of them are wrapped in `<RequireAuth>`, which shows a spinner while the session is loading and redirects to `/login` if nobody is logged in.

Route	Component	Login?	What it shows	APIs / socket events used
`/`	`HomePage`	No	Room code box (digits only, exactly 6) and a link to the host dashboard	None. Navigates to `/join/:code`
`/login`	`LoginPage`	No	Log in / sign up form. If already logged in, redirects to `from` or `/host`	Supabase Auth: `signUp`, `signInWithPassword` (directly from the browser)
`/host`	`QuizListPage`	Yes	The host's quizzes as cards; "Create a new quiz"; log out	`GET /api/quizzes`; `signOut`
`/host/new`	`CreateQuizPage`	Yes	AI generation card, title, question cards, sticky Save bar	`POST /api/quizzes/generate`, `POST /api/quizzes`
`/host/:quizId`	`HostPage`	Yes	Checking → Create room → Lobby (code, link, QR) → Live leaderboard → Final results	Host socket. Sends `host:resume`, `host:create-room`, `quiz:start`, `quiz:end`, `host:close-room`. Receives `room:resumed`, `host:no-room`, `room:created`, `room:players`, `leaderboard:update`, `quiz:error`, `host:error`
`/join/:code`	`PlayerPage`	No	Join form → Lobby → Question → Finished → Final rank; or "Session ended"	Student socket. Sends `player:join`, `quiz:answer`. Receives `room:players`, `player:join:error`, `quiz:question`, `quiz:answer:result`, `quiz:answer:error`, `quiz:ended`, `room:closed`, `connect_error`
`*`	`<Navigate to="/">`	No	Any unknown path redirects to the home page	-

5.1 How a page decides what to show

Neither `HostPage` nor `PlayerPage` uses sub-routes for their stages. Instead they return different JSX depending on state, checked in a fixed order. This is a simple **state machine**:

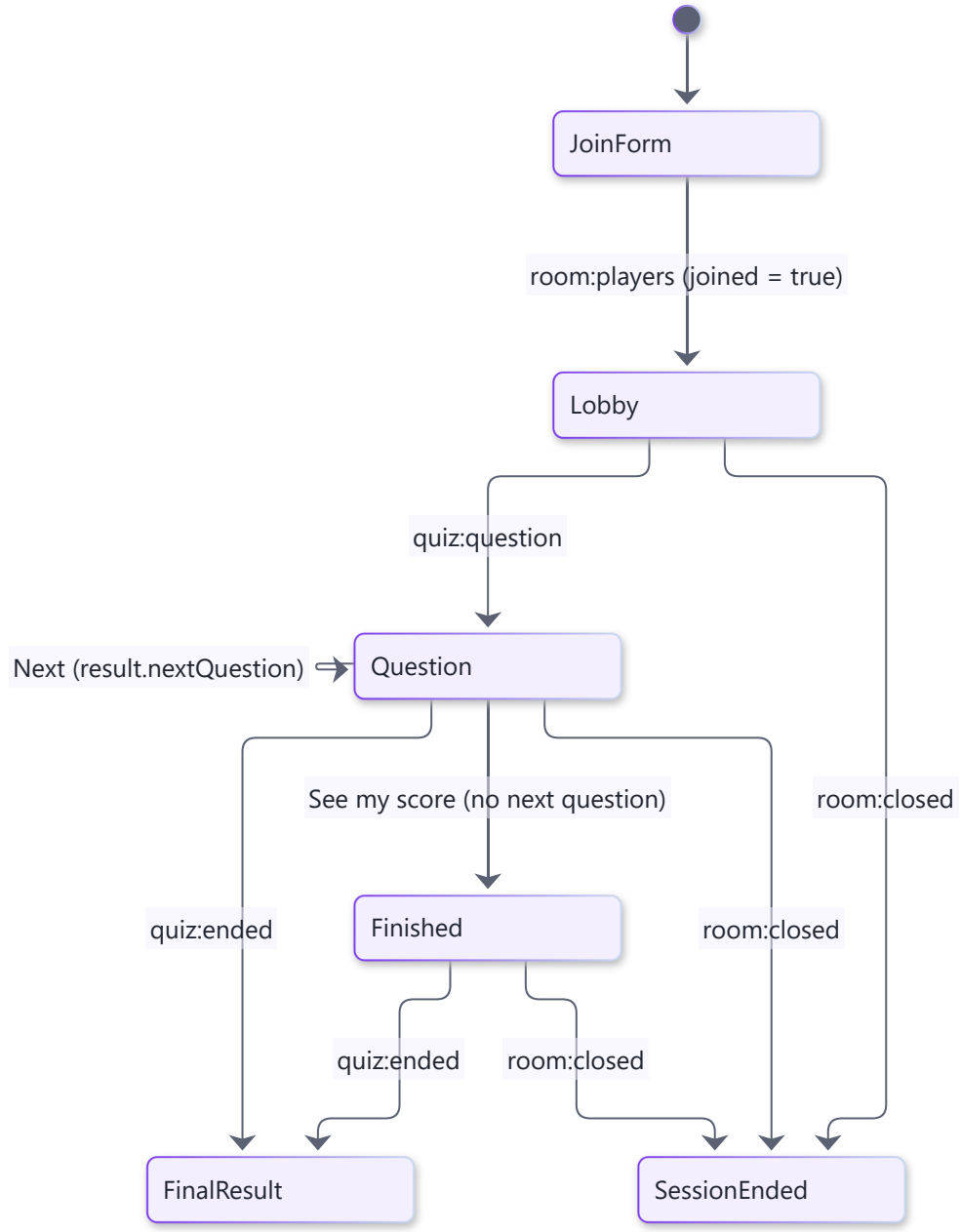


Figure 5.1: The student screen (`PlayerPage`). The order of the `if` checks is finalResult → closedMessage → finished → currentQuestion → joined → join form.

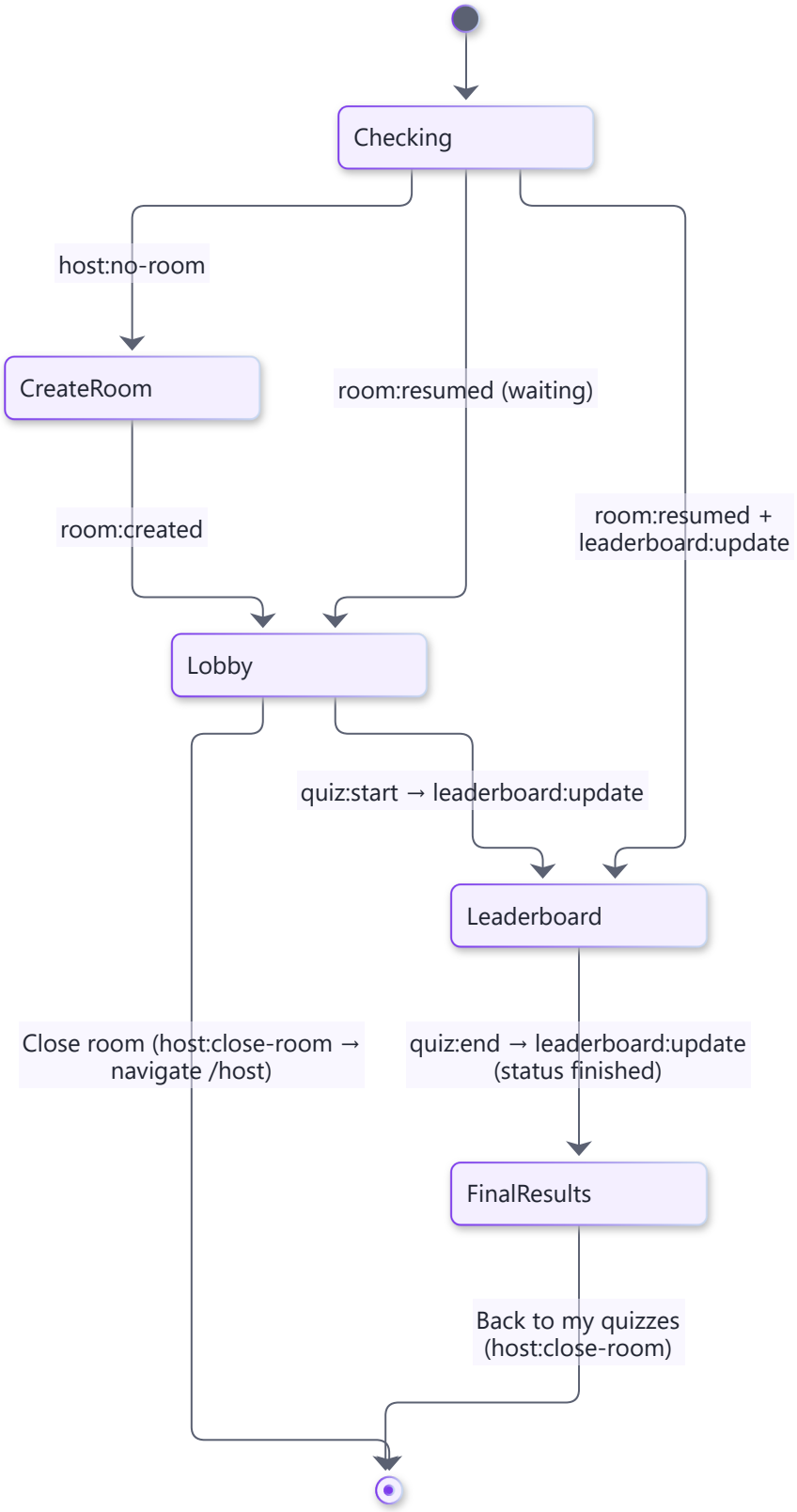


Figure 5.2: The host screen (`HostPage`). Order of checks: checking → no roomCode → leaderboard → lobby.

KEY IDEA

The URL only says *which* quiz or room; the **socket events** decide which stage of the game is on screen. That is why refreshing `/host/:quizId` works: the page reconnects, asks the server "do I have a room?" (`host:resume`) and rebuilds its state from the answer.

COMMON MISTAKE

Forgetting `client/vercel.json`. Without its rewrite rule, opening `/join/123456` directly (which is exactly what a QR scan does) asks Vercel for a file called `/join/123456`, which doesn't exist, and returns a 404. The rewrite serves `index.html` for every path so React Router can handle it.

INTERVIEW TIP

If asked "how do you protect routes in React?", explain that `RequireAuth` is only a **user-experience** guard: it hides pages from logged-out users. The real protection is on the server (`requireAuth` and `socketAuth`), because anyone can change JavaScript running in their own browser.

6. How the frontend talks to the backend

The client talks to the server in two ways. **REST over HTTPS** is used for things that happen once and return an answer: list quizzes, save a quiz, generate questions with AI. **Socket.IO over WSS** is used for the live game, where the server must push updates the moment other people act.

> **KEY IDEA**
>
> Use request/response (REST) when *the user asks for something*. Use a persistent socket when *the server needs to tell you something you didn't ask for*, like "a student just joined" or "Ana just answered question 4".

6.1 REST: apiFetch and the Authorization header

Every REST call from the client goes through one small helper, so the token and base URL are never forgotten:

`client/src/api.js → apiFetch`

```
export const apiFetch = async (path, options = {}) => {
  const { data } = await supabase.auth.getSession();
  const token = data.session?.access_token;

  return fetch(`${API_URL}${path}`, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      ...options.headers,
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
    },
  });
};
```

- `API_URL` comes from `client/src/config/config.js` (`VITE_API_URL`), e.g. the Render address.
- `supabase.auth.getSession()` is called **on every request**. supabase-js refreshes the access token automatically when it is about to expire, so each request carries a fresh token.
- The token goes in the standard header `Authorization: Bearer <JWT>`. "Bearer" means "whoever bears (holds) this token is the user".
- `apiFetch` returns the raw `Response` ; the pages call `res.json()` and check `res.ok` themselves (e.g. `CreateQuizPage → handleSubmit`).

6.2 CORS and CLIENT_URL

Key terms here (defined in section 3): Origin · CORS (Cross-Origin Resource Sharing) · Preflight request · Request header

In production, the page comes from Vercel and the API is on Render: two different origins. So the server must allow the Vercel origin, and it does so in two places, both read from the `CLIENT_URL` environment variable:

server/src/app.js and server/server.js

```
// Express (REST)
app.use(cors({ origin: process.env.CLIENT_URL || "http://localhost:5173" }));

// Socket.IO (its first requests are HTTP long-polling, which CORS also governs)
const io = new Server(httpServer, { cors: { origin: CLIENT_URL } });
```

COMMON MISTAKE

- **Trailing slash or wrong scheme**: `CLIENT_URL=https://quizzy.vercel.app/` does not match the origin `https://quizzy.vercel.app`, so every request fails with a CORS error in the browser console.
- **Localhost fallback in production**: if `CLIENT_URL` is not set on Render, both lines fall back to `http://localhost:5173`, and the deployed site is blocked. The same happens on the client: if `VITE_API_URL` is missing at build time, `config.js` falls back to `http://localhost:3000` and the live site tries to call the user's own computer.
- **Vercel preview URLs**: each preview deployment has a different domain, which won't match the single `CLIENT_URL`.

SECURITY

CORS stops *other websites* from making your users' browsers call your API and read the results. It is **not** authentication: a script outside a browser can still call the API. That is why every quiz endpoint also requires a valid token (section 6.3).

6.3 requireAuth middleware

server/src/middlewares/requireAuth.js → requireAuth

```
const header = req.headers.authorization || "";
const [scheme, token] = header.split(" ");
if (scheme !== "Bearer" || !token) {
  return res.status(401).json({ error: "Not logged in" });
}
const { data, error } = await supabase.auth.getUser(token);
if (error || !data.user) {
  return res.status(401).json({ error: "Invalid or expired session" });
}
req.user = { id: data.user.id, email: data.user.email };
next();
```

`quiz.routes.js` applies it to the whole router with `router.use(requireAuth)`, so every `/api/quizzes` route is protected and every controller can trust `req.user.id`. If Supabase itself can't be reached, the

`catch` replies `500 "Could not verify session"` instead of wrongly letting the request through.

6.4 Every REST endpoint

Method + path	Auth	Input and validation	Success response	Error responses
`GET /`	No	-	200, text "QUIZZY Server is running!"	-
`GET /api/quizzes` listQuizzes	Yes	None. Filters by `creator_id = req.user.id`	200, `[{ id, title, createdAt, questionCount }]`, newest first	401 not logged in / invalid; 500 database error
`POST /api/quizzes` createQuiz	Yes	Body `{ title, questions: [{ text, options[], correctIndex }] }` checked by `quizSchema`: title 1–100 chars; 1–50 questions; text 1–300; 2–6 options of 1–100 chars; `correctIndex` inside the options. Strings are trimmed; unknown fields dropped	201, `{ id }` of the new quiz	400 `{ error }` with the first Zod message; 401; 500 "Could not save quiz"
`GET /api/quizzes/:id` getQuiz	Yes	`:id` must match the UUID pattern (`quiz.service → isValidId`)	200, `{ id, title, creatorId, questions: [{ text, options, correctIndex }] }`	400 invalid id; **404** if missing *or owned by someone else*; 401; 500. Not called by the client today.
`POST /api/quizzes/generate` generateQuiz	Yes (+ rate limit, intended)	Body `{ topic, difficulty, count }` checked by `generateRequestSchema`: topic 3–100 chars; difficulty easy / medium / hard; count integer 1–15	200, `{ questions: [...] }` (not saved: the host reviews, then saves with `POST /api/quizzes`)	400; 401; 502 "Could not generate questions"; 429 is *designed* but currently unreachable (section 14)

Also from Express itself: a body that is not valid JSON gets a 400, and a body over 100 kB (the `express.json()` default limit) gets a 413.

Key terms here (defined in section 3): HTTP status codes · HTTP methods (GET and POST) · REST

6.5 Socket.IO: two client sockets

The client creates two separate Socket.IO connections, each in its own module. Both use `autoConnect: false`, so they only connect when a page calls `socket.connect()` in its `useEffect`, and disconnect in the cleanup function when the page unmounts.

	Student socket	Host socket
File	`client/src/socket/socket.js`	`client/src/hostSocket.js`
Used by	`PlayerPage`	`HostPage`
Sends a token?	No (students are anonymous)	Yes: `auth: async (cb) => cb({ token })`
Server identity	`socket.data.userId = null`; the player is identified by `socket.id`	`socket.data.userId` = the Supabase user ID

Why two? Students have no Supabase session, so their socket simply connects without a token and the server treats it as anonymous. The host's socket needs the token so the server can check ownership. Keeping them as two modules also means each page owns exactly one connection with its own listeners, so host and student code never interfere (for example when testing both roles in two tabs of the same browser). Because the host `auth` option is a **function**, Socket.IO calls it at every connection and reconnection, so a reconnect after an hour picks up a freshly refreshed token instead of an expired one.

`client/src/hostSocket.js`

```
const hostSocket = io(API_URL, {
  autoConnect: false,
  auth: async (cb) => {
    const { data } = await supabase.auth.getSession();
    cb({ token: data.session?.access_token ?? null });
  },
});
```

6.6 Handshake authentication and socket.data

Key terms here (defined in section 3): Handshake · Middleware · Singleton

`socket.data` is a per-connection object where the server keeps its own facts about that socket; the browser can't read or change it. `io.use(fn)` registers a connection middleware that runs before the `connection` event.

`server/src/sockets/socketAuth.js → socketAuth`

```
socket.data.userId = null;
const token = socket.handshake.auth?.token;
if (!token) return next();                  // anonymous student: allowed
const { data, error } = await supabase.auth.getUser(token);
if (!error && data.user) socket.data.userId = data.user.id;
next();                                     // never rejects the connection
```

Note the design: `socketAuth` **never refuses** a connection. It only labels it. Each host-only event handler then checks `socket.data.userId` itself. The fields stored on `socket.data` are:

Field	Set by	Meaning
`userId`	`socketAuth`	Verified Supabase user ID, or `null` for students and invalid tokens
`roomCode`	`host:create-room`, `host:resume`, `player:join`	The room this socket belongs to (cleared by `host:close-room`)
`role`	same handlers	`"host"` or `"player"`; decides what `disconnect` does
`creatingRoom`	`host:create-room`	Temporary flag while the quiz is loaded from the database, to block double clicks

COMMON MISTAKE

The token is checked **only at the handshake**. If the host's access token expires an hour later, the existing socket stays authenticated until it reconnects. For a classroom quiz this is acceptable, but it is worth knowing (and saying in an interview).

6.7 Socket rooms used by QUIZZY

Socket rooms are named groups of connections on the server (remember: a "room" in this guide is QUIZZY's game object; a "socket room" is the Socket.IO group). QUIZZY uses three kinds, and choosing the right one decides who receives each message:



Figure 6.1: The four ways QUIZZY addresses messages, who is in each group, and which events use it.

KEY IDEA

Four ways to send: `socket.emit` (only the sender), `io.to(socketId)` (one specific other socket, because every socket automatically joins a room named after its own ID), `io.to(code)` (everyone in the game) and `io.to("host:" + code)` (only the host's screens). The leaderboard goes to `host:CODE` so students never receive the whole class's scores through it, and so several host tabs all stay in sync.

Key terms here (defined in section 3): Acknowledgement · Emit · Broadcast

6.8 Every Socket.IO event

QUIZZY defines **7 events sent by clients** and **13 events sent by the server**, plus Socket.IO's built-in `connect`, `connect_error` and `disconnect`. All server handlers are inside `server/src/sockets/socket.js → registerSocketHandler`.

Events sent by the client

Event	Sent by	Payload	Server handler does	Sent back (and to whom)
`host:resume`	Host socket, on every `connect` (`HostPage → handleConnect`)	none	Needs `userId`; finds the host's room with `findRoomByHost`; cancels a pending close timer; joins `CODE` and `host:CODE`; sets `socket.data`	`host:error` (not logged in) → sender; `host:no-room` → sender; `room:resumed { code, quizTitle, players }` → sender; if the quiz already started, also `leaderboard:update` → `host:CODE`
`host:create-room`	Host (`handleCreateRoom`)	`{ quizId }`	Checks: logged in; this socket has no room and isn't already creating one; the user has no other room; valid UUID. Loads quiz from DB; checks socket still connected, ownership, at least one question, and no room again (race check). Creates room with `roomService(userId, quiz)`	`room:created { code, quizTitle }` → sender; or `host:error { message }` → sender
`player:join`	Student (`PlayerPage → handleJoin`)	`{ roomCode, nickname }`	`addPlayer`: room exists and is `waiting`; nickname is a string of 1–20 chars after trimming; same socket not already in; nickname not taken (case-insensitive). Joins the socket to room `CODE`	`player:join:error { message }` → sender; on success `room:players [players]` → `CODE` (all students + host)
`quiz:start`	Host (`handleStartQuiz`)	`{ roomCode }`	`startQuiz(code, socket.data.userId)`: room exists, requester is the host, status `waiting`, at least one player. Sets status `playing`, `startedAt`	`quiz:error` → sender; on success, for each player `quiz:question` (question 1, no answer) → `player.id`; `leaderboard:update` → `host:CODE`

Event	Sent by	Payload	Server handler does	Sent back (and to whom)
`quiz:answer`	Student (`handleAnswer`)	`{ roomCode, questionIndex, optionIndex }`	`submitAnswer(code, socket.id, …)`: quiz is `playing`; sender is a player; not finished; `questionIndex` equals the player's current one; option is an in-range integer. Adds 10 points if correct; advances the index; marks finished after the last one	`quiz:answer:error { message }` → sender; or `quiz:answer:result { correct, correctOption, score, nextQuestion }` → sender (`nextQuestion` is `null` when finished), and `leaderboard:update` → `host:CODE`
`quiz:end`	Host (`handleEndQuiz`, after a confirm dialog)	`{ roomCode }`	`endQuiz`: requester is the host; status `playing`. Sets `finished`; computes the leaderboard; saves results with `saveSessionResults`	`quiz:error` → sender; for each player `quiz:ended { rank, score, totalPlayers }` → `player.id`; `leaderboard:update` (status `finished`) → `host:CODE`; if saving fails, `host:error` → sender
`host:close-room`	Host ("Close room" / "Back to my quizzes", `handleCloseRoom`)	acknowledgement callback only	If this socket is the room's host: cancels the timer, deletes the room. Always leaves both socket rooms and clears `socket.data`	`room:closed { message }` → `CODE` (only if not finished); then the ack → client navigates to `/host`

Events sent by the server

Event	Payload	Received by	Client handler
`room:created`	`{ code, quizTitle }`	Host	`HostPage` → `handleRoomCreated`: shows the lobby
`room:resumed`	`{ code, quizTitle, players }`	Host	`handleResumed`: restores state, stops "checking"
`host:no-room`	none	Host	`handleNoRoom`: shows "Create room"
`host:error`	`{ message }`	Host	`handleHostError`: `alert(message)`
`quiz:error`	`{ message }`	Host	`handleQuizError`: `alert(message)`

Event	Payload	Received by	Client handler
`room:players`	Array of player objects `{ id, nickname, score, currentQuestionIndex, finished, connected }`	Everyone in `CODE`	Host: `handlePlayers` (lobby). Student: `handlePlayers`, also sets `joined = true`
`leaderboard:update`	`{ status, totalQuestions, players: [{ id, nickname, score, answered, finished, connected, rank }] }`	Host sockets (`host:CODE`)	`handleLeaderboard`: switches to the leaderboard view
`player:join:error`	`{ message }`	Student	`PlayerPage → handleJoinError`: shows the red error
`quiz:question`	`{ index, total, question: { text, options } }`	One student	`handleQuestion`: shows question, resets selection
`quiz:answer:result`	`{ correct, correctOption, score, nextQuestion }`	One student	`handleAnswerResult`: reveal; the "Next" button then shows `nextQuestion` locally (`handleNext`)
`quiz:answer:error`	`{ message }`	One student	`handleAnswerError`: shows error, clears selection
`quiz:ended`	`{ rank, score, totalPlayers }`	One student	`handleQuizEnded`: final rank screen
`room:closed`	`{ message: "The host has ended the session" }`	Everyone in `CODE`	`handleRoomClosed`: "Session ended" screen

Built-in events

Event	Where	What QUIZZY does
`connect`	Host client	Emits `host:resume`. This runs on the first connection *and* after every automatic reconnection, which is how a host recovers after a network blip
`connect_error`	Student client	Only `console.error`; the student sees nothing (section 14)
`disconnect`	Server	**Player**: `removePlayer`; in the lobby they are removed and `room:players` goes to `CODE`; during or after the game they are marked `connected: false` and the leaderboard is re-sent. **Host**: if no other host socket remains in `host:CODE`, start a 60-second timer; when it fires, send `room:closed` (unless finished) and delete the room

`server/src/sockets/socket.js → emitLeaderboard (used by start, answer, end, resume, disconnect)`

```
const emitLeaderboard = (room) => {
  io.to(`host:${room.code}`).emit("leaderboard:update", {
    status: room.status,
    totalQuestions: room.quiz.questions.length,
    players: roomService.getLeaderboard(room),
  });
};
```

SECURITY

`quiz:question` and `nextQuestion` are built by `question.service.js → getPublicQuestion`, which copies only `text` and `options`. The `correctIndex` never leaves the server before the student has answered that question. Without this, any student could open the browser's developer tools, read the WebSocket messages, and see every correct answer.

INTERVIEW TIP

When asked "how does the server know who to send things to?", walk through the four targets: `socket.emit`, `io.to(socketId)`, `io.to(code)`, `io.to("host:"+code)`, and give one QUIZZY example of each. It shows you understand socket rooms rather than just "broadcasting".

7. Database

The repository contains **no SQL files or migrations**: the tables and the two Postgres functions were created directly in the Supabase dashboard. The tables, columns, types and foreign keys below were **checked against the Supabase Table Editor**. The *bodies* of the two Postgres functions, column defaults, indexes and RLS policy details are not visible there; those parts are marked **Inferred**. Exporting the schema as SQL migrations into the repository would close that gap (section 12).

7.1 Quizzes in the database, live sessions in memory

QUIZZY keeps two very different kinds of data in two different places:

	Permanent data	Live game data
What	Host accounts, quizzes, questions, final results	Rooms: code, players, scores, each player's position, status, timers
Where	Supabase Postgres	The `rooms` Map in `server/src/services/room.service.js`
Lifetime	Until deleted	Until the room is closed, or the server restarts or sleeps
Speed	A network round trip per query (tens of milliseconds)	Instant (a memory lookup)
Written	When a quiz is saved; once at the end of a game	On every join, answer and disconnect

When a host creates a room, the server loads the whole quiz **once** from the database (`quiz.service → getQuizById`) and stores it inside the room object. From then on, answering a question never touches the database: `submitAnswer` checks the answer against `room.quiz.questions` in memory. When the host ends the quiz, the results are written **once** with `session.service → saveSessionResults`.

`server/src/services/room.service.js → roomService (shape of a room)`

```
const room = {
  code,          // "483920", 6 random digits, unique among live rooms
  hostId,        // Supabase user ID of the host
  quiz,          // { id, title, creatorId, questions: [{ text, options, correctIndex }] }
  players: [],   // { id: socket.id, nickname, score, currentQuestionIndex, finished, connected }
  status: "waiting",   // → "playing" → "finished"
  startedAt: null,     // set by startQuiz, saved with the results
  closeTimer: null,    // the 60-second host grace timer, if running
};
rooms.set(code, room);
```

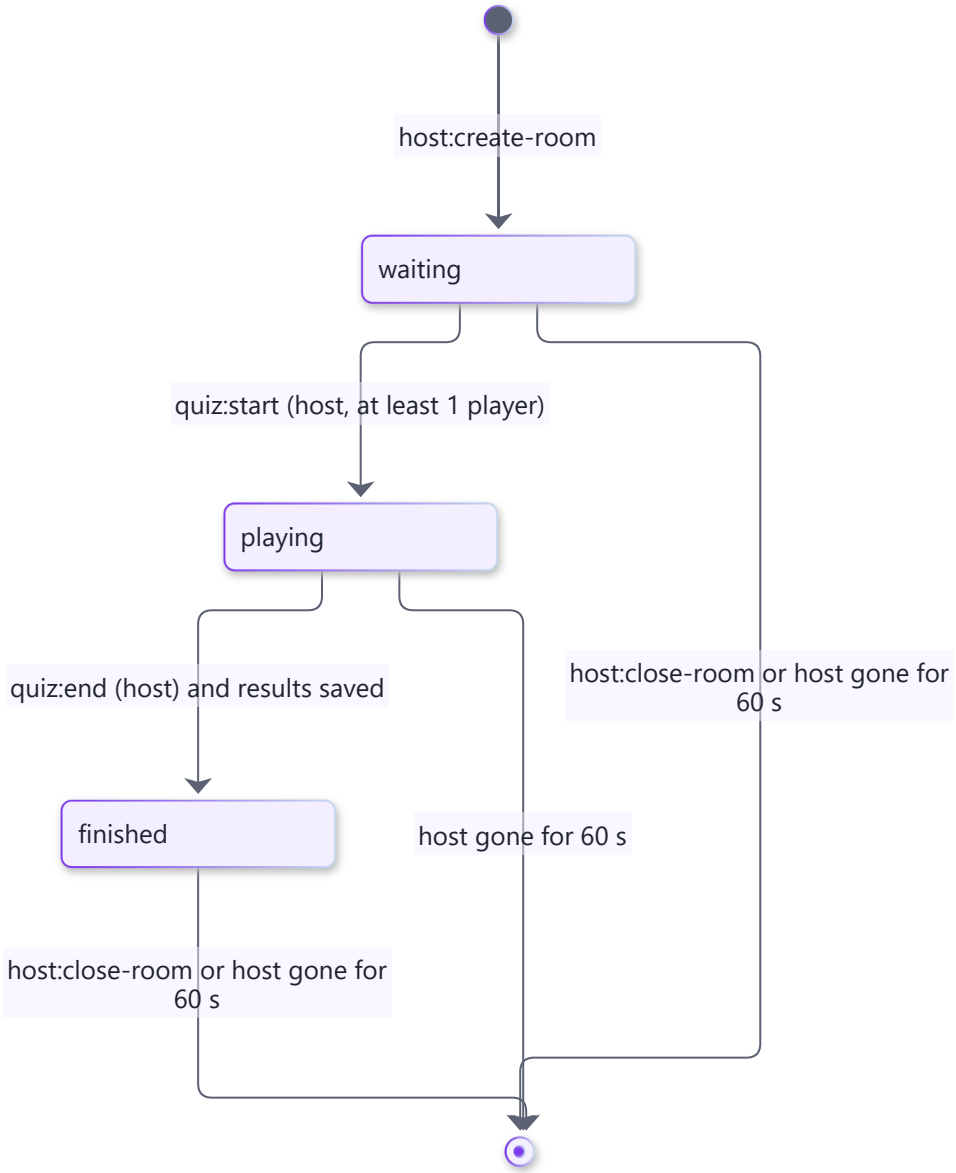


Figure 7.1: Life of a room in memory. "[*]" at the end means `rooms.delete(code)` (`closeRoom`).

REAL-LIFE EXAMPLE

The database is the school's filing cabinet with the printed exam papers. The in-memory room is the whiteboard in the exam hall where the teacher tallies scores during the exam. At the end, the final tally is copied into the filing cabinet; then the whiteboard is wiped.

KEY IDEA

Load once, play in memory, save once. This keeps every answer fast and keeps database traffic tiny (two queries per game), at the cost that a server restart in the middle of a game loses that game.

7.2 Tables and columns

The `public` schema has four tables. Every table uses a `uuid` primary key named `id`.

Table	Column	Type	Meaning / used by
`quizzes`	`id`	`uuid` PK	Quiz ID; checked by `isValidId` before any query
	`title`	`text`	Quiz title
	`created_at`	`timestamptz`	Creation time; the quiz list is ordered by it (newest first). Filled by a default, since `create_quiz` is not given a time **Inferred**
	`creator_id`	`uuid` FK	The host who owns the quiz; a foreign key pointing to the Supabase user (`auth.users.id`) **Inferred target** . Every ownership check compares this with the token's user ID
`questions`	`id`	`uuid` PK	Question ID (selected, not used by the game)
	`quiz_id`	`uuid` FK → `quizzes.id`	Which quiz it belongs to; this foreign key is what lets `getQuizById` embed `questions(...)` in one query
	`position`	`int`	Order inside the quiz, **starting at 0**
	`text`	`text`	The question
	`options`	`jsonb`	The answer options as a JSON array of strings, e.g. `["wire","integer","real","time"]` ; arrives in JavaScript as a normal array
	`correct_index`	`int`	Position of the correct option (0-based); becomes `correctIndex` in `toQuiz`
`sessions`	`id`	`uuid` PK	One row per finished game
	`quiz_id`	`uuid` FK → `quizzes.id`	Which quiz was played (`p_quiz_id`)
	`room_code`	`text`	The 6-digit code used (`p_room_code`); codes can repeat across sessions
	`started_at`	`timestamptz`	When the host pressed Start (`p_started_at` , from `room.startedAt`)
	`ended_at`	`timestamptz`	When results were saved. The server never sends this value, so it comes from a default or from `now()` inside the function **Inferred**

Table	Column	Type	Meaning / used by
`results`	`id`	`uuid` PK	One row per player per game
	`session_id`	`uuid` FK → `sessions.id`	Which game the row belongs to
	`nickname`	`text`	Player nickname (players have no accounts, so there is no user ID here)
	`score`	`int4`	Final score (multiples of 10)
	`rank`	`int4`	Final rank (equal scores share a rank)
	`answered`	`int`	Number of questions answered
	`finished`	`bool`	Whether the player reached the end before the host ended the quiz

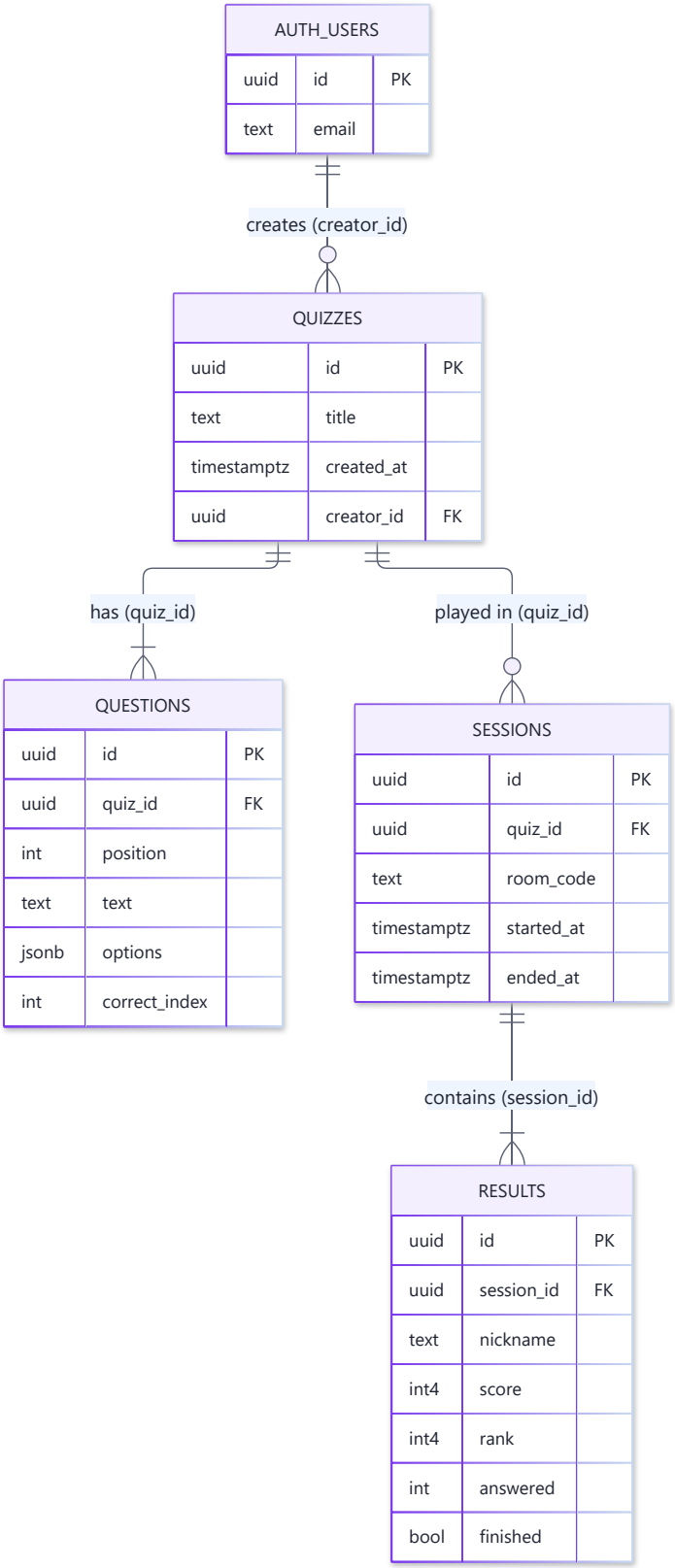


Figure 7.2: Data model. `AUTH_USERS` is Supabase's built-in `auth.users` table (managed by Supabase Auth); the target of `creator_id` is inferred, all other links are foreign keys shown in the Table Editor.

KEY IDEA

Two pairs of tables, the same shape twice: a **parent** with one row per thing (`quizzes` , `sessions`) and a **child** with many rows pointing back to it (`questions` , `results`). Each pair is written in one go by one Postgres function, so a parent never exists without its children.

COMMON MISTAKE

Mixing up naming styles. The database uses `snake_case` (`correct_index` , `creator_id`) while JavaScript uses `camelCase` (`correctIndex` , `creatorId`). `quiz.service → toQuiz` converts on the way out, but on the way in `create_quiz` receives the JavaScript objects as JSON, so inside the function the key is `correctIndex` , not `correct_index` . Getting this wrong silently stores `NULL` or fails the insert.

Key terms here (defined in section 3): Primary key · Foreign key · One-to-many relationship · JOIN · UUID

7.3 Queries: which server function runs what

Server function	Query (supabase-js)	Called from
`quiz.service → listQuizzes(creatorId)`	`from("quizzes").select("id, title, created_at, questions(count)").eq("creator_id", creatorId).order("created_at", { ascending: false })` . `questions(count)` counts each quiz's questions in the same query	`GET /api/quizzes`
`quiz.service → getQuizById(id)`	`from("quizzes").select("id, title, creator_id, questions(id, position, text, options, correct_index)").eq("id", id).order("position", { referencedTable: "questions" }).maybeSingle()` . One query returns the quiz with its questions in order	`GET /api/quizzes/:id` ; socket `host:create-room`
`quiz.service → createQuiz(...)`	`rpc("create_quiz", { p_creator_id, p_title, p_questions })`	`POST /api/quizzes`
`session.service → saveSessionResults(...)`	`rpc("save_session_results", { p_quiz_id, p_room_code, p_started_at, p_results })`	socket `quiz:end`
`requireAuth` , `socketAuth`	`auth.getUser(token)` (Supabase Auth, not a table query)	every REST request; every socket connection with a token

`toQuiz` in `quiz.service.js` converts database names (`snake_case` like `creator_id`) into JavaScript names (`camelCase` like `creatorId`), so the rest of the server never sees database column names.

SECURITY

supabase-js sends every value as a parameter, never by pasting text into SQL, so a quiz titled `'); DROP TABLE quizzes; --` is stored as a harmless string. This prevents **SQL injection**.

7.4 Postgres functions and transactions

Key terms here (defined in section 3): Transaction · ACID · Database function (RPC)

Why QUIZZY uses functions here: saving a quiz writes one row into `quizzes` and N rows into `questions`. With two separate `insert` calls from Node, a failure in between (bad data in question 7, network drop) would leave a quiz with no questions. supabase-js cannot run a multi-statement transaction from the client, so the inserts are moved into a Postgres function. The same reasoning applies to `save_session_results` (one game row plus one row per player). It is also one network round trip instead of several.

create_quiz

Called with `p_creator_id` (from the verified token, never from the request body), `p_title` and `p_questions`: the Zod-cleaned array `[{ text, options, correctIndex }]` sent as JSON. It returns the new quiz's ID, which the controller sends back as `201 { id }`. The real function body isn't visible in the Table Editor; a body matching the real tables (0-based `position`, `jsonb` options) would look like this **Illustrative**:

```
create function create_quiz(p_creator_id uuid, p_title text, p_questions jsonb)
returns uuid language plpgsql as $$
declare new_id uuid;
begin
  insert into quizzes (creator_id, title) values (p_creator_id, p_title)
  returning id into new_id;

  insert into questions (quiz_id, position, text, options, correct_index)
  select new_id,
         q.ord - 1,                          -- position starts at 0
         q.item->>'text',
         q.item->'options',                  -- jsonb array, stored as is
         (q.item->>'correctIndex')::int      -- camelCase key, as sent by Node
  from jsonb_array_elements(p_questions) with ordinality as q(item, ord);

  return new_id;
end $$;
```

save_session_results

Called once from the `quiz:end` handler with `p_quiz_id`, `p_room_code`, `p_started_at` (ISO timestamp from `room.startedAt`) and `p_results`, an array of `{ nickname, score, rank, answered, finished }`. Note what `session.service.js` deliberately *leaves out*: the socket IDs and the `connected` flag, which mean nothing after the game. It inserts one row into `sessions` and one row per player into `results`, all in one transaction. A matching body would look like this **Illustrative**:

```
create function save_session_results(p_quiz_id uuid, p_room_code text,
                                     p_started_at timestamptz, p_results jsonb)
returns uuid language plpgsql as $$
declare new_session uuid;
begin
  insert into sessions (quiz_id, room_code, started_at, ended_at)
  values (p_quiz_id, p_room_code, p_started_at, now())
  returning id into new_session;

  insert into results (session_id, nickname, score, rank, answered, finished)
  select new_session, r->>'nickname', (r->>'score')::int, (r->>'rank')::int,
         (r->>'answered')::int, (r->>'finished')::boolean
  from jsonb_array_elements(p_results) as r;

  return new_session;
end $$;
```

If the call fails, the game still ends normally for students and the host gets `host:error` "The quiz ended, but the results could not be saved". Because it is one transaction, a failure never leaves a `sessions` row without its `results`.

7.5 Secret key vs publishable key

	Publishable key	**Secret key**
Variable	`VITE_SUPABASE_PUBLISHABLE_KEY` (Vercel / `client/.env`)	`SUPABASE_SECRET_KEY` (Render / `server/.env`)
Used in	`client/src/supabase/supabaseClient.js`	`server/src/db/supaBase.js`
Is it secret?	No. It is bundled into the JavaScript every visitor downloads	Yes. Never sent to a browser, never committed
Power	Acts as an anonymous or logged-in user; table access is limited by Row Level Security	Full access; **bypasses** Row Level Security
QUIZZY uses it for	Sign up, log in, sessions, sign out	Verifying tokens and every table read/write

(Older Supabase projects call these the *anon* key and the *service_role* key.) The server client is created with `persistSession: false`, `autoRefreshToken: false` and `detectSessionInUrl: false` because a server has no browser storage, URL or logged-in user of its own.

COMMON MISTAKE

Putting the secret key in a `VITE_` variable "to make it work". Anything named `VITE_*` is published inside the client bundle, so anyone could read it and get full control of the database. Another easy slip: naming the variable differently on Render than in code (`SUPABASE_SERVICE_ROLE_KEY` vs `SUPABASE_SECRET_KEY`). `supaBase.js` throws "Missing SUPABASE_URL or SUPABASE_SECRET_KEY" at startup, which is good: it fails loudly instead of half-working.

7.6 Row Level Security (RLS)

Key terms here (defined in section 3): Row Level Security (RLS) · Secret key vs publishable key · Defense in depth

What the code tells us: the browser never queries tables (it only uses `supabase.auth`), and the server uses the secret key, which bypasses RLS anyway. So the safest correct configuration is **RLS enabled with no policies** on every table: the server keeps working, and the publishable key opens nothing.

The Table Editor shows an **"Add RLS policy"** button on all four tables, which is the prompt Supabase shows for tables that have RLS switched on but no policies yet, rather than an "RLS disabled" warning. That suggests QUIZZY already uses exactly this configuration **Likely**; confirm it under *Authentication → Policies*, where each table should show RLS as enabled with no policies.

> **SECURITY**
> Because the publishable key is public, if RLS were *disabled* on `questions`, anyone could call Supabase's REST API directly with that key and download every quiz, **including the correct answers**, completely bypassing QUIZZY's server and its answer hiding. Check this in the Supabase dashboard (Table Editor shows "RLS disabled" warnings).

7.7 Indexes

Key terms here (defined in section 3): Index (database)

Which indexes exist beyond the automatic primary-key ones is not visible in the Table Editor **Unknown**. The queries above would benefit from `quizzes(creator_id, created_at desc)` (the quiz list) and `questions(quiz_id, position)` (loading a quiz in order), and `results(session_id)` for a future "past sessions" page. At classroom scale (a few hundred rows) the difference is invisible, but naming them is a good interview answer.

> **INTERVIEW TIP**
> "Why a Postgres function?" is a favourite question. Answer: "Saving a quiz writes to two tables, `quizzes` and `questions`. I needed it to be atomic, and supabase-js can't open a transaction from Node, so I moved the inserts into a Postgres function and call it with RPC. One call, one transaction, all-or-nothing." Then mention that the creator ID comes from the verified token, never from the request body.

8. Authentication and authorization

Key terms here (defined in section 3): Authentication · Authorization · Session · Access token · Refresh token · JWT (JSON Web Token)

8.1 Sign-up and login in the browser

`LoginPage` has one form with two modes (`mode` = `"signin"` or `"signup"`). On submit, `LoginPage → handleSubmit` calls Supabase Auth **directly from the browser**; the QUIZZY server is not involved:

`client/src/pages/LoginPage.jsx → handleSubmit`

```
const { data, error } = isSignup
  ? await supabase.auth.signUp({ email, password })
  : await supabase.auth.signInWithPassword({ email, password });
if (error) { setError(error.message); return; }
navigate(redirectTo, { replace: true });   // redirectTo = location.state?.from ?? "/host"
```

Supabase stores the password (hashed) and returns a **session**. supabase-js saves the session in the browser's `localStorage` and refreshes the access token automatically before it expires. The form also enforces `minLength={6}` on the password, matching Supabase's default minimum.

Email confirmation is a Supabase project setting, not visible in the code:

- **If it is off**: `signUp` returns a session immediately; the host lands on `/host`, logged in.
- **If it is on**: `signUp` succeeds but returns *no* session until the email link is clicked. `handleSubmit` still navigates to `/host`, `RequireAuth` finds no user and sends the host back to `/login` with no explanation (listed in section 14).

8.2 AuthContext and RequireAuth

Key terms here (defined in section 3): Context (React) · Custom hook

- `AuthContext.jsx → AuthProvider`: on mount, reads the saved session with `getSession()` (then sets `loading = false`), and subscribes to `onAuthStateChange` so login, logout and token refreshes update React state everywhere. It exposes `{ session, user, loading, signOut }`.
- `RequireAuth.jsx`: while `loading`, shows a spinner (so a logged-in host isn't wrongly bounced to login during the first instant); if there is no `user`, returns `<Navigate to="/login" replace state={{ from: location.pathname }} />`; otherwise renders the page. `from` lets the login page send the host back where they were going.
- `QuizListPage` shows `user.email` and calls `signOut`, which clears the session; `onAuthStateChange` fires, `user` becomes `null`, and `RequireAuth` redirects to login.

8.3 Sessions and JWTs end to end

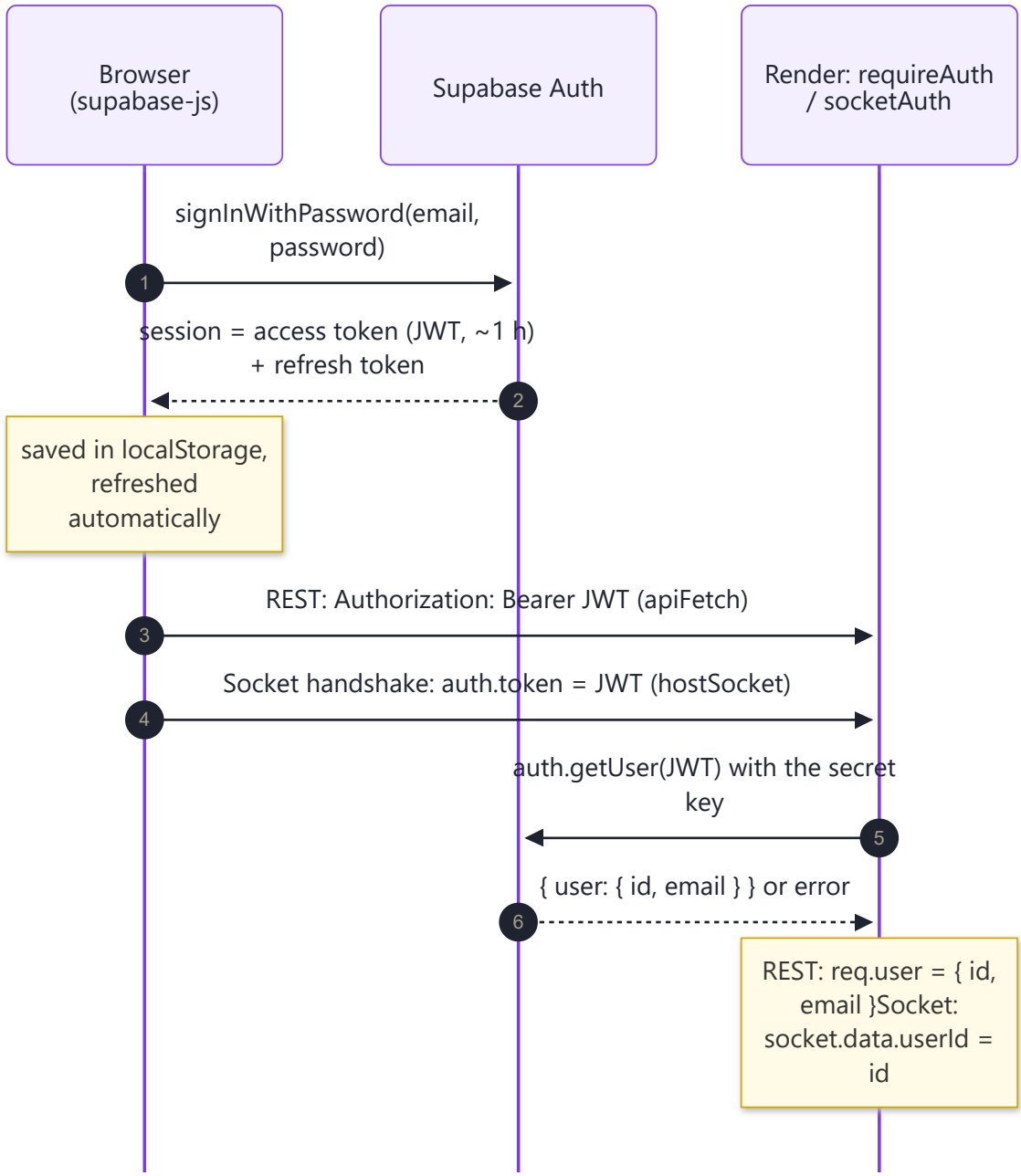


Figure 8.1: One token, two transports. The server never sees the password.

8.4 How hosts are identified

A host is always identified by the **Supabase user ID taken from a verified token**, never by anything the client claims:

- **Creating a quiz**: `quiz.controller → createQuiz` calls `quizService.createQuiz({ creatorId: req.user.id, ...result.data })`. A malicious body containing `creatorId` is harmless only because Zod's `quizSchema` **drops unknown fields**: note that the spread comes *after* `creatorId`, so if the schema ever allowed extra keys, a body value would override the real ID. Putting `creatorId` last would make this safe by construction (section 14).

- **Creating a room**: `roomService(userId, quiz)` stores `hostId = socket.data.userId`.
- **Host-only events** compare `room.hostId` with `socket.data.userId`.
- **Finding your room again**: `findRoomByHost(userId)` loops over all rooms. This is what makes refresh-and-resume work: a new socket, with a new `socket.id`, but the same user ID.

Students, by contrast, are identified only by `socket.id`: a random ID that exists for one connection. That is enough for an anonymous classroom game, but it means a student who loses their connection becomes a new, unknown socket (section 12).

8.5 Every authorization rule

Action	Who may do it	Enforced in	If not allowed
Any `/api/quizzes` request	Logged-in users	`requireAuth` via `router.use`	401
List quizzes	Owner sees only their own	`listQuizzes`: `.eq("creator_id", req.user.id)`	(simply not listed)
Read one quiz (with answers)	Owner only	`getQuiz`: `quiz.creatorId !== req.user.id`	404 "Quiz not found"
Save a quiz	Any logged-in user, as themselves	`createQuiz`: `creatorId: req.user.id`	-
Generate with AI	Any logged-in user; max 10 / hour (intended)	`requireAuth`; `aiRateLimit` (not reached, section 14)	401 / 429
Create a room	Logged-in owner of the quiz; one room per host at a time; quiz must have questions	`socket.js` `host:create-room`	`host:error` "Quiz not found" / "You already have a running room" / ...
Resume a room	Its host (by user ID)	`host:resume` → `findRoomByHost(userId)`	`host:no-room`
Start / end the quiz	The room's host	`room.service → startQuiz`, `endQuiz`: `room.hostId !== requesterId`	`quiz:error` "Only the host can ..."
Close the room	The room's host	`host:close-room`: `role === "host"` and `room.hostId === userId`	Ignored (just acknowledged)
Join a room	Anyone with the code, while the room is `waiting`, with a free 1–20 character nickname	`room.service → addPlayer`	`player:join:error`

Action	Who may do it	Enforced in	If not allowed
Answer	Sockets that joined as players, for their current question only, while `playing`	`room.service → submitAnswer`	`quiz:answer:error`

8.6 Why "not yours" returns 404, not 403

When host A asks for host B's quiz, `getQuiz` answers **404 Not Found**, exactly the same as for a quiz that doesn't exist; the socket handler likewise says "Quiz not found". A **403 Forbidden** would confirm "this quiz exists, you just can't see it", which leaks information and lets an attacker probe which IDs are real (**enumeration**). Returning 404 for both cases reveals nothing. GitHub does the same for private repositories.

SECURITY

Without the ownership check, any logged-in host who obtained another quiz's ID could load it with all the correct answers, or open a room with someone else's quiz. Quiz IDs are random UUIDs and hard to guess, but security should not depend on secrecy of IDs; the ownership check makes guessing pointless.

INTERVIEW TIP

Separate the layers clearly: "Supabase handles authentication and issues JWTs. My server verifies each token with `getUser`, then does authorization itself: ownership of quizzes via `creator_id`, and ownership of rooms via `hostId`. The React `RequireAuth` is only for user experience."

9. End-to-end flows

This section follows real actions through the whole system. Each flow has a **sequence diagram** (time runs downward; arrows are messages) and numbered steps naming the exact file and function. Dashed arrows are replies.

Key terms here (defined in section 3): Sequence diagram

9.1 Host signs up and logs in

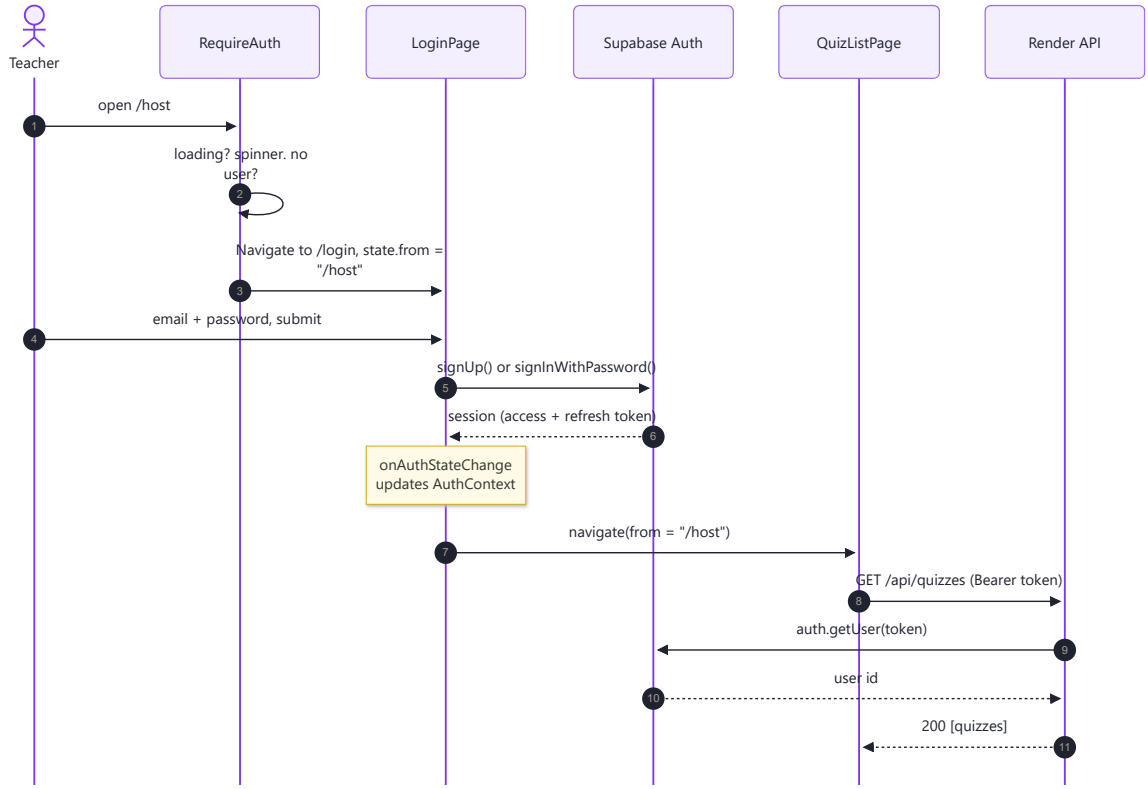


Figure 9.1: Sign up / log in, then the first authenticated request.

1. The teacher opens `/host` (or clicks "Go to your dashboard" on `HomePage`). `App.jsx` wraps that route in `RequireAuth`.
2. `auth/RequireAuth.jsx` reads `useAuth()`. While `AuthProvider` is still loading the saved session it shows a spinner; with no user it redirects to `/login`, remembering `from: "/host"`.
3. `pages/LoginPage.jsx → handleSubmit` calls `supabase.auth.signUp` (sign-up mode) or `signInWithPassword` (log-in mode). Errors such as "Invalid login credentials" are shown with `ErrorMessage`.
4. On success, supabase-js stores the session; `AuthContext.jsx`'s `onAuthStateChange` listener sets the new `session`, and `handleSubmit` navigates to `from`.
5. `QuizListPage`'s `useEffect → loadQuizzes` calls `apiFetch("/api/quizzes")`; the server runs `requireAuth`, then `quiz.controller → listQuizzes` → `quiz.service → listQuizzes(req.user.id)`.

9.2 Host creates a quiz manually

1. `QuizListPage` → "Create a new quiz" links to `/host/new` (`CreateQuizPage`).
2. The page starts with one empty question from `emptyQuestion()`: a random `id` (`crypto.randomUUID()`, used only as a React key), empty text, four empty options and `correctIndex: 0`.
3. Typing calls `updateQuestion` / `updateOption`, which copy the array and change one field (**immutable updates**, so React notices the change). The radio button sets `correctIndex`. `addQuestion` / `removeQuestion` add or filter by `id`.
4. **Save** submits the form: `handleSubmit` sends `POST /api/quizzes` with `{ title, questions }`. The temporary `id` fields are included, but Zod strips them on the server.
5. Server: `requireAuth` → `quiz.controller → createQuiz` → `quizSchema.safeParse(req.body)`. If invalid, the *first* problem is returned as `400 { error }` (e.g. "Options cannot be empty") and shown in the sticky Save bar.
6. If valid, `quiz.service → createQuiz` calls the `create_quiz` Postgres function: one transaction inserts the quiz and all its questions. The controller replies `201 { id }`.
7. The client navigates back to `/host`, which reloads the list.

9.3 Host creates a quiz with AI (generate → review → save)

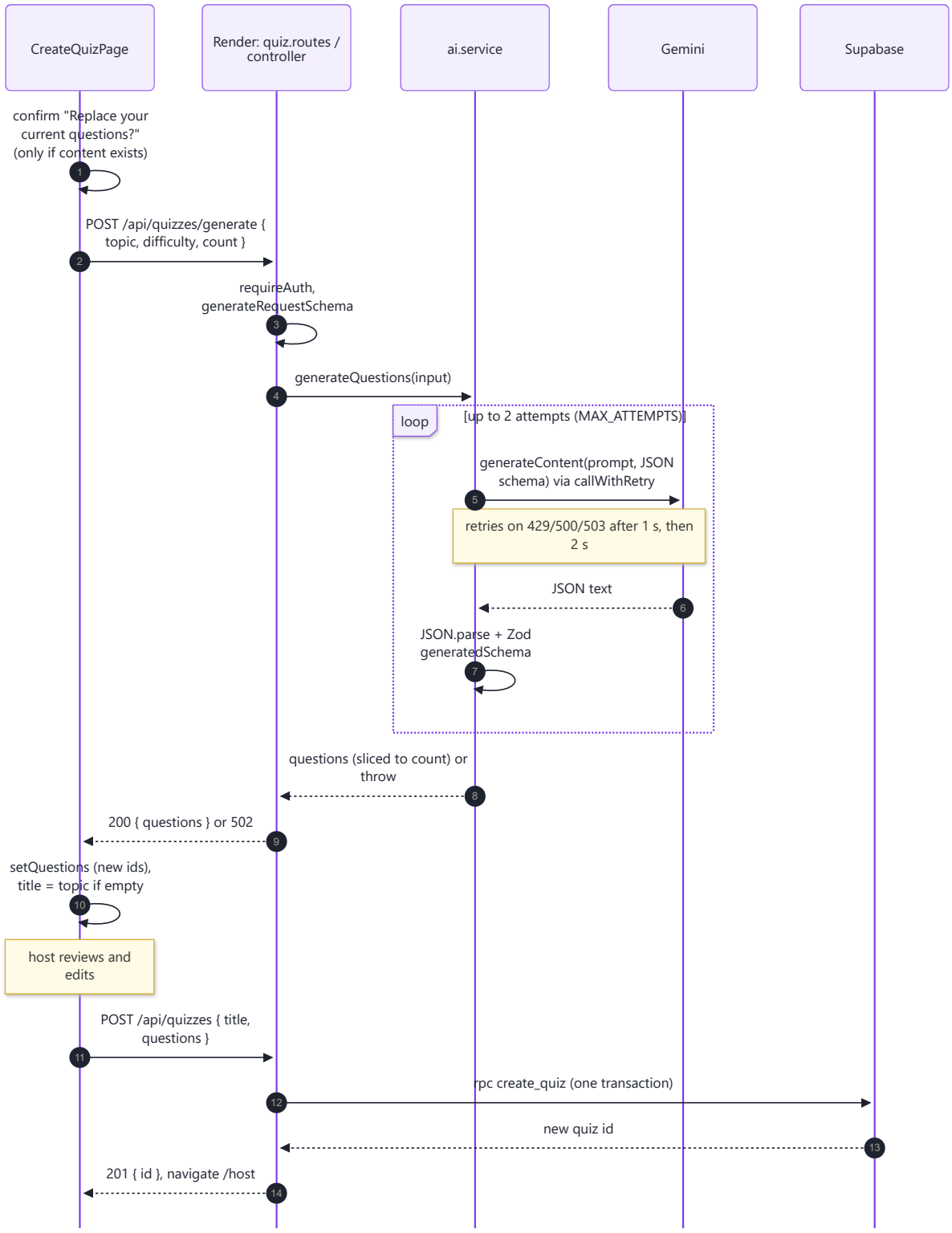


Figure 9.3: AI generation never saves anything by itself; the host always reviews and then saves.

1. `CreateQuizPage → handleGenerate`: if any question already has text, it asks `window.confirm("Replace your current questions with AI-generated ones?")`. It then sets `generating` (spinner, "This can take a few seconds.") and sends `topic`, `difficulty` and `count: Number(aiCount)`.

2. Server: `requireAuth`, then `quiz.controller → generateQuiz` validates with `ai.schema → generateRequestSchema` (topic 3–100 chars, difficulty in the allowlist, count 1–15).
3. `ai.service → generateQuestions` builds the prompt with `buildPrompt`: exactly 4 options, one correct, vary the correct position, plausible distractors, no "all of the above", length limits.
4. It calls `ai.models.generateContent` with `responseMimeType: "application/json"` and `responseJsonSchema` (an object with a `questions` array of `{ text, options, correctIndex }`), wrapped in `callWithRetry`: on HTTP 429, 500 or 503 from Gemini it waits 1 s, then 2 s (**exponential backoff**), up to 2 retries.
5. The response text is parsed with `JSON.parse` and checked with `generatedSchema` (which reuses `questionSchema`). If it fails, a second full attempt is made. After 2 failed attempts it throws "AI returned invalid questions", and the controller replies **502**.
6. On success the questions are cut to the requested `count` (`.slice(0, input.count)`) and returned. Nothing is saved yet.
7. The page replaces its questions (giving each a new React key) and, if the title is empty, uses the topic as the title. The calm note "AI can make mistakes. Review each question…" stays visible.
8. The host edits anything wrong and clicks Save: from here it is exactly flow 9.2, steps 4–7. Human and AI questions pass the **same** `questionSchema`.

> **KEY IDEA**
>
> The AI's output is treated as **untrusted input**: structured output narrows its format, Zod verifies it, invalid output triggers a retry, and a human reviews it before it is ever saved.

9.4 Host creates a room; student joins

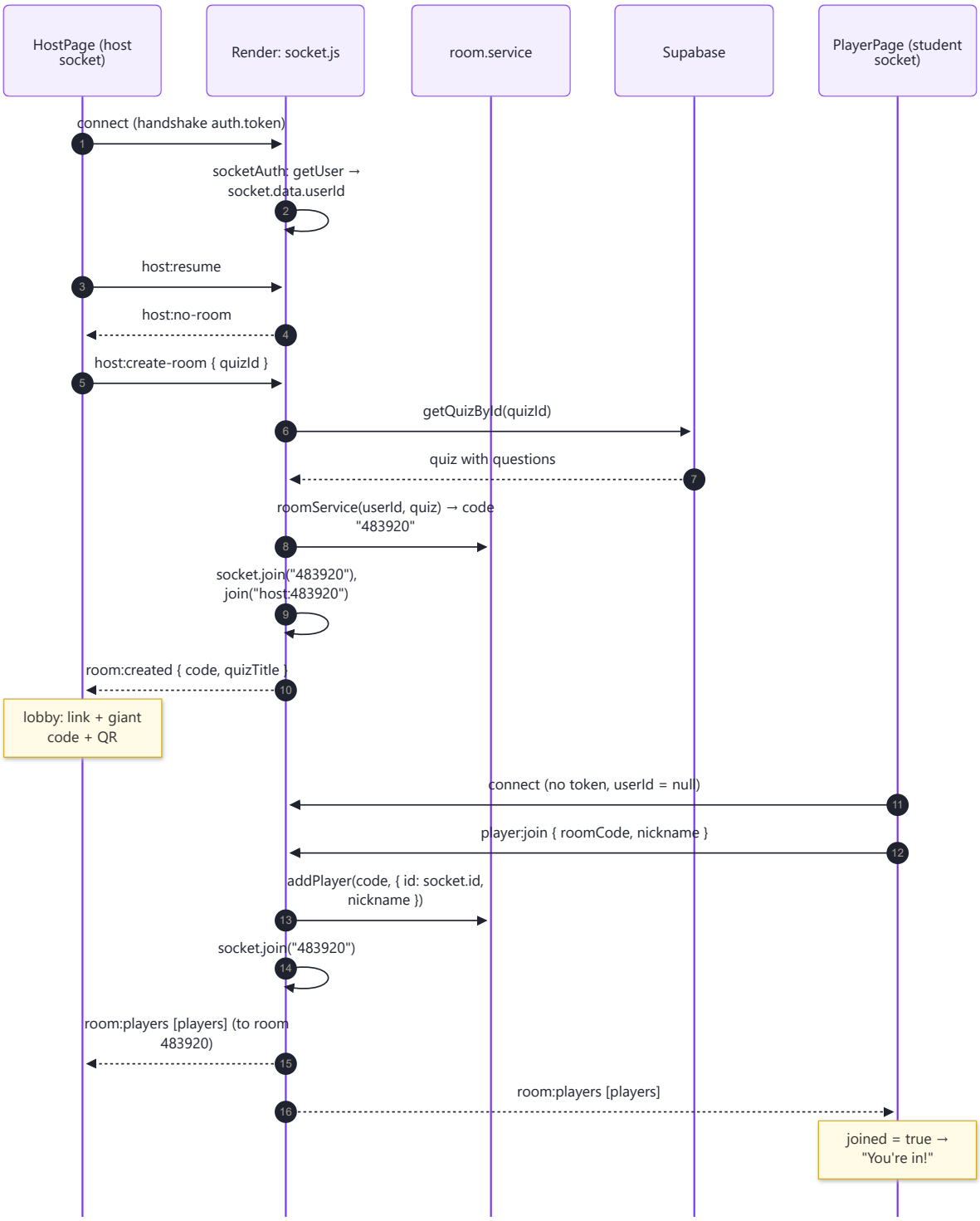


Figure 9.4: Creating a room and joining it.

1. Host clicks **Host** on a quiz card → `/host/:quizId`. `HostPage`'s `useEffect` registers listeners and calls `socket.connect()` on `hostSocket`; the `auth` callback attaches the current token.
2. Server: `socketAuth` verifies the token and sets `socket.data.userId`. On the client, the `connect` event triggers `handleConnect` → `host:resume`. No room exists, so the server answers `host:no-room` and the page shows **Create room**.

3. `handleCreateRoom` emits `host:create-room { quizId }`. The handler in `socket.js` checks login, "already has a room" (on this socket, via the `creatingRoom` flag, and across all rooms with `findRoomByHost`), and the UUID format.
4. It loads the quiz with `quiz.service → getQuizById`. After the `await` it re-checks: is the socket still connected? Does this user own the quiz? Does it have questions? Did another tab create a room meanwhile?
5. `room.service → roomService` creates the room with a random unused 6-digit code (`generateRoomCode` loops until the code is free). The socket joins rooms `CODE` and `host:CODE`; `socket.data.role = "host"`.
6. `room:created` → `handleRoomCreated` stores the code and title. The lobby shows `joinLink = window.location.origin + "/join/" + code`, the giant code and `<QRCodeSVG value={joinLink} />`.
7. **Student, via QR:** the phone camera opens `https://<site>/join/483920`. Vercel's rewrite serves `index.html`; React Router renders `PlayerPage` with `code` from `useParams()`. **Via the home page:** `HomePage` accepts digits only (`replace(/\D/g, "")`), enables Join at exactly 6 digits, and `handleJoin` navigates to `/join/:code`.
8. `PlayerPage` connects the anonymous student socket. The student types a nickname; `handleJoin` trims it, refuses an empty one locally, and emits `player:join { roomCode: code, nickname }`.
9. `room.service → addPlayer` validates (room exists and is waiting, nickname 1–20 chars and not taken case-insensitively, socket not already in) and pushes `{ id: socket.id, nickname, score: 0, currentQuestionIndex: 0, finished: false, connected: true }`. Errors go back as `player:join:error` (e.g. "That nickname is already taken").
10. On success the socket joins room `CODE` and the server sends `room:players` to the whole room. The student's `handlePlayers` sets `joined = true` (that is how the student knows the join worked) and shows the lobby; the host's lobby gets a new pill.

9.5 Quiz starts; students answer; live leaderboard

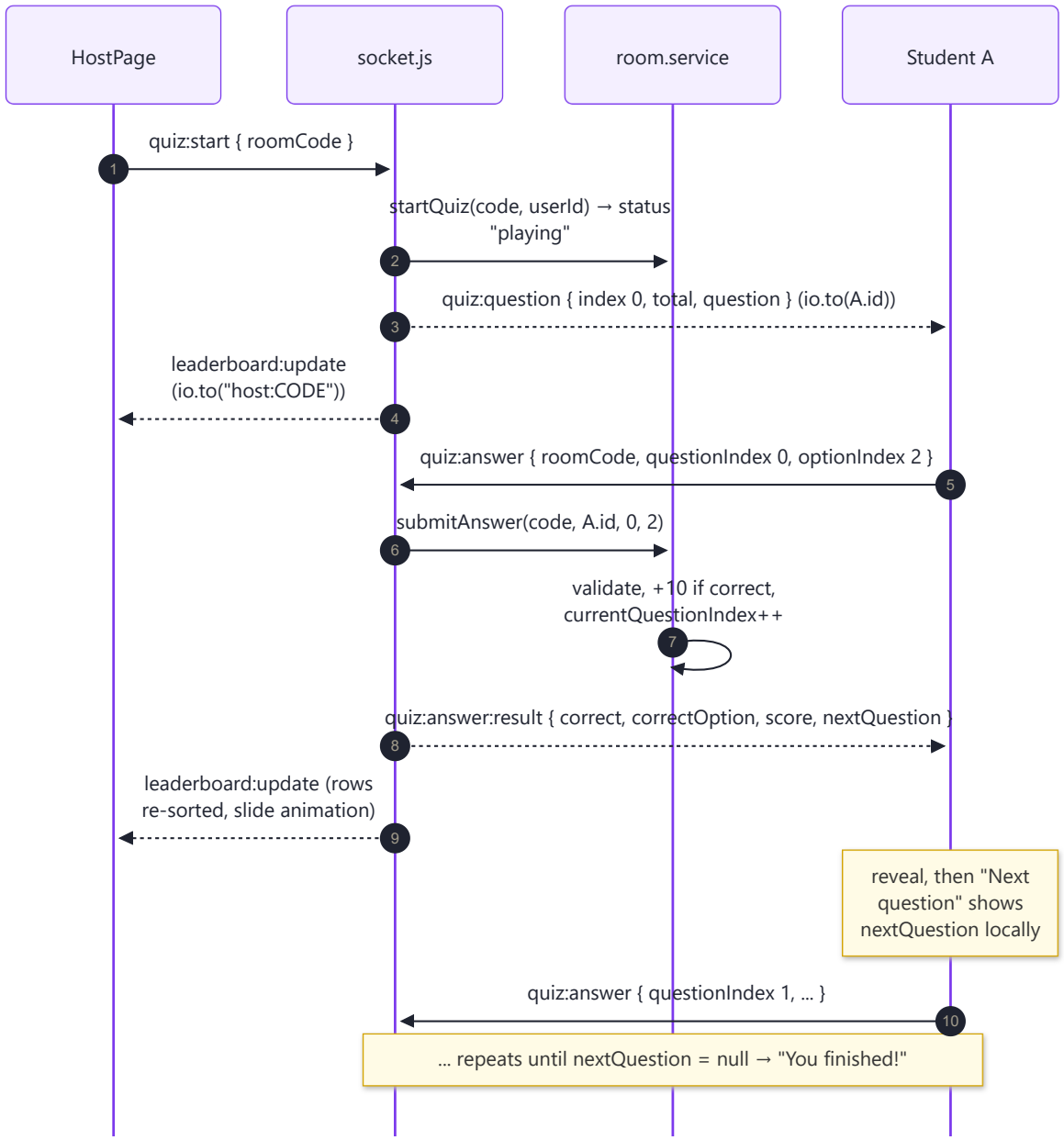


Figure 9.5: The answer loop. Each student runs this loop at their own speed.

1. The Start button is disabled until `players.length > 0`. `handleStartQuiz` emits `quiz:start { roomCode }`.
2. `room.service → startQuiz` checks the requester is the host (`room.hostId === socket.data.userId`), the room is `waiting`, and there is at least one player; then sets `status = "playing"` and `startedAt = Date.now()`. Joining is now closed (`addPlayer` rejects with "This quiz has already started").
3. For **each player**, the server emits `quiz:question` to that player's personal room (`io.to(player.id)`) with `question.service → getPublicQuestion`: `{ index, total, question: { text, options } }`, without the answer. `emitLeaderboard` sends the first board to `host:CODE`, and the host screen switches to the leaderboard.

4. Student taps a tile: `PlayerPage → handleAnswer` ignores the tap if already submitting or showing a result; otherwise it marks the tile, sets `submitting`, and emits `quiz:answer { roomCode, questionIndex: currentQuestion.index, optionIndex }`.
5. `room.service → submitAnswer` validates: room exists and is `playing`; the sender is a player; not finished; `questionIndex === player.currentQuestionIndex` (blocks double answers and skipping ahead); `optionIndex` is an integer within the options. Then it scores (`POINTS_PER_CORRECT = 10`), increments `currentQuestionIndex`, and sets `finished` after the last question.
6. The sender gets `quiz:answer:result`: `correct`, `correctOption` (now safe to reveal), the new `score`, and `nextQuestion` (the public version of the next question, or `null`). `QuestionView` shows the green ring and checkmark on the correct option and a red ring and cross on a wrong pick.
7. The host gets a new `leaderboard:update`. `room.service → getLeaderboard` sorts by score (ties by nickname) and gives equal scores the same rank (`findIndex` of the first player with that score, plus one); `answered` is the player's `currentQuestionIndex`. `Leaderboard.jsx`'s `motion.li layout` slides rows to their new places.
8. The student clicks **Next question**: `handleNext` simply displays `result.nextQuestion`; there is no extra round trip because the server already advanced the player. When `nextQuestion` is `null`, the button says "See my score" and leads to the "You finished!" screen.

> **SECURITY**
> The score is computed only on the server. The client sends *which option* was chosen, never "I was correct" or "my score is 100". Without this, a student could edit the JavaScript in their browser and send themselves any score.

9.6 Host ends the quiz; final ranks; results saved

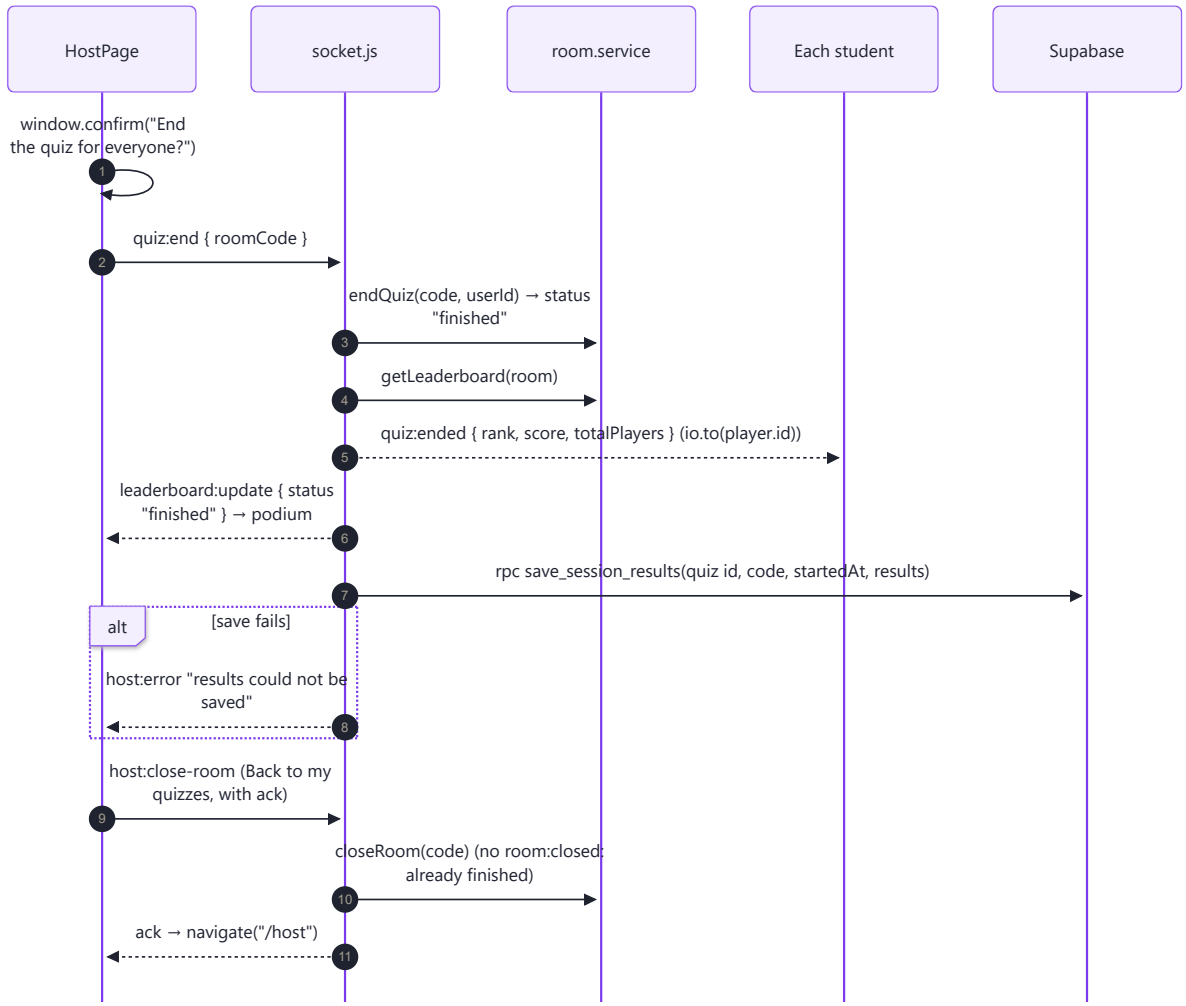


Figure 9.6: Ending a quiz and saving the results.

1. "End quiz" is only shown while `leaderboard.status === "playing"`. `handleEndQuiz` asks for confirmation, then emits `quiz:end { roomCode }`.
2. `room.service → endQuiz` checks host and `playing` status and sets `finished`. Students who had not finished simply stop where they are.
3. The final leaderboard is computed once. Each player gets their own `quiz:ended { rank, score, totalPlayers }`; `PlayerPage → handleQuizEnded` shows the big rank, with a medal for ranks 1–3.
4. The host gets `leaderboard:update` with `status: "finished"`: `Leaderboard.jsx` shows "Final results", a podium for the top 3, then the rest.
5. `session.service → saveSessionResults` calls the `save_session_results` Postgres function. On failure, the error is logged and the host is told via `host:error`; the game result on screen is unaffected.
6. "Back to my quizzes" → `handleCloseRoom` → `socket.timeout(3000).emit("host:close-room", ack)`. The server deletes the room (no `room:closed`, because students already have their final screen) and calls the ack; the client navigates to `/host` (also after 3 s if the server never answers).

9.7 Host refresh, student disconnect, closing the room

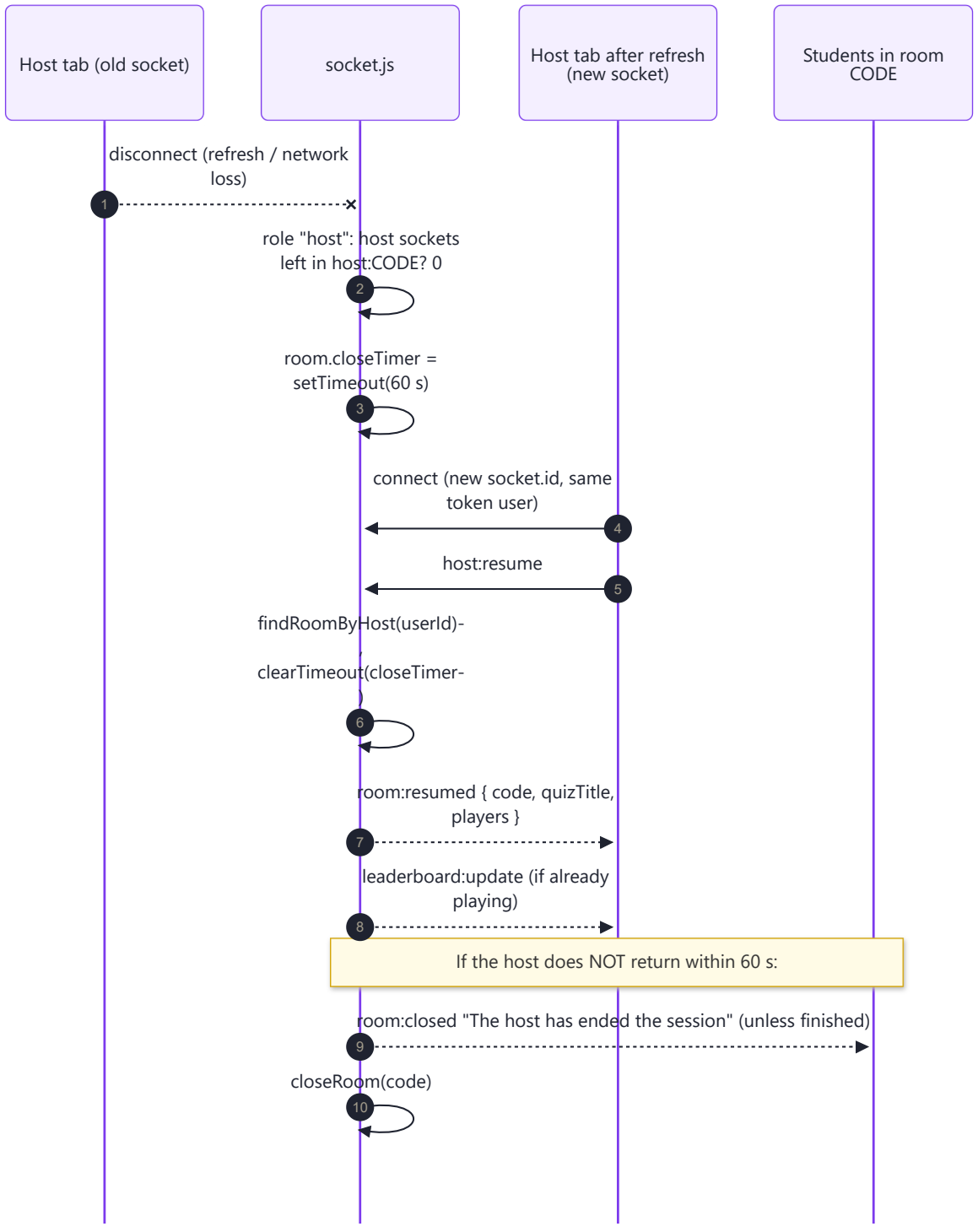


Figure 9.7: The 60-second grace period (`HOST_GRACE_MS = 60 * 1000`).

Host refreshes mid-quiz

1. The old page unmounts or the network drops; the server's `disconnect` handler sees `socket.data.role === "host"`.
2. It counts how many sockets are still in `host:CODE` (`io.sockets.adapter.rooms.get(...)?.size`). If another host tab is open, nothing happens. Otherwise it starts a **60-second timer** stored in `room.closeTimer`. The game keeps running: students can keep answering.

3. The reloaded `HostPage` connects a new socket (same user, fresh token). On `connect` it emits `host:resume`; the server finds the room with `findRoomByHost(userId)`, cancels the timer, re-joins both socket rooms, and sends `room:resumed` plus, if the game has started, the current `leaderboard:update`. The screen returns to exactly where it was.
4. If the host never returns, the timer callback first checks the room is still the same object (`getRoom(room.code) !== room` guards against a deleted or replaced room), sends `room:closed` to all students (unless the quiz is already finished), and deletes the room.

A student disconnects

1. `disconnect` with role `"player"` calls `room.service → removePlayer(code, socket.id)`.
2. In the **lobby** (`waiting`), the player is removed from the list and `room:players` is re-sent, so their pill disappears and their nickname is free again.
3. **During or after the game**, the player is kept (their score still counts) but marked `connected: false`; the host's board fades the row and shows "(disconnected)". On the client, `Leaderboard.jsx`'s `allFinished` check ("Everyone has finished!") counts disconnected players as done.
4. There is no way back in: a reconnecting phone has a new `socket.id`, and `addPlayer` rejects joins once the quiz has started (section 12).

Host closes the room

1. From the lobby ("Close room", after a confirm) or the final results ("Back to my quizzes"), `handleCloseRoom` emits `host:close-room` with an acknowledgement and a 3-second timeout.
2. The server verifies `role === "host"` and `room.hostId === userId`, cancels any timer, sends `room:closed` if the quiz isn't finished (students see the calm "Session ended" screen), deletes the room, leaves both socket rooms and clears `socket.data`.
3. The ack fires and the host goes back to `/host`. Now `findRoomByHost` finds nothing, so the host can create a new room.

> **REAL-LIFE EXAMPLE**
>
> The grace period is like a teacher stepping out of the exam hall: the exam keeps going, and if they come back within a minute nothing changes. If they don't come back, the invigilator sends everyone home.

> **INTERVIEW TIP**
>
> "How do you handle a host refreshing the page?" is a great question to be asked, because QUIZZY has a real answer: identity by user ID instead of socket ID, `host:resume` on every `connect`, and a 60-second grace timer with a stale-room guard. Mention the trade-off: students don't have this yet, because they have no stable identity.

10. Deployment

In production the client and server are deployed separately, both automatically from the same GitHub repository. Dashboard settings are not stored in the repository, so the values below marked **Assumed** describe the standard setup for this project layout. Check them against your Vercel and Render dashboards.

10.1 Vercel (client)

Setting	Value
Root directory	`client` **Assumed**
Framework preset	Vite **Assumed**
Build command	`npm run build` (runs `vite build`, from `client/package.json`)
Output directory	`dist` (Vite's default)
Environment variables	`VITE_API_URL`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY`

`client/vercel.json`

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

This one rule makes the SPA work on Vercel: any path that is not a real file (`/host`, `/join/483920`) is answered with `index.html`, and React Router then picks the page. Real files such as `/assets/index-abc123.js` are still served normally.

10.2 Render (server)

Setting	Value
Service type	Web Service, Node runtime **Assumed**
Root directory	`server` **Assumed**
Build command	`npm install` **Assumed**
Start command	`npm start` → `node server.js` (from `server/package.json`)
Instance type	Free **Assumed**
Port	Render sets `PORT`; `server.js` listens on `process.env.PORT || 3000`
Environment variables	`CLIENT_URL`, `SUPABASE_URL`, `SUPABASE_SECRET_KEY`, `GEMINI_API_KEY`, `GEMINI_MODEL`

Note that `npm run dev` (`npx nodemon server.js`) is only for local development: nodemon restarts the server on every file change, which would wipe all rooms.

10.3 Every environment variable

Name	Set in	Read by	Purpose / notes
`VITE_API_URL`	Vercel; local `client/.env`	`client/src/config/config.js`	The Render server's `https://` address. Falls back to `http://localhost:3000` if missing. Baked in at build time
`VITE_SUPABASE_URL`	Vercel; local `client/.env`	`client/src/supabase/supabaseClient.js`	Supabase project URL. Public
`VITE_SUPABASE_PUBLISHABLE_KEY`	Vercel; local `client/.env`	`supabaseClient.js`	Public key for Auth in the browser. App throws on startup if missing
`CLIENT_URL`	Render (not in the local `server/.env`; locally the default is used)	`server/server.js`, `server/src/app.js`	The Vercel site's exact origin, for CORS. Falls back to `http://localhost:5173`
`PORT`	Set automatically by Render	`server/server.js`	Port to listen on; default 3000 locally
`SUPABASE_URL`	Render; local `server/.env`	`server/src/db/supaBase.js`	Same project URL as the client's
`SUPABASE_SECRET_KEY`	Render; local `server/.env`	`supaBase.js`	**Secret.** Full database access. Server throws on startup if missing
`GEMINI_API_KEY`	Render; local `server/.env`	`server/src/services/ai.service.js`	**Secret.** Billed per use. Server throws on startup if missing
`GEMINI_MODEL`	Render; local `server/.env`	`ai.service.js`	Which Gemini model to call; change models without code changes

Both `.env` files are listed in `.gitignore` (root and `client/.gitignore`), and the Git history confirms no `.env` file has ever been committed.

COMMON MISTAKE

- **Wrong variable names.** The client only sees variables that start with `VITE_`; naming it `API_URL` on Vercel gives `undefined` and the localhost fallback. On the server, a typo such as `GEMINI_KEY` stops the server at startup with "Missing GEMINI_API_KEY or GEMINI_MODEL in .env". The message mentions `.env` even on Render, where the fix is in the dashboard.
- **Localhost values in production.** Copying `VITE_API_URL=http://localhost:3000` into Vercel means the live site calls the visitor's own laptop.
- `??` **versus** `||`**.** `config.js` uses `??`, which only replaces `null` / `undefined`. If the variable exists but is an *empty string*, `API_URL` becomes `""` and requests go to the Vercel site itself.
- **Forgetting to redeploy** the client after changing a `VITE_` variable (see Vite in section 3.10).

10.4 Why the server can't run on Vercel

Key terms here (defined in section 3): Serverless function · Long-running process

QUIZZY's server breaks all three serverless assumptions:

1. **Long-lived connections.** A Socket.IO connection stays open for the whole game (minutes). Serverless functions end shortly after responding and cannot host a WebSocket server.
2. **State in memory.** The `rooms` Map must survive between events. In serverless, the next event could reach a different, fresh copy with an empty Map.
3. **Timers.** The 60-second host grace timer (`setTimeout`) needs a process that is still running a minute later.

Render runs `node server.js` as one continuously running process, which is exactly what this design needs.

10.5 Continuous deployment from GitHub

Key terms here (defined in section 3): Continuous deployment · Production vs preview · Deployment

Both platforms are connected to the GitHub repository. A push to `main` makes Vercel rebuild `client/` and Render rebuild and restart `server/`. Two consequences:

- **A server deploy restarts the process**, which deletes every live room. Avoid pushing during a class.
- Commits like `deployement chnages` and `chore: trigger Vercel deploy` in the history show the usual early-deployment loop: fix a setting, push, check.

10.6 HTTPS and wss://

Vercel and Render both provide HTTPS certificates. Render's proxy handles the encryption (TLS) and forwards plain HTTP to the Node process on `PORT`, so the code never touches certificates. When `VITE_API_URL` starts with `https://`, socket.io-client automatically uses `wss://` for the WebSocket upgrade.

COMMON MISTAKE

Using `http://` for `VITE_API_URL` in production. A page loaded over HTTPS may not open `http://` or `ws://` connections (**mixed content**), so the browser silently blocks every API call and socket.

10.7 Free-tier sleep behaviour

Render's free web services **spin down after about 15 minutes without incoming traffic**. The next request wakes the service, which can take up to around a minute (a **cold start**). Because rooms live in memory, a spin-down or restart also **deletes every room**.

- What users see: the host's quiz list spinner hangs for a while; a host socket may take a long time to connect ("Checking for a running session...").
- Workaround: open the host dashboard a few minutes before class; the `GET /api/quizzes` call wakes the server. The health route `GET /` can be used to wake it too.
- Supabase's free plan can also pause projects after a long period of inactivity, and Gemini's free tier has request quotas, which is one reason `callWithRetry` retries on HTTP 429.

INTERVIEW TIP

Mention the cold start yourself, before you are asked: "The free tier sleeps; in production I'd use a paid always-on instance, because a sleep or redeploy also wipes the in-memory rooms." It shows you understand the link between hosting and your state design.

11. Security measures

Each protection below names where it lives, what attack it prevents, and what would happen without it.

Protection	Where	Prevents	Without it
Answer hiding	`question.service → getPublicQuestion`	Reading correct answers from WebSocket traffic	Any student could open DevTools and see every `correctIndex`
Server-side scoring	`room.service → submitAnswer`	Clients claiming points	A modified client could send "score: 1000"
Question-order check (`questionIndex === currentQuestionIndex`)	`submitAnswer`	Answering the same question twice, replaying messages, skipping ahead	Resend the correct answer 50 times for 500 points
Option bounds check	`submitAnswer` (`Number.isInteger` , range)	Malformed indexes (`"2"` , `-1` , `99`)	Wrong-type comparisons; nonsense data in results
Join rules: only while waiting; nickname 1–20 chars, must be a string, unique (case-insensitive)	`room.service → addPlayer`	Late joiners, impersonation ("Ana" vs "ana"), huge or non-string nicknames	Two "Ana"s on the board; a 10,000-character name breaking the projector layout
Escaped output	React (no `dangerouslySetInnerHTML` anywhere)	**XSS**: a nickname like `<img src=x onerror=…>`	Script running on the host's (logged-in) machine via the projector page
Input validation (Zod)	`quiz.schema.js` , `ai.schema.js`	Malformed, oversized or inconsistent quizzes and requests	A quiz with `correctIndex: 7` of 4 options would break the game later
Field allowlists	`z.object` strips unknown keys; explicit `select(...)` columns; `toQuiz` ; `getLeaderboard` ; `saveSessionResults` mapping	**Mass assignment** (extra fields like `creatorId`) and leaking internal fields	Clients could set fields they shouldn't; socket IDs saved to the DB
ID validation	`quiz.service → isValidId`	Malformed IDs reaching the database	Database errors turned into 500s; noisy probing
Authentication	`requireAuth` (REST), `socketAuth` + per-event checks (sockets)	Anonymous use of host features	Anyone could list, create, or host quizzes

Protection	Where	Prevents	Without it
Ownership checks + 404	`getQuiz`, `host:create-room`, `startQuiz`, `endQuiz`, `host:close-room`	Using or controlling other hosts' quizzes and rooms; ID enumeration	A student who learns the code could start or end the game
Identity from the token	`createQuiz` (`creatorId: req.user.id`), `roomService(userId, …)`	Creating quizzes as someone else	Spoofed ownership
Race-condition guards	`host:create-room`: `creatingRoom` flag, re-check after `await`, `socket.connected`	Double rooms from double clicks or two tabs	Two rooms for one host; orphan rooms
AI input limits	`generateRequestSchema` (topic ≤ 100 chars, difficulty enum, count ≤ 15)	Cost abuse and very long prompts	Asking for 10,000 questions per call
AI output validation	`ai.service`: JSON schema + `generatedSchema` + retry + human review	Malformed or nonsensical AI output reaching the database	Questions with no correct answer saved and played
Socket payload validation (missing)	Should be in every `socket.on` handler	Malformed messages from any client	This is the current state: one payload-less `quiz:start` crashes the server (section 14, issue 1)
Rate limiting (10/hour/user)	`aiRateLimit.js`	Draining the Gemini quota / bill	This is the current state: the limiter is never reached (section 14)
CORS allowlist	`app.js`, `server.js` (`CLIENT_URL`)	Other websites using visitors' browsers to call the API	Any site could script requests from a logged-in host's browser
Secret key on the server only; `.env` git-ignored	`db/supaBase.js`, `.gitignore`	Full database takeover	Anyone reading the bundle or repo controls the database
Parameterized queries	supabase-js / RPC	SQL injection	Crafted titles could run SQL
Generic error messages	All `catch` blocks reply with fixed text; details only in `console.error`	Leaking stack traces, table names, keys	Attackers learn the internals from error responses
HTTPS / WSS	Vercel, Render	Eavesdropping on tokens and answers	Tokens readable on school Wi-Fi

Protection	Where	Prevents	Without it
Body size limit	`express.json()` default 100 kB, plus Zod's array limits	Memory exhaustion with giant bodies	One request could slow the server for everyone
Row Level Security	Supabase dashboard; likely enabled with no policies (section 7.6)	Direct table access with the public key	Quizzes and answers downloadable without QUIZZY's server

SECURITY

The guiding rule is **never trust the client**. Everything the browser sends (nicknames, answers, quiz data, room codes) is checked on the server, and everything the server sends is the minimum the browser needs. The AI is treated the same way: its output is untrusted input.

COMMON MISTAKE

Thinking the front-end checks are security. The Start button being disabled with 0 players, `RequireAuth`, and the 6-digit check on `HomePage` are all *convenience*; anyone can bypass them. Each one has a matching server check (`startQuiz`, `requireAuth`, `addPlayer`).

INTERVIEW TIP

Pick three and explain them deeply: answer hiding (what's in the payload and why), server-side scoring with the question-index check (replay attacks), and the 404-instead-of-403 ownership check. Then volunteer one weakness (the rate limiter bug) and how you would fix it. Honesty about a flaw scores better than claiming perfection.

12. Known limitations and future improvements

Limitation	Why it exists	Improvement
Single server instance	Rooms live in one process's memory; socket rooms exist only in that process	Move room state to Redis and use the Socket.IO Redis adapter so several instances share rooms and broadcasts
Rooms lost on restart, deploy or sleep	In-memory design (decision 2 in `DECISIONS.md`)	Persist room snapshots (Redis or Postgres); paid always-on instance; deploy outside class times
Students can't reconnect	Players are identified by `socket.id`; joins are closed once playing	Give each player a random `playerId` stored in `sessionStorage`, send it on join, and add a `player:rejoin` event that re-attaches the new socket
No feedback when the server is unreachable	`connect_error` only logs; the host spinner never ends	Show a "Can't reach the server, retrying…" message; a timeout on the "Checking" screen
Simple scoring	Fixed 10 points per correct answer, no time element	Speed bonus, streaks, per-question time limits (server-timed)
No editing or deleting quizzes	Only list, create and (unused) get endpoints exist	`PUT` / `DELETE /api/quizzes/:id` with the same ownership check; an edit page using `GET /:id`
Saved results are never shown	`save_session_results` fills `sessions` and `results`, but no endpoint or page reads them	A "Past sessions" page with per-question statistics for teachers
One room per host	`findRoomByHost` returns one room; resume ignores which quiz you opened	Key rooms by host + quiz, or show "You have a running room for X: resume or close?"
Rate limiter in memory (and currently bypassed)	A Map in one process; duplicate route	Fix the route; move counts to Redis or a database table so limits survive restarts and multiple instances
Token checked only at socket handshake	`socketAuth` runs once per connection	Re-verify on sensitive events or disconnect sockets when the token expires
A network call per authenticated request	`auth.getUser(token)` asks Supabase every time	Verify the JWT locally with the project's signing keys (JWKS) and cache results
Errors shown with `alert()` on the host page	Quick implementation	Inline error banners, like the student side

Limitation	Why it exists	Improvement
No automated tests or CI	`"test": "echo \"Error: no test specified\""`	Unit tests for `room.service` (pure functions, easy to test), Socket.IO integration tests, GitHub Actions running lint and tests before deploy
Database schema not in the repo	Created in the Supabase dashboard	Supabase CLI migrations committed to Git, so the database can be recreated and reviewed
Fixed four options in the editor	`emptyQuestion()` always creates 4; the server allows 2–6	Add/remove option buttons; the student grid already handles any count

KEY IDEA

Almost every limitation traces back to one deliberate trade-off: **in-memory, single-process game state**. It made the first version simple and fast; the "next step" for most improvements is a shared store such as Redis.

INTERVIEW TIP

When asked "how would you scale this to 10,000 concurrent games?", answer in steps: (1) move room state to Redis, (2) add the Socket.IO Redis adapter, (3) run several Render instances behind a load balancer with sticky sessions, (4) give players stable IDs so reconnects can land on any instance.

13. Interview questions with answers

58 questions in four groups. The answers are written the way you would say them about *your* project: specific, with the reason and the trade-off. Don't memorize them word for word; make sure you can explain each idea and point to the file.

> **INTERVIEW TIP**
>
> A strong answer has three parts: **what** you did (with a concrete name from the code), **why** (the problem it solves), and **the trade-off** (what it costs or what you'd do at larger scale).

13.1 Basic

Q1. What is QUIZZY, in two sentences?

> A student-paced live quiz app: teachers create multiple-choice quizzes, by hand or with Gemini, open a room with a 6-digit code and QR code, and students join from their phones and answer at their own speed. The teacher sees a live leaderboard with scores and progress, and the final results are saved to Postgres.

Q2. What is the tech stack and where does each part run?

> React 19 with Vite, React Router and Tailwind v4 on the client, deployed as static files on Vercel. Node.js with Express 5 and Socket.IO on the server, running as one long-lived process on Render. Supabase provides Postgres and authentication, and Gemini generates questions. The browser only talks to Supabase for login; all data goes through my server.

Q3. Why do you use both REST and WebSockets?

> REST is for actions the user starts and that have one answer: list quizzes, save a quiz, generate with AI (`/api/quizzes`). The live game needs the server to push updates the moment *someone else* acts, like a student joining or answering, so it uses a persistent Socket.IO connection. Using REST for the game would mean polling, which adds delay and wasted requests.

Q4. Why Socket.IO rather than plain WebSockets?

> Socket.IO gives me named events (`quiz:answer`), rooms for broadcasting (`io.to(code)`), automatic reconnection (which my host resume relies on), acknowledgements (used by `host:close-room`), a handshake `auth` payload for the token, and a fallback to HTTP long-polling. With raw WebSockets I'd have to build all of that myself.

Q5. How does a student join a game?

> They scan the QR code, which is just the URL `/join/<code>`, or type the code on the home page. `PlayerPage` connects an anonymous socket and emits `player:join` with the code and nickname. `room.service → addPlayer` checks the room exists and is still waiting, and that the nickname is 1–20 characters and not taken; then the server broadcasts `room:players` to the room, and receiving that list is how the student's page knows it has joined.

Q6. How are room codes generated? What about collisions?

> `generateRoomCode` picks a random number from 100000 to 999999 and loops until it finds one that isn't already a key in the `rooms` Map, so live codes never collide. Six digits are easy to read on a projector and type on a phone. They are not secret (anyone in the class sees them), which is fine because joining only gives you a player seat; host actions need the host's token.

Q7. What does the useEffect in PlayerPage do, and why is there a cleanup function?

> It registers all socket listeners (`room:players`, `quiz:question`, ...) and calls `socket.connect()` once when the page mounts. The cleanup removes every listener with `socket.off` and disconnects. Without the cleanup, navigating away and back would add duplicate listeners, so each event would be handled twice, and the socket would stay connected as a ghost player.

Q8. Which environment variables does the app need?

> On Vercel: `VITE_API_URL`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY`. They need the `VITE_` prefix to reach browser code and are baked in at build time. On Render: `CLIENT_URL` for CORS, `SUPABASE_URL`, `SUPABASE_SECRET_KEY`, `GEMINI_API_KEY`, `GEMINI_MODEL`; `PORT` is set by Render. The server throws at startup if a required one is missing, so misconfiguration fails loudly.

Q9. What is CORS and why does QUIZZY need it?

> Browsers block a page from reading responses from a different origin unless that server allows it. My page is on Vercel and my API on Render, which are different origins, so the server sends `Access-Control-Allow-Origin` for exactly `CLIENT_URL`, in Express (`app.js`) and in the Socket.IO server options (`server.js`). It protects users' browsers from other sites; it's not authentication, which is why the endpoints also require a token.

Q10. What is a JWT and how does QUIZZY use it?

> A signed token with a payload, including the user ID and an expiry. Supabase issues it at login; supabase-js stores it and refreshes it. The client sends it as `Authorization: Bearer` on REST calls (`apiFetch`) and in the socket handshake (`hostSocket`). The server verifies it with `supabase.auth.getUser` and then trusts the user ID for authorization.

Q11. How do you protect pages in React?

> `RequireAuth` wraps the host routes: it shows a spinner while the session loads, redirects to `/login` with a `from` path if there is no user, and otherwise renders the page. But that's only user experience: the real protection is server-side in `requireAuth` and the socket checks, because client code can be modified.

Q12. What does Zod do in your project?

> It validates data at the server's edges: `quizSchema` for saved quizzes, `generateRequestSchema` for AI requests, and `generatedSchema` for the AI's output. `safeParse` gives me cleaned data or the first readable error message, which I return as a 400. It also strips unknown fields, which blocks mass assignment.

Q13. What is vercel.json for?

> It rewrites every path to `index.html`. QUIZZY is a single-page app, so `/join/483920` isn't a real file; without the rewrite, scanning the QR code would give a Vercel 404. With it, React Router reads the path and shows `PlayerPage`.

Q14. Why are quizzes in the database but games in memory?

> Quizzes must survive forever and are read rarely; games are short-lived, change on every answer, and need instant updates. So I load the quiz once when the room is created, run the whole game in the `rooms` Map, and write the results once at the end. That's two database operations per game instead of one per answer. The trade-off is that a restart loses running games.

Q15. How is the score calculated?

> On the server, in `room.service → submitAnswer`: 10 points (`POINTS_PER_CORRECT`) for a correct answer, nothing for a wrong one, no time bonus. The client only sends which option it chose; it never sends whether it was right or its score.

Q16. Why Tailwind CSS?

> Utility classes let me style quickly and consistently, and Tailwind v4's `@theme` turns my design tokens (answer colors, fonts, card shadow) into classes like `bg-answer-red`. Responsive prefixes made it easy to build phone-first student screens and projector-sized host screens, and `motion-reduce:` lets me respect reduced-motion settings.

Q17. What does `express.json()` do?

> It's middleware that parses a JSON request body into `req.body`. It also rejects invalid JSON with a 400 and bodies larger than 100 kB by default, which is a cheap first line of defence before Zod validation.

Q18. What is the difference between the host and the student in the system?

> Hosts have Supabase accounts and are identified by their verified user ID; they own quizzes (`creator_id`) and rooms (`hostId`). Students are anonymous, identified only by their `socket.id` for one connection. That asymmetry is deliberate (no sign-up friction in class), and it explains why hosts can resume after a refresh but students can't yet.

13.2 Intermediate

Q19. Walk me through what happens when a student taps an answer.

> `handleAnswer` ignores the tap if a result is already shown or a request is in flight, marks the tile and emits `quiz:answer` with the room code, question index and option index. `submitAnswer` checks the room is playing, the sender is a player, the index matches their current question and the option is a valid integer; then it scores and advances `currentQuestionIndex`. The student gets `quiz:answer:result` with `correct`, `correctOption`, `score` and the next public question; the host gets `leaderboard:update`. "Next" just displays `nextQuestion` locally.

Q20. How do you stop students from cheating?

> Four layers. The answer key never leaves the server before a student answers (`getPublicQuestion` only sends text and options). Scoring happens only on the server. `questionIndex` must equal the player's current index, so you can't answer twice or replay a correct answer. And options are range-checked. What I can't stop is two students sharing a phone screen; that is outside software.

Q21. Explain `socket.emit`, `io.to(id)` and `io.to(room)` with QUIZZY examples.

> `socket.emit` replies only to the sender, e.g. `quiz:answer:result` or `player:join:error`. `io.to(player.id)` targets one other socket through its automatic personal room, e.g. sending each player their first `quiz:question` when the host starts. `io.to(code)` reaches everyone in the game, e.g. `room:players` and `room:closed`. `io.to("host:"+code)` reaches only host screens, for `leaderboard:update`.

Q22. Why a separate `host:CODE` room?

> The leaderboard contains everyone's scores and is sent on every answer. Sending it to `CODE` would push the whole class's data to every phone on every answer: wasted bandwidth and information students don't need. A host-only room also lets several host tabs stay in sync, and the disconnect handler uses its size to decide whether the host is truly gone.

Q23. How are sockets authenticated?

> The host socket's `auth` option is an async function that reads the current session and returns `{ token }`; Socket.IO calls it on every connection and reconnection, so it's always fresh. On the server, `io.use(socketAuth)` verifies it with `supabase.auth.getUser` and stores the user ID in `socket.data.userId`, which the client can't modify. Host-only handlers check that value.

Q24. Why doesn't `socketAuth` reject connections without a token?

> Because students are supposed to connect anonymously through the same Socket.IO server. The middleware only labels the connection; authorization happens per event: `host:create-room` requires a user ID, and `startQuiz` / `endQuiz` compare it with `room.hostId`. An invalid token simply becomes an anonymous socket, which can do nothing a student can't.

Q25. Why return 404 instead of 403 for someone else's quiz?

> A 403 confirms the quiz exists, which helps an attacker enumerate valid IDs. Returning the same 404 for "doesn't exist" and "not yours" reveals nothing. `getQuiz` does this, and the socket handler uses the same "Quiz not found" message. GitHub does the same for private repos.

Q26. Why did you use a Postgres function to save a quiz?

> Saving writes one row to `quizzes` and many rows to `questions`, and it must be all-or-nothing. supabase-js can't run a transaction across several calls, but a Postgres function call is a single statement and therefore atomic. So `create_quiz` does both inserts and returns the ID, in one round trip. The same pattern is used for `save_session_results`.

Q27. How do you make the AI output reliable?

> Four steps in `ai.service.js`. A strict prompt (exactly 4 options, one correct, vary the position). Structured output with a JSON schema, so Gemini must return `{ questions: [...] }`. Then `JSON.parse` plus Zod validation with the same `questionSchema` human quizzes use, with a second attempt if it fails. Finally, nothing is saved automatically: the host reviews and edits before saving.

Q28. What happens if Gemini is down or overloaded?

> `callWithRetry` retries transient errors (HTTP 429, 500, 503) twice with exponential backoff (1 s, then 2 s). Other errors fail immediately. If everything fails, the controller logs the details and returns a generic 502 "Could not generate questions. Please try again.", and the page shows it in the AI card. Worst case is 2 attempts × 3 calls = 6 Gemini calls for one click, which is why rate limiting matters.

Q29. How are leaderboard ranks computed, including ties?

> `getLeaderboard` sorts by score descending, with nickname as a tie-breaker for a stable order. Each player's rank is the index of the first player with the same score, plus one. So scores 30, 20, 20, 10 give ranks 1, 2, 2, 4: standard "competition ranking". The same ranks are sent to students in `quiz:ended` and saved to the database.

Q30. How does the host page survive a refresh?

> Hosts are identified by user ID, not socket ID. On every `connect`, `HostPage` emits `host:resume`; the server finds the host's room with `findRoomByHost`, cancels the pending close timer, re-joins the socket to `CODE` and `host:CODE`, and sends `room:resumed` plus the current leaderboard. The UI rebuilds itself from that, and the game never stopped for students.

Q31. Explain the 60-second grace period.

> When the last host socket of a room disconnects, the server doesn't close the room immediately: it starts a `setTimeout` stored in `room.closeTimer`. If the host resumes, the timer is cleared. If not, the callback first checks `getRoom(room.code) === room` (the room might have been closed and the code reused), then sends `room:closed` to students and deletes the room. Without the grace period, a teacher's accidental refresh would kick out the whole class.

Q32. What happens when a student's phone disconnects?

> `removePlayer` removes them entirely if the game is still in the lobby (and re-broadcasts the list); during the game it keeps their score but sets `connected: false`, and the host's board fades their row with " (disconnected)". They currently can't come back, because their new socket has a new ID and joins are closed once playing.

Q33. What is the difference between the Supabase publishable and secret keys?

> The publishable key is public (it's in the JavaScript bundle) and acts as an anonymous or logged-in user, restricted by Row Level Security; I only use it for Auth in the browser. The secret key bypasses RLS and gives full database access; it lives only in Render's environment and `server/.env`, used by `db/supaBase.js`. Putting the secret key in a `VITE_` variable would publish it.

Q34. Why are there two socket modules on the client?

> `socket/socket.js` is the anonymous student socket used by `PlayerPage`; `hostSocket.js` sends the JWT in its handshake and is used by `HostPage`. Separating them keeps student connections token-free, gives each page exactly one connection with its own listeners, and makes it possible to test host and student in two tabs of one browser.

Q35. Node is single-threaded. Can two events still conflict?

> Synchronous code can't interleave, so `submitAnswer` never runs twice at the same instant. But an `await` lets other events run in between. In `host:create-room` I await the database; meanwhile a double click could start a second creation. So I set `socket.data.creatingRoom` before awaiting, and after the await I re-check `socket.connected` and `findRoomByHost` before creating the room.

Q36. Why identify students by socket.id, and what does that cost?

> It needs zero setup: no account, no stored ID, and the server already has it. It also lets me use `io.to(player.id)` directly. The cost is that identity dies with the connection: a dropped phone becomes a stranger and its score is orphaned. The fix is a random player ID kept in `sessionStorage` and a rejoin event.

Q37. How do the leaderboard rows slide when ranks change?

> Each row is a `motion.li` with the `layout` prop and a stable `key={player.id}`. When the sorted list re-renders, motion measures each row's old and new position, applies a transform that puts it back where it was, then animates the transform to zero: the FLIP technique. `MotionConfig reducedMotion="user"` turns the movement off for users who prefer reduced motion.

Q38. How does the student's page know the join succeeded?

> There's no dedicated "joined" event: `handlePlayers` sets `joined = true` when a `room:players` list arrives, and the server only sends that to sockets in the room, which happens after a successful `addPlayer`. It works, but it's implicit; an explicit acknowledgement on `player:join` would be clearer and would let the student's own nickname come from the server.

13.3 Advanced

Q39. How would you scale QUIZZY to many server instances?

> Today all state is in one process: the `rooms` Map and Socket.IO's socket-room membership. To scale out I would move room state into Redis (atomic operations per answer), add the Socket.IO Redis adapter so `io.to(code)` reaches sockets on any instance, enable sticky sessions on the load balancer (needed for long-polling), and move the rate-limit counters into Redis. Timers like the grace period would become Redis keys with expiry.

Q40. Describe a race condition you handled.

> `host:create-room` awaits `getQuizById`. During that await, the same host could click again, or open a second tab, or disconnect. I guard with a per-socket `creatingRoom` flag (cleared in `finally`), and after the await I re-check that the socket is still connected and that `findRoomByHost` still finds nothing. Otherwise a host could end up with two rooms, or an orphan room nobody can control.

Q41. What's the weakness of checking the token only at the socket handshake?

> A host socket stays authorized for as long as it is connected, even after the JWT expires or the user signs out elsewhere. For a 30-minute class game that's acceptable. For stricter needs I'd re-verify on sensitive events (start, end, close), or schedule a disconnect at the token's `exp` so the client reconnects with a fresh token.

Q42. `getUser(token)` vs verifying the JWT locally: which and why?

> `getUser` asks Supabase on every request: simple, and it catches revoked sessions or deleted users, but it adds latency and depends on Supabase being reachable (hence the 500 "Could not verify session" path). Local verification with the project's public signing keys is fast and works offline, but a token stays valid until it expires. At higher traffic I'd verify locally and keep tokens short-lived.

Q43. There's a bug in the AI rate limiting. Explain it and the fix.

> `quiz.routes.js` registers `POST /generate` twice: first `router.post("/generate", quizController.generateQuiz)`, then the version with `aiRateLimit`. Express runs the first match; `generateQuiz` sends a response and never calls `next()`, so the second registration and the limiter never run. The fix is to delete the first line. I'd also add a test that the eleventh request in an hour gets a 429.

Q44. How would you let students reconnect?

> Generate a random `playerId` on the client (`crypto.randomUUID()`), store it in `sessionStorage`, and send it with `player:join`. The server keys players by `playerId` and stores the current `socketId` separately. On reconnect, a `player:rejoin { roomCode, playerId }` event updates the socket ID, joins the socket room, sets `connected: true` and re-sends the player's current question. Personal messages would then use `io.to(player.socketId)`.

Q45. Can a malicious client crash the server?

> Yes, and I verified it. The `quiz:start` handler destructures `({ roomCode })` without a default, so emitting `quiz:start` with no payload throws a `TypeError` that nothing catches, and the Node process exits, ending every game. Any anonymous socket can send it. `host:close-room` calls `ack()` without checking that it is a function. I'd validate every socket payload with Zod (like the REST side), give every handler a default parameter, and wrap handlers in a helper that catches and logs errors so one bad message can never affect other games.

Q46. What about prompt injection through the topic field?

> The topic is inserted into the prompt, so a host could write "ignore the rules and...". The impact is limited: only logged-in hosts can call it, the topic is capped at 100 characters, the response must match a JSON schema and pass Zod, and the host reviews everything before saving. The worst case is odd questions in the host's own quiz. I'd still keep the topic clearly delimited in the prompt and could add content moderation for a school product.

Q47. How would you make running games survive a server restart?

> Store room snapshots outside the process, ideally in Redis, updated on every change, with a TTL. On startup or on `host:resume`, load the room back. Socket connections would still drop during a deploy, but both hosts (by user ID) and students (with the stable `playerId` from Q44) could resume. Also, deploys should avoid class times, and a paid instance avoids free-tier sleep.

Q48. How would you add a time limit per question fairly?

> The server must own time. When it sends a question to a player, store `questionSentAt` on the player; in `submitAnswer`, reject or score zero if `Date.now() - questionSentAt` exceeds the limit (plus a small network allowance). The client countdown is only a display. Speed bonuses would be computed the same way, never from a client timestamp.

Q49. Are there memory leaks?

> Slow ones. A finished room stays in the Map until the host closes it or disconnects for 60 s; a host who leaves the results open for days keeps it alive. The rate-limit Map keeps an entry for every user who ever generated (old timestamps are filtered but the key stays). Fixes: a periodic sweep that deletes rooms finished more than N minutes ago or idle too long, and deleting empty rate-limit entries (or a TTL store).

Q50. Your server bypasses RLS with the secret key. Why bother with RLS?

> Defence in depth. The publishable key is public, so if RLS were off, anyone could query Supabase's REST API directly and download every quiz including answers, bypassing my server entirely. With RLS enabled and no policies, that key can't touch the tables, and my server, which does its own ownership checks, is the only door.

Q51. How would you test this?

> `room.service.js` and `question.service.js` are mostly pure functions over plain objects, so I'd unit-test them first: joining rules, duplicate nicknames, answer order, scoring, tie ranks. Then integration tests that start the HTTP + Socket.IO server on a random port and drive two `socket.io-client` instances through a full game. REST controllers can be tested with supertest and a mocked Supabase client. Finally, a GitHub Actions workflow running lint and tests before deploys.

Q52. What if saving the results fails at the end of a game?

> Today the error is logged and the host gets a `host:error`, but the results are lost once the room is closed. Because `save_session_results` is one transaction, there's never a half-saved session. To make it reliable I'd retry with backoff, and keep the payload until it succeeds (or write it to a local queue, the "outbox" pattern), and only allow closing the room after a successful save or explicit confirmation.

Q53. How does QUIZZY handle a host with two tabs open?

> Both tabs resume the same room via `host:resume` and join `host:CODE`, so both get leaderboard updates. On disconnect, the server counts the remaining sockets in `host:CODE` and only starts the close timer when the count is zero, so closing one tab doesn't threaten the game. A second tab's `host:create-room` is refused by `findRoomByHost`.

Q54. Why is `quiz:question` only sent once?

> Only the first question is pushed, at start. Every following question rides along in `quiz:answer:result.nextQuestion`, because the server advances the player at the moment of answering. That saves a round trip per question and keeps the server as the single source of truth for each player's position; the "Next" button is purely a UI pause to read the feedback.

13.4 Behavioral

These answers must be honest and in your own words. The notes below point to real material in this codebase you can build them from.

Q55. What was the hardest bug you faced?

> Pick a real one and use the STAR shape (Situation, Task, Action, Result). This codebase contains good candidates: the **production deployment** (commits "deployement chnages" and "chore: trigger Vercel deploy"), where things that work locally break because of `CLIENT_URL` /CORS, a missing `VITE_API_URL` falling back to localhost, or a `VITE_` change that needs a rebuild; or the **host refresh problem** that led to identifying hosts by user ID plus `host:resume` and the grace timer. Explain how you found the cause (browser console, Network tab, Render logs, the debug `console.log`s still in `socketAuth.js` and `quiz.routes.js`) and what you changed so it can't recur.

Q56. Which decision would you change?

> A strong, honest answer: "Identifying students by `socket.id`. It was the quickest path, but it means a phone that drops Wi-Fi loses the game. I'd give each player a stable random ID from the start." Alternatives: creating the schema in the dashboard instead of committed migrations, or not validating socket payloads with Zod the way REST bodies are. Always add what you'd do instead and why you didn't at first (speed of the first version).

Q57. What did you learn from this project?

> Good themes with evidence: the difference between authentication and authorization (Supabase vs your ownership checks); never trusting the client, or the AI; how deployment shapes architecture (why the server is on Render); and the cost of in-memory state. Close with one concrete next step, like adding tests for `room.service`.

Q58. If you had two more weeks, what would you build?

> Prioritize and justify: (1) fix the rate-limit route and validate socket payloads, because they're security issues; (2) student reconnection, because it's the biggest real-classroom problem; (3) a "past sessions" page, since results are already saved but never shown; (4) tests and CI. Prioritizing by risk and user impact is itself the signal interviewers look for.

14. Issues found while reading the code

These were found while writing this guide. **None of them have been fixed**; the code was not modified. Severity: High security or availability risk, Medium a real bug users can hit, Low clean-up, inconsistency or dead code.

14.1 Bugs and risks

1. One malformed socket message crashes the whole server High

`server/src/sockets/socket.js`, handler `quiz:start`: `socket.on("quiz:start", ({ roomCode }) => …)` destructures its argument with no default. Emitting `quiz:start` with no payload throws a `TypeError` inside the handler. **Verified** with a minimal reproduction using the project's own `socket.io` version: the error is not caught, and the Node process exits. Any anonymous student socket can do this, and it ends every running game on the server. The other handlers use `(data = {})`, which only helps for `undefined`; a `null` payload would throw the same way. `host:close-room` calls `ack()` without checking it is a function.

Suggested fix: validate every socket payload with Zod (as REST bodies are), and wrap handlers in a helper that catches and logs errors.

2. The AI rate limiter never runs High

`server/src/routes/quiz.routes.js` lines 13–14 register `POST /generate` twice; the first registration has no `aiRateLimit`. Express stops at the first matching handler, which sends the response, so the limiter is unreachable and AI generation is unlimited (up to 6 Gemini calls per request with retries).

Suggested fix: delete line 13.

3. Quiz ownership depends on Zod stripping fields Medium

`quiz.controller.js → createQuiz` calls `createQuiz({ creatorId: req.user.id, ...result.data })`. Because the spread comes last, a `creatorId` inside `result.data` would override the real one. It is safe today only because `quizSchema` (a plain `z.object`) drops unknown keys; a future `.passthrough()` or schema change would silently allow creating quizzes as another user.

Suggested fix: put `creatorId: req.user.id` after the spread.

4. `room:players` sends too much to every student Medium

`socket.js` emits the raw `room.players` array (socket IDs, scores, progress, connection state) to the whole room on every join and lobby disconnect. Students only need nicknames. It is the one place that does not use a field allowlist like `getLeaderboard`.

5. Resume ignores which quiz was opened Medium

`host:resume` uses `findRoomByHost(userId)` and ignores the `:quizId` in the URL. If quiz A has a room and the host opens `/host/<quiz B>`, the page resumes quiz A's room. Creating a room for B fails with "You already have a running room" until A's room is closed.

6. Finished rooms and rate-limit entries are never cleaned up Medium

A `finished` room stays in the `rooms` Map until the host clicks "Back to my quizzes" or disconnects for 60 s; a host who leaves the results tab open keeps it in memory indefinitely. `aiRateLimit.js` filters old timestamps but never deletes a user's key. Both grow slowly over a long-running process.

7. Students can't recover from a dropped connection Medium

Players are identified by `socket.id`, and `addPlayer` rejects joins after the start. After a Wi-Fi drop, Socket.IO reconnects with a new ID, but `PlayerPage` never re-joins: the student stays on a question whose answers now fail with "Only players can answer".

8. No feedback when the server is unreachable Medium

`PlayerPage`'s `connect_error` handler only calls `console.error`, so Join appears to do nothing. On `HostPage`, `checking` only becomes `false` after a server reply, so the "Checking for a running session..." spinner never ends if the server is asleep or down.

9. Sign-up with email confirmation silently bounces Medium

`LoginPage.jsx → handleSubmit` navigates after `signUp` without checking `data.session` (the `data` variable is unused). If "Confirm email" is enabled in Supabase, there is no session yet, `RequireAuth` redirects back to `/login`, and the user is never told to check their email.

10. Results are lost if saving fails Medium

`quiz:end` calls `saveSessionResults` once. On failure the host gets `host:error`, but there is no retry, and closing the room discards the data.

11. Smaller client-side bugs Low

- `LoginPage` ignores `loading` from `useAuth()`, so a logged-in host briefly sees the login form before the redirect.
- `Leaderboard.jsx → allFinished` counts disconnected players as finished, so "Everyone has finished!" appears if everyone disconnects.
- `CreateQuizPage`: an empty "Questions" box sends `count: Number("") = 0` (the server rejects it with "Ask for at least 1 question"); Save is not disabled while AI generation runs, so the old questions can be saved just before they are replaced.
- `CreateQuizPage → handleSubmit`: if the server answers with non-JSON (e.g. an HTML 413 or 500 page), `res.json()` throws and the user sees the misleading "Could not reach the server".
- `crypto.randomUUID()` (in `emptyQuestion`) only exists in secure contexts (HTTPS or `localhost`); opening the create page over plain `http://` on a LAN IP during testing crashes it.
- `HostPage` reports errors with `alert()`.
- `HomePage`'s room-code input has a placeholder but no label (accessibility), and doesn't use the shared `Input` / `Card` components.

12. Socket token checked only at connection Low

`socketAuth` runs once per connection; a host socket stays authorized after its JWT expires or the user signs out elsewhere, until it reconnects.

14.2 Inconsistencies

13. DECISIONS.md is out of date Low

- It describes storing questions in `questions.js` as hardcoded test data; that file is now empty and quizzes come from Supabase.
- Decision 4 says HTTP is used "for operations such as creating a room"; rooms are created with the `host:create-room` socket event, and the HTTP room route is commented out.
- It stops at decision 5 and doesn't record Supabase, authentication, the Postgres functions, Gemini, the grace period, or hosting.

14. Misleading messages and configuration details Low

- Startup errors say "Missing … in .env" even on Render, where there is no `.env` and the fix is in the dashboard.
- `client/src/config/config.js` uses `??`, so an empty `VITE_API_URL` becomes `""` (requests go to the Vercel site) instead of failing.
- `server/package.json` has `"main": "index.js"` (no such file) and `"dev": "npx nodemon …"` without nodemon as a dependency.
- The function that creates a room is named `roomService`, the same as the module, giving calls like `roomService.roomService(...)`; `createRoom` would be clearer.
- `api.js` and `hostSocket.js` import with `"../src/…"` paths that leave and re-enter `src`; `"./…"` is the normal form.
- There is no root `README.md`; `client/README.md` is the unmodified Vite template. The database schema (four tables) and the two Postgres functions exist only in the Supabase dashboard, not as SQL in the repository.

14.3 Dead code and leftovers

15. Unused files, functions and imports Low

- `server/src/routes/room.routes.js` and `server/src/controllers/room.controller.js`: entirely commented out; also the commented `app.use("/api/rooms", …)` and `// app.use(cors());` in `app.js`.
- `server/src/data/questions.js`: empty file.
- `quiz.service.js → toPublicQuiz`: exported, never used.
- `GET /api/quizzes/:id` (`getQuiz`): never called by the client.
- `client/src/App.css` (empty), `client/src/assets/hero.png`, `react.svg`, `vite.svg` (unused), and `// window.io = io;` in `App.jsx`.
- Unused imports reported by ESLint: `API_URL` in `QuizListPage.jsx` and `CreateQuizPage.jsx`; `StrictMode` in `main.jsx` (the app is not wrapped in it); `data` in `LoginPage.jsx`. ESLint also flags `AuthContext.jsx` for exporting a hook next to a component (breaks fast refresh for that file).

16. Debug logging left in production Low

- `quiz.routes.js`: `console.log("controller exports:", …)` at startup.
- `socketAuth.js`: logs every socket connection with its user ID.
- `socket.js` `disconnect`: `console.log("DISCONNECT:", socket.id, socket.data)` logs user IDs and room codes on every disconnect.

Appendix: quick reference

A. The 20 custom socket events at a glance

Client → server	Server → client
`host:resume`, `host:create-room`, `host:close-room`	`room:created`, `room:resumed`, `host:no-room`, `host:error`
`quiz:start`, `quiz:end`	`quiz:error`, `leaderboard:update`, `quiz:ended`
`player:join`	`room:players`, `player:join:error`, `room:closed`
`quiz:answer`	`quiz:question`, `quiz:answer:result`, `quiz:answer:error`

B. Five files to know by heart

File	Why
`server/src/sockets/socket.js`	Every live event, the grace timer, who receives what
`server/src/services/room.service.js`	All game rules and the in-memory state
`server/src/services/ai.service.js`	Prompt, structured output, retry, validation
`client/src/pages/PlayerPage.jsx`	The student state machine and answer flow
`client/src/pages/HostPage.jsx`	Resume, lobby, QR code, leaderboard, end and close

C. Numbers worth remembering

Room code	6 digits, 100000–999999, unique among live rooms
Points per correct answer	10
Host grace period	60 seconds
Nickname length	1–20 characters, unique (case-insensitive)
Quiz limits	title ≤ 100; 1–50 questions; text ≤ 300; 2–6 options ≤ 100 chars
AI request limits	topic 3–100 chars; 1–15 questions; easy / medium / hard
AI reliability	2 attempts × (1 call + 2 retries at 1 s and 2 s)
AI rate limit (intended)	10 generations per user per hour
Close-room acknowledgement timeout	3 seconds

D. Index of terms

Every term defined in section 3, alphabetically. Click a term to jump to its definition.

Access token 68
ACID 63
Acknowledgement 30
Allowlist vs denylist 52
API (Application Programming Interface) 16
API key 75
async/await 44
Asynchronous code 43
Authentication 66
Authorization 67
Backend 16
Bearer token 70
Broadcast 29
Build 77
Bundling 78
Cascade delete 65
CDN (Content Delivery Network) 24
Cleanup function 35
Client 14
Client-side routing 40
Cold start (free-tier sleep) 83
Component 31
Conditional rendering 37
Context (React) 36
Continuous deployment 82
Controlled input 37
Controller 50
CORS (Cross-Origin Resource Sharing) 22
Custom hook 36
Database 58
Database function (RPC) 65
Defense in depth 74
Deployment 79
Destructuring 48
Early return 38
Emit 28
Enumeration attack 73
Environment variable 46
Event 28
Event loop 43
Exponential backoff 54
Express (with cors and dotenv) 86
Foreign key 60
Frontend 15
Google Gemini (@google/genai) 89
Grace period 56
Handshake 30
Health check route 57
Hook 33
Hosting 79
HTTP 17
HTTP methods (GET and POST) 20
HTTP status codes 21
HTTPS and TLS 17
Immutable update 39
In-memory state 55
Index (database) 62
JOIN 61
JSON 19
JSON Schema 76
JSX 32
JWT (JSON Web Token) 69
Key (in lists) 38
Large language model (LLM) 74
Load balancer and sticky sessions 56
localStorage 41
Long-polling 26
Long-running process 81
Map (JavaScript) 47
Mass assignment 72
Middleware 49
Migration (database) 64
Minification 78
Mixed content 24
Model alias vs pinned model 77
Module (import and export) 45
Monorepo 81
motion (animation library) 91

Node.js 86
npm package (dependency) 45
One-to-many relationship 61
Origin 18
Password hashing 67
Polling 23
Preflight request 23
Primary key 60
Production vs preview 82
Promise 44
Prompt 75
Prompt injection 73
Props 32
Pure function 47
qrcode.react (QR codes) 91
Race condition 55
Rate limiting 53
React 83
React Router 84
Real-time 25
Reconnection 31
Redis 57
Reduced motion 42
Refresh token 69
Relational database 58
Render 90
Request header 22
REST 20
Route 50
Row Level Security (RLS) 64
Runtime 42
Schema 52
Secret key vs publishable key 71
Sequence diagram 6
Server 15
Serverless function 80
Service layer 51
Session 68
Signature (digital) 70
Single-page application (SPA) 39
Singleton 46
Sliding window 53
snake_case vs camelCase 66
Socket room 29
Socket.IO 27
Spread operator 48
SQL 59
SQL injection 72
State (in React) 33
State machine 41
Static site 80
Structured output 76
Supabase and PostgreSQL 87
Supabase Auth 88
Table, row and column 59
Tailwind CSS 85
Transaction 63
Transient error 54
URL 18
URL parameter 40
useEffect 35
useState 34
UUID 62
Validation 51
Vercel 90
Vite 84
WebSocket 25
WSS (secure WebSocket) 26
XSS (Cross-Site Scripting) 71
Zod 88